



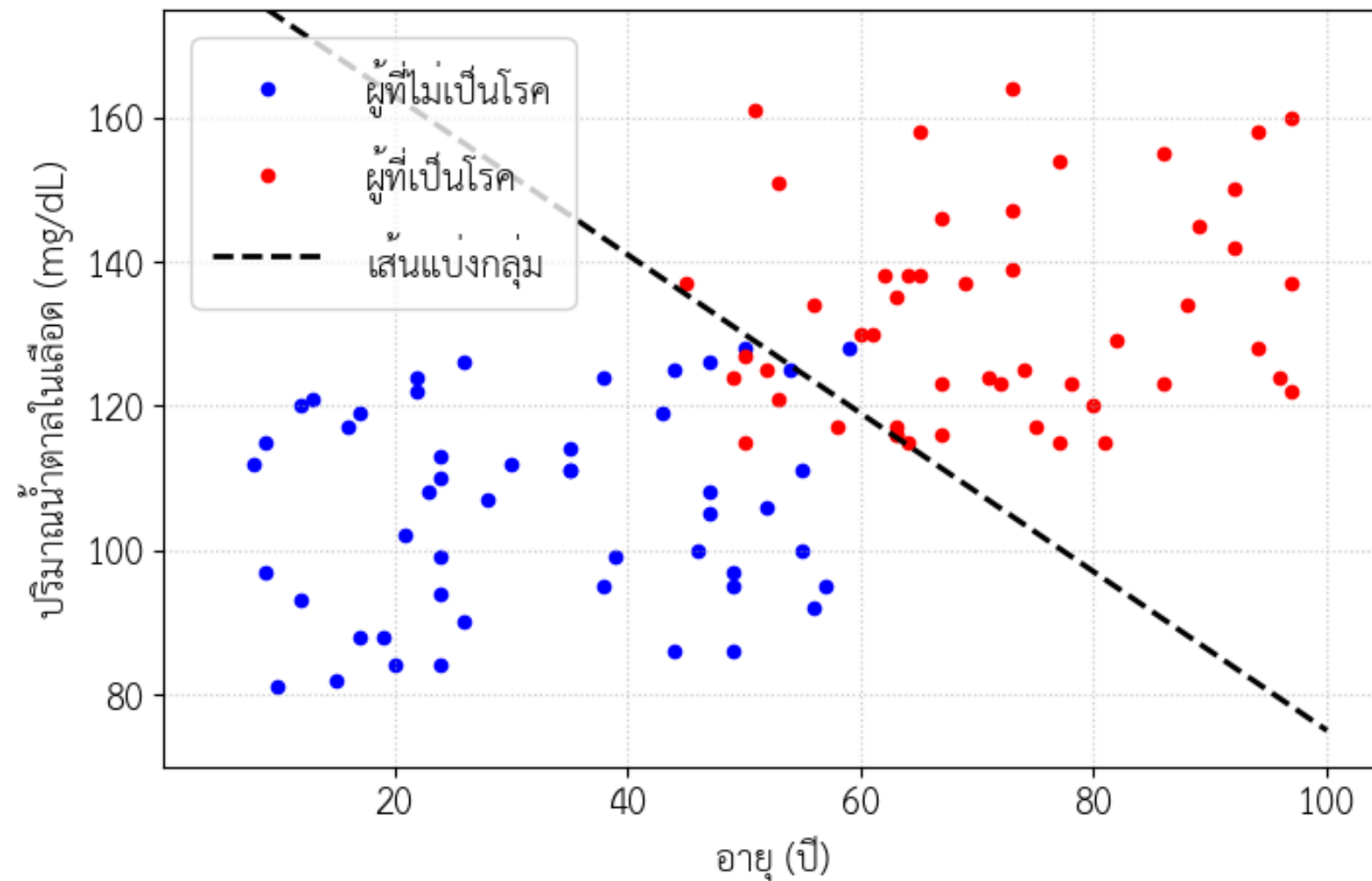
บทที่ 8

การจำแนกกลุ่ม

- ✓ 8.1 แนะนำการจำแนกกลุ่ม
- ✓ 8.2 K-Nearest Neighbours (KNN)
- ✓ 8.3 Naïve Bayes (NB)
- ✓ 8.4 Decision Tree (DT)
- ✓ 8.5 Artificial Neural Network (ANN)
- ✓ 8.6 Logistic Regression (LR)
- ✓ 8.7 Support Vector Machine (SVM)
- ✓ 8.8 แบบฝึกหัด

8.1 แนะนำการจำแนกกลุ่ม

- **การจำแนกกลุ่ม (Classification)** เป็นเทคนิคการทำนายกลุ่มข้อมูล (class) โดยเรียนรู้จากข้อมูลที่มีการระบุกลุ่มไว้แล้ว (labeled class)
- เป็นเทคนิคหนึ่งของ ML ในกลุ่ม Supervised Learning
- ตัวแปรตาม (Dependent/Target) มีลักษณะของเป็นตัวแปรเชิงกลุ่ม (Categorical)



ภาพที่ 8-1 : ตัวอย่างการจำแนกกลุ่มข้อมูล

- **การจำแนกกลุ่ม (Classification) แบ่งเป็น**

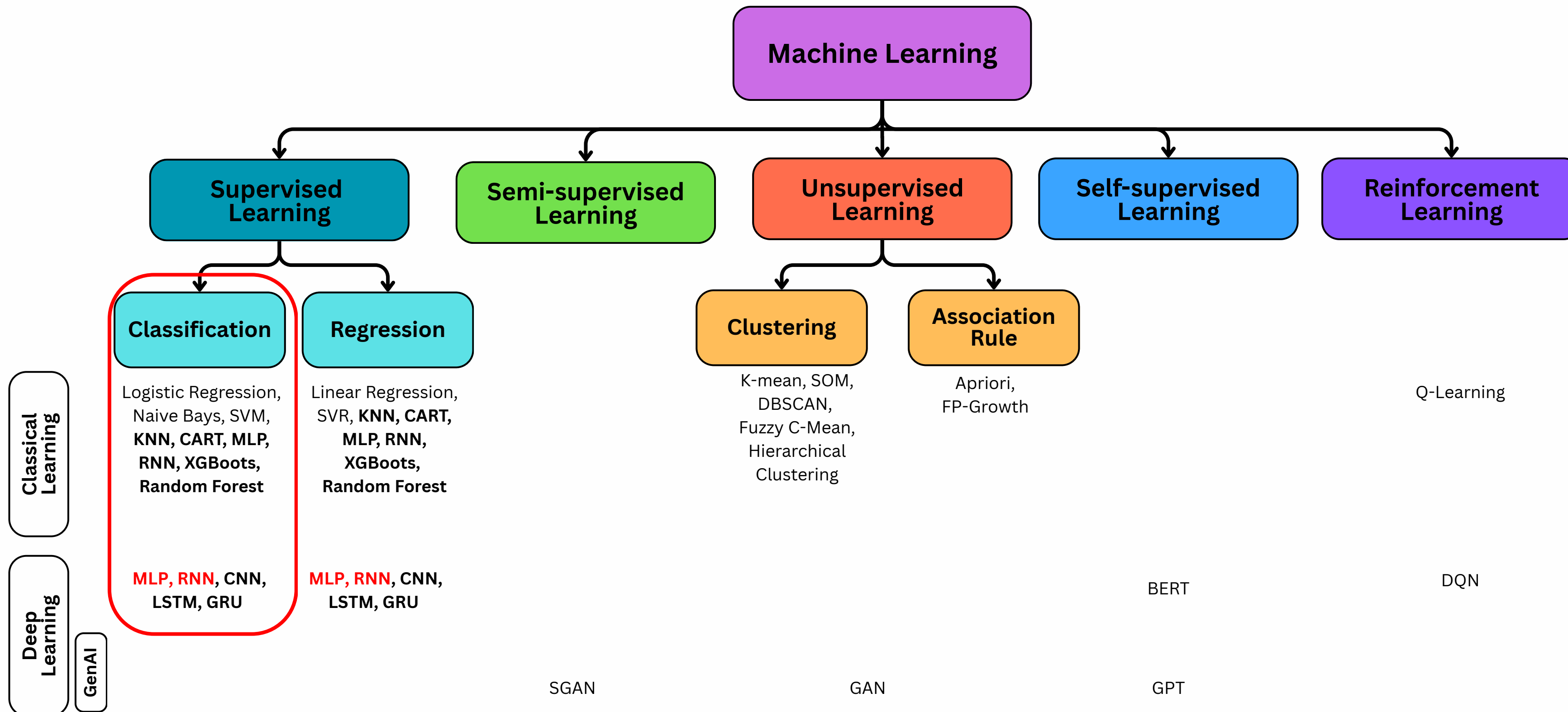
- Binary Classification : การจำแนกข้อมูลออกเป็น 2 กลุ่ม
- Multi-class Classification : การจำแนกข้อมูลออกเป็นมากกว่า 2 กลุ่ม

- **ตัวอย่างแบบ Binary Classification**

- การทำนายการเกิดโรค (เป็น/ไม่เป็น)
- การตรวจจับ Spam Mail (เป็น/ไม่เป็น)
- การทำนายโอกาสเกิดการล้มละลาย (ล้มละลาย/ไม่ล้มละลาย)
- การอนุมัติสินเชื่อของธนาคาร (อนุมัติ/ไม่อนุมัติ)

- **ตัวอย่างแบบ Multi-Classification**

- การทำนายอารมณ์ของพนักงานหรือลูกค้า (Sad/Neutral/Happy)
- การยืนยันตัวตนโดยการสแกนใบหน้า (John/Victoria/Adam/Robert)
- การทำนายผลฟุตบอล (แพ้/ชนะ/เสมอ)
- การแยกแยะวัตถุของรถยนต์ขับเคลื่อนอัตโนมัติ (รถยนต์/คน/สัตว์)



ภาพที่ 8-2 : Machine Learning Techniques

8.1.1 Model Validation

การทดสอบ (Validation) เป็นวิธีการที่ช่วยให้การประเมินประสิทธิภาพ (Efficiency) และประสิทธิผล (Effectiveness) ของโมเดล ที่สร้างขึ้นมีความน่าเชื่อถือมากขึ้น ประกอบด้วยเทคนิคต่าง ๆ ดังนี้

- Hold Out
- Repeat Random Sub-sampling
- Cross-Validation

8.1.1.1 Hold Out

- แบ่งข้อมูล Training/Test data เพียงครั้งเดียว
- เป็นวิธีที่สะดวก
- ประสิทธิภาพ/ประสิทธิผล มีความแปรปรวนสูง
- (การสุ่มอาจจะทำให้ได้ Training set และ Test set แต่ละครั้งต่างกัน)
- มีแค่บาง samples ถูกใช้ในการ Test
- ตัวอย่างการแบ่งแบบ Hold out เช่น
 - แบ่ง 70/30 (Training Data=70%, Test Data=30%)
 - แบ่ง 80/20 (Training Data=80%, Test Data=20%)

โค้ดไพทอน สำหรับ Hold Out

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 x = np.array(
4     [[1, 2],
5      [8, 4],
6      [9, 6],
7      [7, 8],
8      [4, 3]])
9 y = np.array([1, 7, 8, 4, 2])
10 x_train, x_test, y_train, y_test = train_test_split(x, y,
11     |         |         |         |         |         |         |         |
12     test_size=0.3, random_state=0)
13
14 print("x_train:\n", x_train)
15 print("\nx_test:\n", x_test)
16 print("\ny_train:\n", y_train)
17 print("\ny_test:\n", y_test)
```

x_train:	x_test:	y_train:	y_test:
[[8 4]	[[9 6]	[7 4 2]	[8 1]
[7 8]	[1 2]]		
[4 3]]			

Method: train_test_split

Parameter:

x คือ ข้อมูลตัวแปรต้นทุก records

y คือ ข้อมูลตัวแปรตามทุก records

test_size=0.3 คือ การสุ่มเอา Test data 30% ของทั้งหมด

random_state=0 คือ การสุ่มลำดับเดิมเสมอ ค่าอยู่ในช่วง [0 - 2³²]

shuffle=False คือ ไม่มีการสุ่ม ให้ใช้ลำดับเดิม

Output:

x_train คือ Training data ส่วนที่เป็นตัวแปรต้น

x_test คือ Test data ส่วนที่เป็นตัวแปรต้น

y_train คือ Training data ส่วนที่เป็นตัวแปรตาม

y_test คือ Test data ส่วนที่เป็นตัวแปรตาม

8.1.1.2 Repeat Random Sub-sampling

- เป็นวิธีการที่ปรับปรุงจาก Hold out
- สุ่มแบ่งข้อมูล Training/Test data หลายครั้ง แทนการแบ่งแค่ครั้งเดียว
- ประสิทธิภาพ/ประสิทธิผล แปรปรวนลดลงจาก Hold out
- ตัวอย่างการแบ่งแบบ Repeat random sub-sampling เช่น
 - แบ่ง 70/30 จำนวน 20 รอบ
 - แบ่ง 80/20 จำนวน 20 รอบ

โค้ดไพทอน สำหรับ Repeat Random Sub-sampling

```
1  import numpy as np
2  from sklearn.model_selection import ShuffleSplit
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 ss = ShuffleSplit(n_splits=20,
11                  test_size=0.3, random_state=0).split(x)
12 for i, (train_index, test_index) in enumerate(ss):
13     print(f"Round {i+1}:")
14     print(f"    Train index={train_index}")
15     print(f"    Test index={test_index}")
16     x_train, x_test = x[train_index], x[test_index]
17     y_train, y_test = y[train_index], y[test_index]
```

Method: ShuffleSplit

Parameter:

n_splits=20 คือ การสุ่มเอาข้อมูล จำนวน 20 ชุด
test_size=0.3 คือ การสุ่มเอา Test data 30% ของทั้งหมด
random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ
ถ้าต้องการให้ สุ่มใหม่ทุกครั้งให้กำหนดเป็น "None"
x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Round 1:

Train index=[1 3 4]

Test index=[2 0]

Round 2:

Train index=[1 4 3]

Test index=[0 2]

⋮

Round 20:

Train index=[1 2 0]

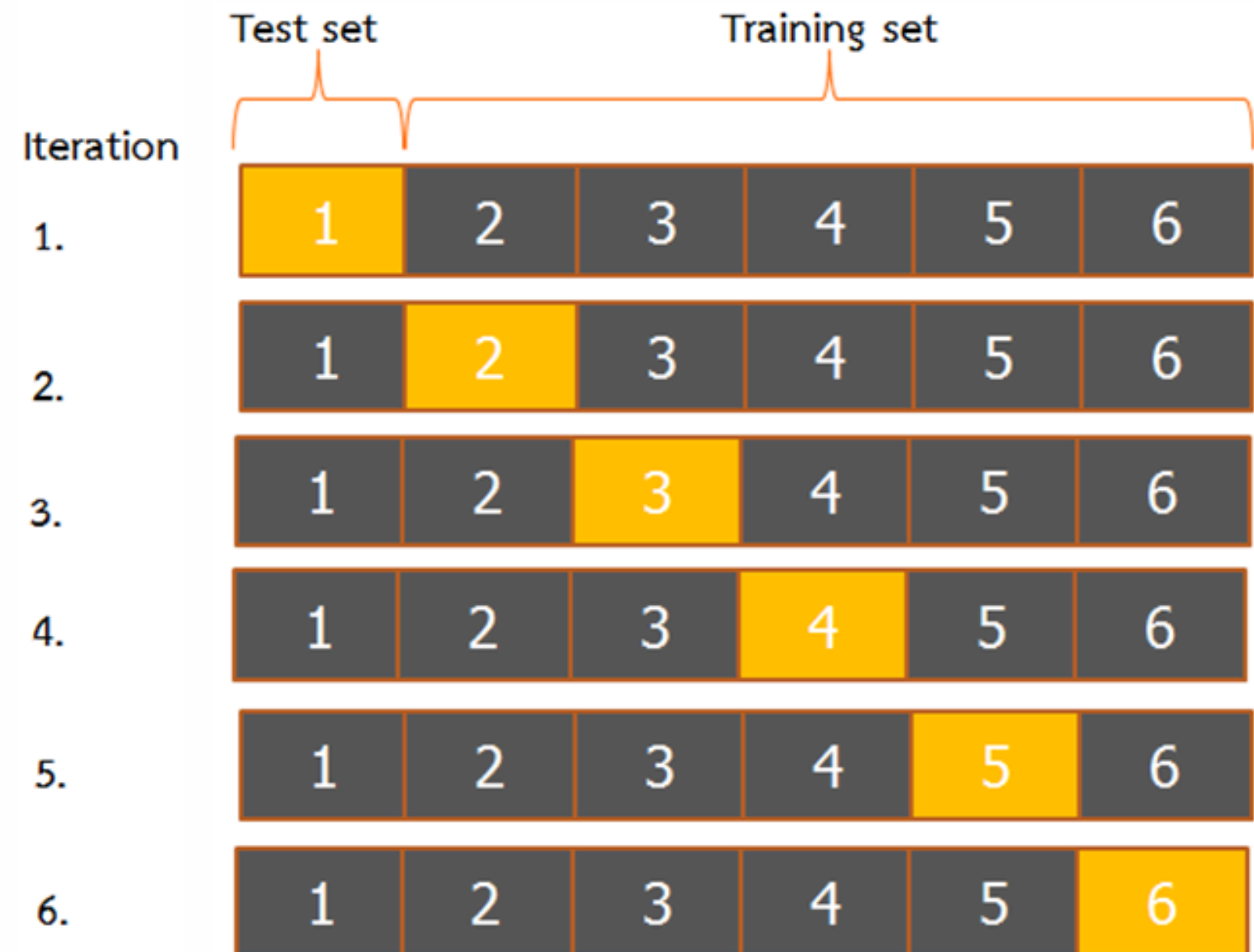
Test index=[3 4]

8.1.1.3 Cross-Validation

- เป็นวิธีการที่ปรับปรุงจาก Hold out และ Repeat random sub-sampling
- แบ่งข้อมูลครั้งเดียวแต่แบ่งออกเป็นหลายกอง (Fold) แต่ละกองจะมีข้อมูลไม่ซ้ำกับข้อมูลกองอื่น
- ประสิทธิภาพ/ประสิทธิผล ประปรวนลดลงจาก Repeat random sub-sampling
- ตัวอย่างการแบ่งแบบ Cross-validation เช่น
 - k -fold Cross Validation
 - 3-fold Cross Validation แบ่งข้อมูลออกเป็น 3 กอง
 - 6-fold Cross Validation แบ่งข้อมูลออกเป็น 6 กอง
 - 10-fold Cross Validation แบ่งข้อมูลออกเป็น 10 กอง
 - Leave-One-Out Cross Validation (LOOCV) หรือ n -fold Cross Validation แบ่งข้อมูลออกเป็น n กอง กองละ 1 record

ขั้นตอนการแบ่งข้อมูลด้วย k -fold cross validation

- สุ่มแยกข้อมูลจาก dataset ออกเป็น k กอง (ในที่นี้กำหนด $k=6$)
- รอบแรกใช้กองที่ 1 เป็นข้อมูลสำหรับ Test set แล้วนำ 5 กองที่เหลือมารวมกันเพื่อใช้เป็น Training test
- รอบสองใช้กองที่ 2 เป็น test set แล้วใช้ 5 กองที่เหลือรวมเป็น Training set
- ทำเช่นนี้ไปจนครบ 6 รอบ
- ในแต่ละรอบจะใช้ข้อมูล Test set เพื่อวัดประสิทธิภาพของโมเดล ด้วย Evaluation Metrics เช่น MAPE, Accuracy, Precision, Recall แล้วหาค่าเฉลี่ยจากทั้ง 6 รอบ เพื่อกำหนดเป็นประสิทธิภาพของโมเดล



ภาพที่ 8-3 : 6-fold cross validation

โค้ดไพทอน สำหรับ 3-fold cross validation

```
1 import numpy as np
2 from sklearn.model_selection import StratifiedKFold
3 x = np.array(
4     [[1, 2],
5      [8, 4],
6      [9, 6],
7      [7, 8],
8      [4, 3]])
9 y = np.array([1, 1, 0, 0, 1])
10 ss = StratifiedKFold(n_splits=3, shuffle=True, random_state=0).split(x, y)
11 for i, (train_index, test_index) in enumerate(ss):
12     print(f"Fold {i+1}:")
13     print(f"  Train index={train_index}")
14     print(f"  Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: StratifiedKFold

Parameter:

n_splits=3 คือ จำนวนกองของข้อมูลที่ต้องการแบ่ง

shuffle=True คือ กำหนดให้มีการสุ่มข้อมูล

random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ

x คือ ข้อมูลตัวแปรต้นทุก records

y คือ ข้อมูลตัวแปรตามทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[0 1 3]

Test index=[2 4]

Fold 2:

Train index=[0 2 4]

Test index=[1 3]

Fold 3:

Train index=[1 2 3 4]

Test index=[0]

โค้ดไพทอน สำหรับ LOOCV

```
1  import numpy as np
2  from sklearn.model_selection import LeaveOneOut
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 loo = LeaveOneOut().split(x)
11 for i, (train_index, test_index) in enumerate(loo):
12     print(f"Fold {i+1}:")
13     print(f"    Train index={train_index}")
14     print(f"    Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: LeaveOneOut

Parameter:

x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[1 2 3 4]

Test index=[0]

Fold 2:

Train index=[0 2 3 4]

Test index=[1]

Fold 3:

Train index=[0 1 3 4]

Test index=[2]

Fold 4:

Train index=[0 1 2 4]

Test index=[3]

Fold 5:

Train index=[0 1 2 3]

Test index=[4]

8.1.2 Performance Evaluation for Classification

- **Confusion Matrix**

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

TP = True Positive (ทำนายว่าจริง ค่าเป็นจริง)

TN = True Negative (ทำนายว่าเท็จ ค่าเป็นเท็จ)

FP = False Positive (ทำนายว่าจริง ค่าเป็นเท็จ)

FN = False Negative (ทำนายว่าเท็จ ค่าเป็นจริง)

- **Evaluation Metrics**

- $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- $\text{Precision} = TP / (TP + FP)$
- $\text{Recall/Sensitivity} = TP / (TP + FN)$
- $\text{Specificity} = TN / (TN + FP)$
- $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- **ตัวอย่างการคำนวณ สำหรับ Performance Evaluation for Classification**

จงคำนวณหาค่า Accuracy, Precision, Recall, Specificity, และ F1-score เมื่อค่าที่เกิดขึ้นจริง (y_{test}) และค่าที่ทำนายได้ (y_{pred}) เป็นดังนี้

$y_{test} = [1, 1, 0, 0, 1]$ #ค่าที่เกิดขึ้นจริง

$y_{pred} = [1, 1, 0, 1, 1]$ #ค่าที่ทำนายได้

วิธีทำ

- **คำนวณ Confusion Matrix**

ลำดับที่ข้อมูล	ค่าที่ทำนายได้	ค่าที่เกิดขึ้นจริง	TP	TN	FP	FN
1	1	1	1			
2	1	1	1			
3	0	0		1		
4	1	0			1	
5	1	1	1			
รวม			3	1	1	0

		Predicted Value	
		0	1
Actual Value	0	1	1
	1	0	3

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

- **คำนวณ Evaluation Metrics**

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) = (3 + 1) / (3 + 1 + 1 + 0) = 4 / 5 = 0.8$$

$$\text{Precision} = TP / (TP + FP) = 3 / (3 + 1) = 3 / 4 = 0.75$$

$$\text{Recall/Sensitivity} = TP / (TP + FN) = 3 / (3 + 0) = 3 / 3 = 1.00$$

$$\text{Specificity} = TN / (TN + FP) = 1 / (1 + 1) = 1 / 2 = 0.5$$

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 \times (0.75 \times 1) / (0.75 + 1) = 0.86$$

- โค้ดไพทอน สำหรับ Performance Evaluation for Classification

```

1  from sklearn.metrics import confusion_matrix
2  from sklearn.metrics import classification_report
3
4  y_test=[1, 1, 0, 0, 1] #ค่าที่เกิดขึ้นจริง
5  y_pred=[1, 1, 0, 1, 1] #ค่าที่ทำนายได้จากโมเดล
6
7  print('Confustion matrix: ')
8  print(confusion_matrix(y_test, y_pred))
9
10 print('\nEvaluation Metrics: ')
11 print(classification_report(y_test,y_pred))

```

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

Confustion matrix:

```
[[1 1]
 [0 3]]
```

Evaluation Metrics:

	precision	recall	f1-score	support
0	1.00	0.50	0.67	2
1	0.75	1.00	0.86	3
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5

8.1.3 Distance Measurement

การวัดระยะทาง (Distance Measurement) คือ การหาความแตกต่าง หรือ ความเหมือนระหว่างจุด 2 จุด ใด ๆ ถ้าระยะทางสั้นแสดงว่ามีความแตกต่างกันน้อย แต่มีความเหมือนมาก ถ้าระยะทางยาวแสดงว่ามีความแตกต่างกันมาก แต่มีความเหมือนน้อย ตัวอย่างเทคนิคการวัดระยะทาง มีดังนี้

- Euclidean Distance
- Manhattan / City-Block Distance
- Minkowski Distance
- Mahalanobis Distance
- Cosine Distance
- Haversine Distance

8.1.3.1 Euclidean Distance

สูตรการคำนวณ

$$d = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

เมื่อ

d คือ ระยะทางระหว่างจุด

x_i คือ ค่าในมิติที่ i ของจุด x

y_i คือ ค่าในมิติที่ i ของจุด y

m คือ จำนวนมิติ

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1, 2) และจุด (4, 6) ด้วย Euclidean Distance

วิธีทำ

$$= \sqrt{(1 - 4)^2 + (2 - 6)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

8.1.3.2 Manhattan Distance

สูตรการคำนวณ

$$d = \sum_{i=1}^m |x_i - y_i|$$

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1, 2) และจุด (4, 6) ด้วย Manhattan Distance

วิธีทำ

$$= |1 - 4| + |2 - 6| = 3 + 4 = 7$$

8.1.3.3 Minkowski Distance

สูตรการคำนวณ

$$d = \left(\sum_{i=1}^m |x_i - y_i|^p \right)^{1/p}$$

เมื่อ

p คือ พารามิเตอร์ที่กำหนดรูปแบบ (ถ้า $p = 1$ คือ Manhattan, ถ้า $p = 2$ คือ Euclidean)

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1,2) และจุด (4, 6) ด้วย Minkowski Distance เมื่อ $p = 3$

วิธีทำ

$$= (|1 - 4|^3 + |2 - 6|^3)^{1/3} = (27 + 64)^{1/3} = \sqrt[3]{91} \approx 4.5$$

8.1.3.4 Cosine Distance

สูตรการคำนวณ

$$d = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

เมื่อ

d คือ ระยะทาง (ทิศทาง) ระหว่างเวกเตอร์ $\mathbf{x}$ และ $\mathbf{y}$

$\mathbf{x} \cdot \mathbf{y}$ คือ Dot product ระหว่างเวกเตอร์ $\mathbf{x}$ และ $\mathbf{y}$

$\|\mathbf{x}\|$ คือ ขนาด ของเวกเตอร์ $\mathbf{x}$

$\|\mathbf{y}\|$ คือ ขนาด ของเวกเตอร์ $\mathbf{y}$

Cosine Distance	Cosine Similarity	มุม	ความหมาย
0	1	0°	เหมือนกันมาก
1	0	90°	ไม่สัมพันธ์
2	-1	180°	ตรงข้าม

ตัวอย่างการคำนวณ

จงหาระยะทางระหว่างจุด (1, 2) และจุด (4, 6) ด้วย Cosine Distance

วิธีทำ

$$\mathbf{x} \cdot \mathbf{y} = (1 \times 4) + (2 \times 6) = 16$$

$$\|x\| = \sqrt{1^2 + 2^2} = \sqrt{5}, \quad \|y\| = \sqrt{4^2 + 6^2} = \sqrt{52}$$

$$d = 1 - \frac{16}{\sqrt{5}\sqrt{52}} = 1 - 0.992 = 0.008$$

8.1.3.5 Mahalanobis Distance

สูตรการคำนวณ

$$d = \sqrt{(\mathbf{x} - \mathbf{y})^T \mathbf{S}^{-1} (\mathbf{x} - \mathbf{y})}$$

เมื่อ

$\mathbf{S}$ คือ Covariance matrix

$\mathbf{S}^{-1}$ คือ Inverse Covariance matrix

T คือ Transpose

8.1.3.6 Haversine Distance

สูตรการคำนวณ

$$d = 2r \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

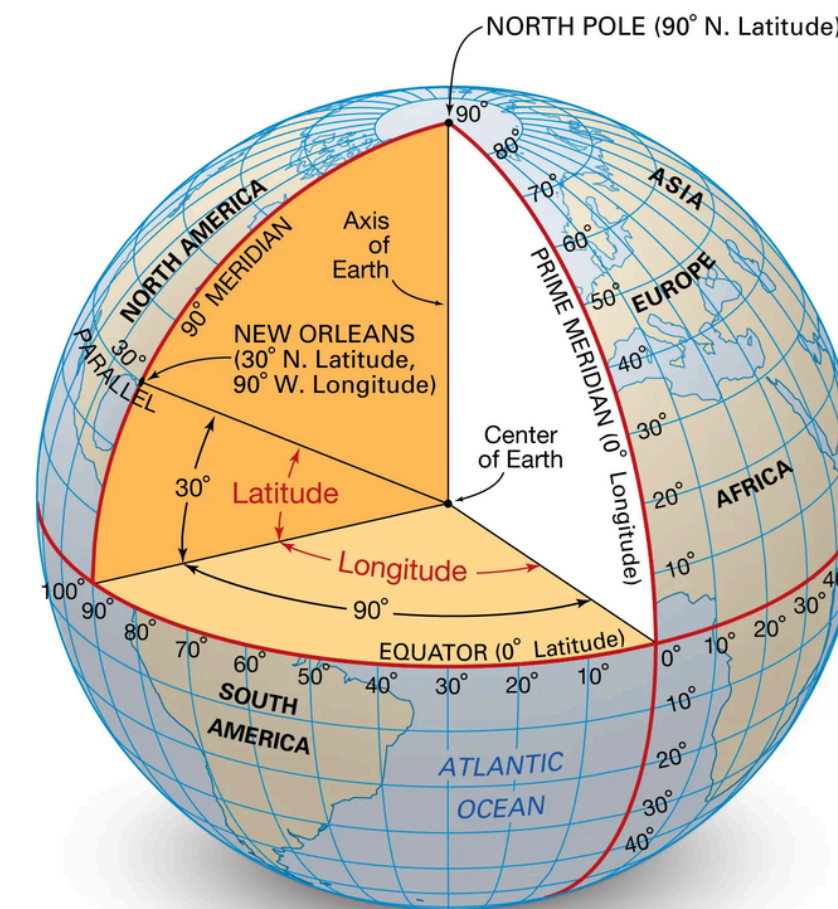
เมื่อ

d คือ ระยะทางระหว่างจุดสองจุดบนผิวทรงกลม

r คือ รัศมีทรงกลม

ϕ_1, ϕ_2 คือ ละติจูดของจุดที่ 1 และ 2 ซึ่งต้องแปลงเป็น Radian ด้วยการคูณ $\pi/180$ ก่อน

λ_1, λ_2 คือ ลองจิจูดของจุดที่ 1 และ 2 ซึ่งต้องแปลงเป็น Radian ด้วยการคูณ $\pi/180$ ก่อน



© Encyclopædia Britannica, Inc.

ตัวอย่างการคำนวณ

จงหาระยะทางจาก กรุงเทพฯ ($13.75^\circ N, 100.52^\circ E$) ไปยัง โตเกียว ($35.68^\circ N, 139.69^\circ E$)

รัศมีของโลกเฉลี่ยประมาณ 6,371 กิโลเมตร

วิธีทำ

$$d = 2r \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

- แปลงเป็น Radian

กรุงเทพฯ : $\phi_1 \approx 0.240, \lambda_1 \approx 1.754$

โตเกียว : $\phi_2 \approx 0.622, \lambda_2 \approx 2.438$

- คำนวณแต่ละส่วน

$$\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) = \sin^2 \left(\frac{0.622 - 0.240}{2} \right) = \sin^2(0.191) = 0.036$$

$$\sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right) = \sin^2 \left(\frac{2.438 - 1.754}{2} \right) = \sin^2(0.342) = 0.113$$

$$\cos(\phi_1) = \cos(0.240) = 0.971$$

$$\cos(\phi_2) = \cos(0.622) = 0.813$$

- คำนวณรวม

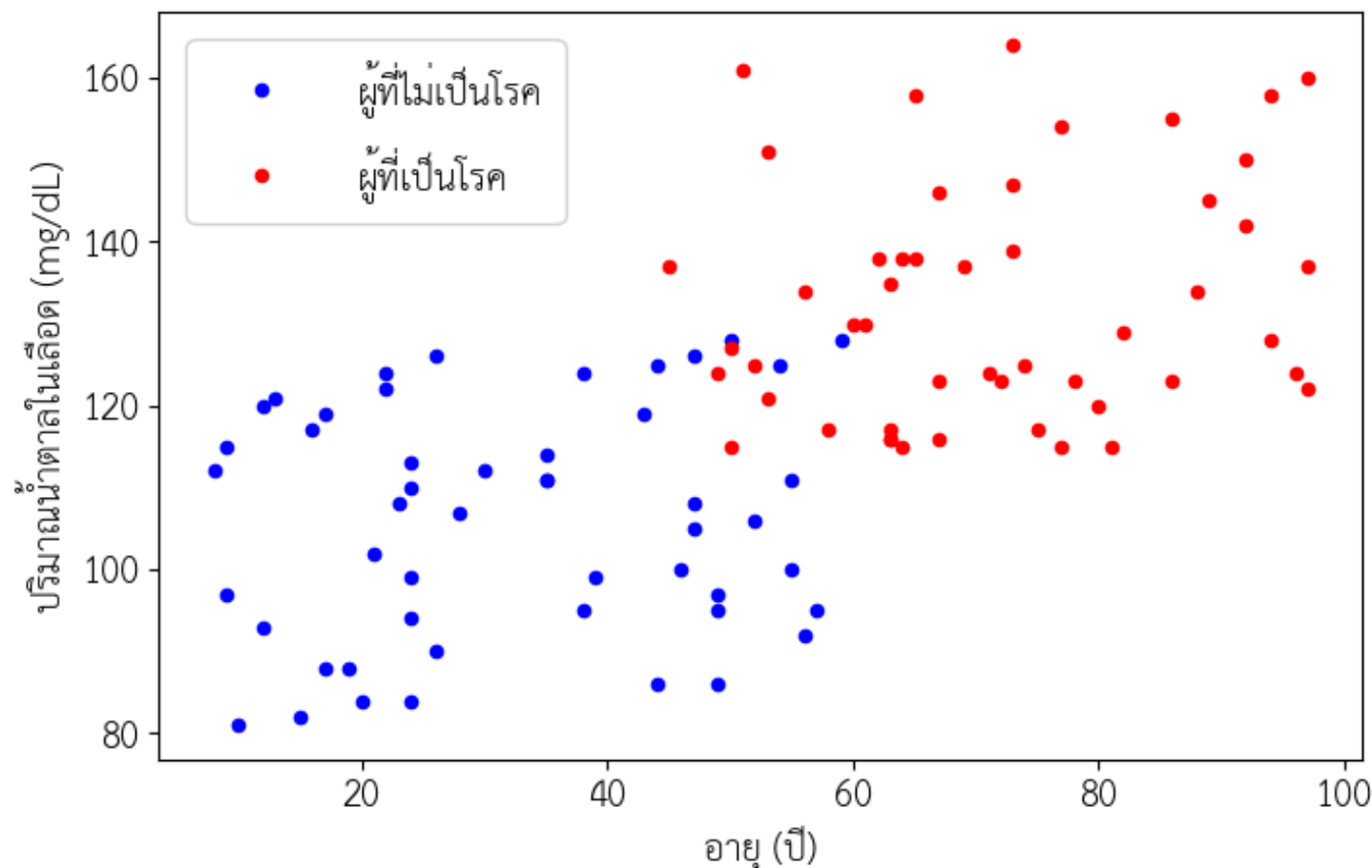
$$d = 2 \times 6,731 \times \arcsin \left(\sqrt{0.036 + 0.971 \times 0.813 \times 0.113} \right)$$

$$= 2 \times 6,731 \times \arcsin (0.3536)$$

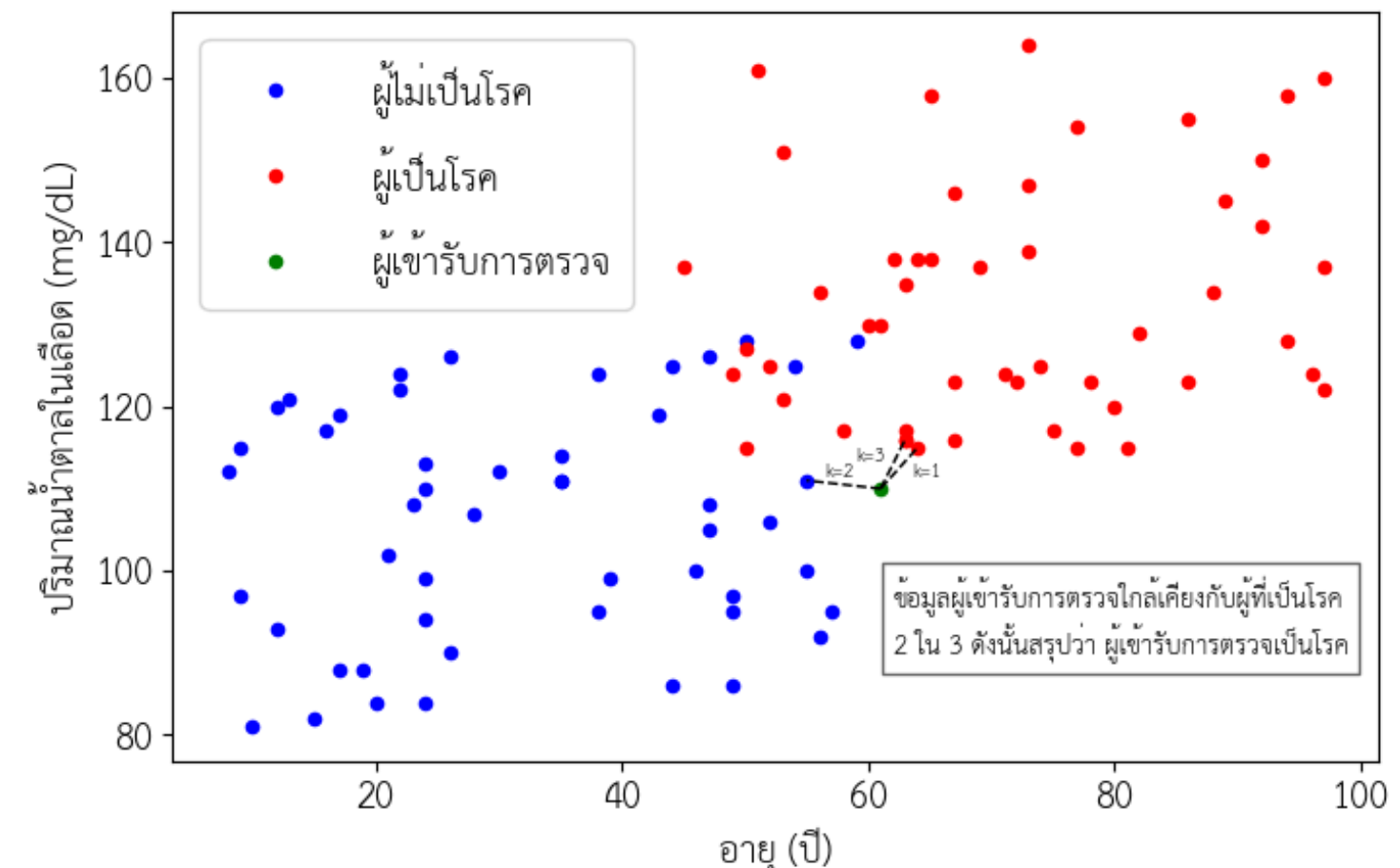
$$= 2 \times 6,731 \times 0.361 = 4859.782 \text{ กิโลเมตร}$$

8.2 K-Nearest Neighbours (KNN)

- อาศัยหลักการวัดระยะทาง ซึ่งนิยมวัดระยะทางโดยใช้วิธี Euclidian Distance
- ง่ายต่อการทำความเข้าใจและการคำนวณ
- Sensitive ต่อ Outliers ทำให้ผลการทำนายผิดพลาด ควสอบ outliers ก่อนหรือทำนายโดย weight ตามระยะทาง
- ข้อมูลที่มีช่วงต่างกันมาก เช่น อายุ [0-100] กับเงินเดือน [1 พัน, 1 แสน] จะทำให้ข้อมูลที่มีช่วงกว้างกว่า มีอิทธิพลต่อการคำนวณระยะทางมากกว่า ควรทำการ Normalize/Standardize ข้อมูล เช่น ทำให้ข้อมูลอยู่ในช่วง [0, 1]
- หากข้อมูลมีขนาดใหญ่ จะทำให้การทำนายใช้เวลานานมากเพราะต้องคำนวณทุก Training Points



ภาพที่ 8-4ก : ชุดข้อมูลฝึก (Training Set)



ภาพที่ 8-4ข : การทดสอบ (Test)

ภาพที่ 8-4 : ตัวอย่างการจำแนกกลุ่มข้อมูลด้วย KNN

8.2.1 การคำนวณสำหรับ KNN

สูตรการคำนวณ

ขั้นที่ 1 : คำนวณระยะทาง (ความแตกต่าง) ระหว่าง test point x กับ ทั้งหมดของ training point x^n

$$d^n = d(x, x^n), \quad n = 1, \dots, N$$

เมื่อ $d(x, x^n)$ **คือ** ระยะทางระหว่าง test point x กับ training point x^n

N **คือ** จำนวน training point ทั้งหมด

หากใช้วิธีวัดระยะทางแบบยุคลิด (Euclidean Distance) จะได้

$$d(x, x^n) = \sqrt{\sum_{i \in m} (x_i - x_i^n)^2}$$

เมื่อ m คือ จำนวนตัวแปรต้น, จำนวนปัจจัย (Factor), หรือจำนวนคุณลักษณะ (Feature)

ขั้นที่ 2 : หา training point x^n ซึ่งมีระยะทาง $d(x, x^n)$ ใกล้เคียงกับ test point x มากที่สุด จำนวน k อันดับแรก

ขั้นที่ 3 : กำหนดผลลัพธ์ (Output) สำหรับงาน Classification (จำแนกกลุ่ม) และ Regression (ทำนาย)

- **Classification :** กำหนด class label (y) โดยพิจารณาค่า Mode ของ class จาก k training points
- **Regression :** กำหนด predicted value (y) โดยพิจารณาค่า Average ของ class จาก k training points

8.2.2 ตัวอย่างการคำนวณ KNN

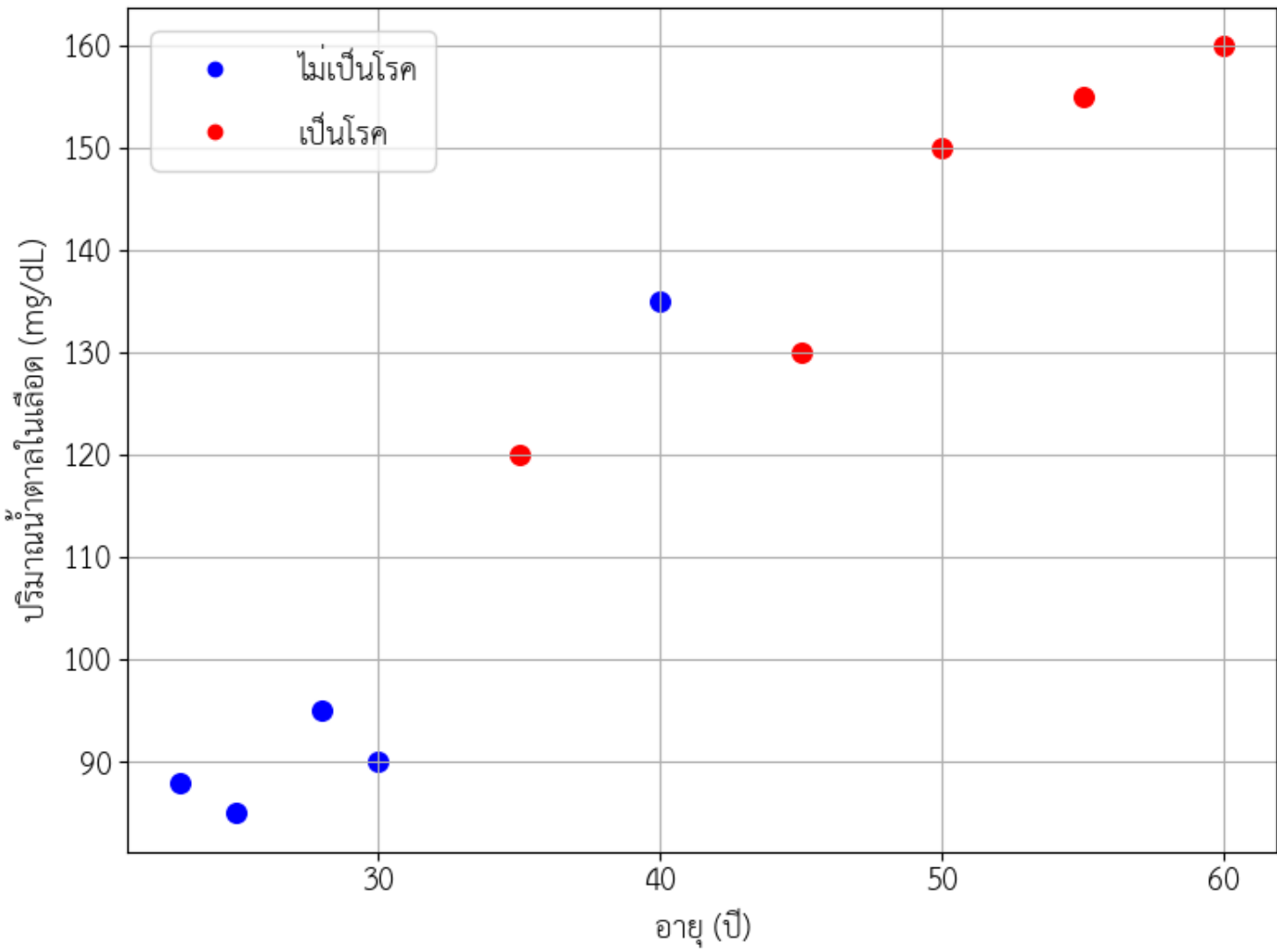
จงแสดงวิธีการคำนวณ KNN เพื่อวินิจฉัยการป่วยเป็นโรคเบาหวาน

ตารางที่ 8-1ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

ตารางที่ 8-1ข : ชุดข้อมูลทดสอบ (Test Set)

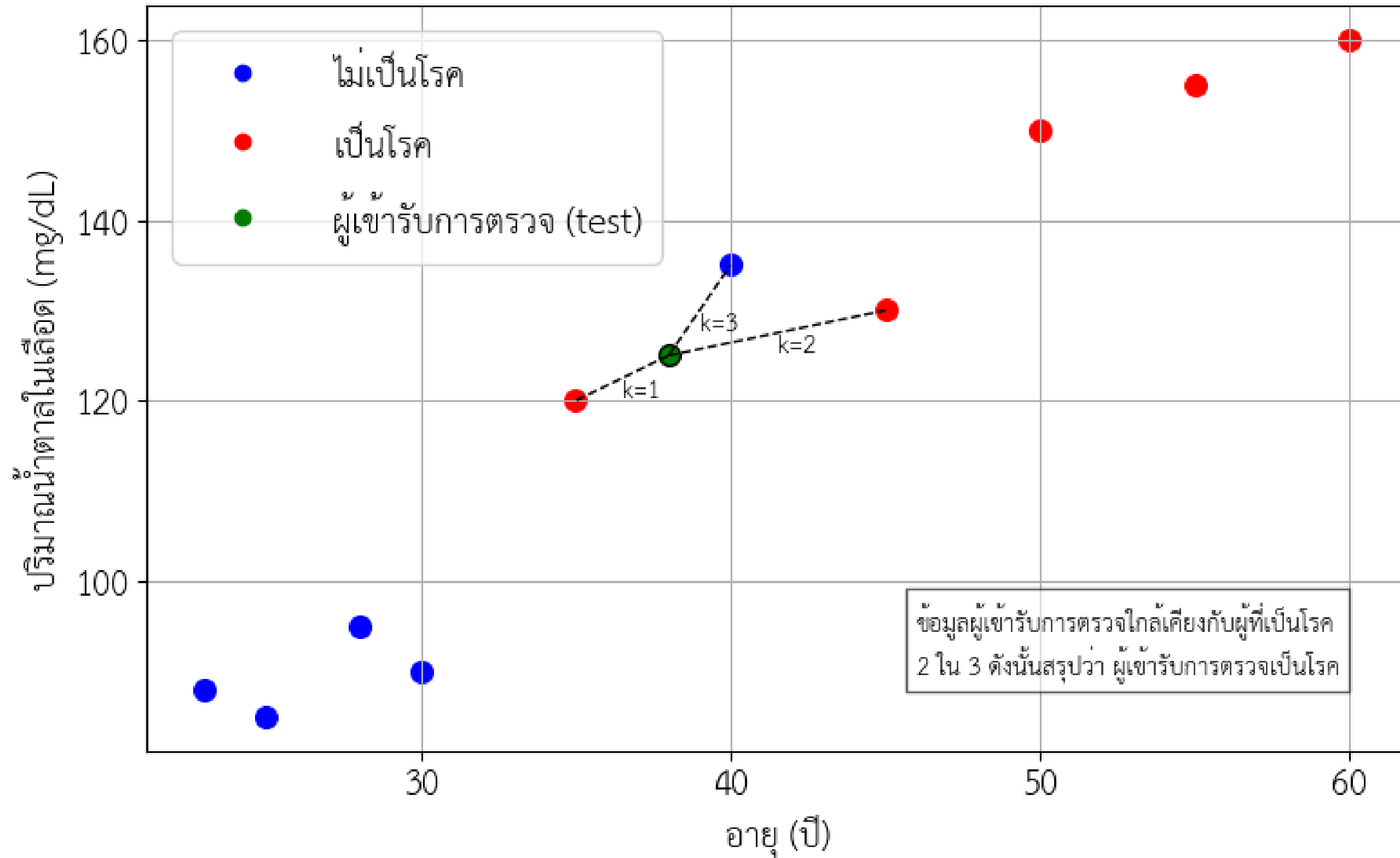
รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



ภาพที่ 8-5 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ชุดข้อมูลทดสอบ (Test Set)						
	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)	(คำนวณนิยามของคำตอบจากข้อมูลฝึก 3 อันดับแรก)		
	38	125	เป็นโรค.....			
ชุดข้อมูลฝึก (Traning Set)						
รายการที่	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)	การคำนวณระยะทาง ด้วยวิธีแบบยุคลิด	ระยะทาง	รายการข้อมูลฝึก ที่ใกล้ที่สุด 3 อันดับ
1	25	85	ไม่เป็นโรค	$\text{SQRT}((38-25)^2 + (125-85)^2)$	42.06	
2	30	90	ไม่เป็นโรค	$\text{SQRT}((38-30)^2 + (125-90)^2)$	35.9	
3	45	130	เป็นโรค	$\text{SQRT}((38-45)^2 + (125-130)^2)$	8.6	x
4	35	120	เป็นโรค	$\text{SQRT}((38-35)^2 + (120-120)^2)$	5.83	x
5	50	150	เป็นโรค	$\text{SQRT}((38-50)^2 + (125-150)^2)$	27.73	
6	23	88	ไม่เป็นโรค	$\text{SQRT}((38-23)^2 + (125-88)^2)$	39.92	
7	60	160	เป็นโรค	$\text{SQRT}((38-60)^2 + (125-160)^2)$	41.34	
8	55	155	เป็นโรค	$\text{SQRT}((38-55)^2 + (125-155)^2)$	34.48	
9	40	135	ไม่เป็นโรค	$\text{SQRT}((38-40)^2 + (125-135)^2)$	10.2	x
10	28	95	ไม่เป็นโรค	$\text{SQRT}((38-28)^2 + (125-95)^2)$	31.62	



ภาพที่ 8-6 : การทดสอบ (Test) หรือการวินิจฉัย

8.2.3 ตัวอย่างโค้ด KNN

- เขียนโค้ดแบบไม่ใช่ไลบรารี ML

```
1 #Import Python library
2 import math
3 from scipy import stats
4
5 #กำหนดค่าเริ่มต้น
6 x_train = [[25, 85], [30, 90], [45, 130], [35, 120], [50, 150], [23, 88], [60, 160], [55, 155], [40, 135], [28, 95]]
7 y_train = [0, 0, 1, 1, 1, 0, 1, 1, 0, 0]
8 x_test = [38, 125]
9 k = 3
10
11
12 #ขั้นที่ 1 . คำนวณระยะทางระหว่าง test point กับ training points
13 dist = []
14 for i in range(len(x_train)):
15     d = math.sqrt( pow(x_test[0] - x_train[i][0], 2) +
16                   pow(x_test[1] - x_train[i][1], 2) )
17     dist.append(round(d,2))
18
19 print('ขั้นที่ 1   ระยะทาง คือ', dist)
20
```

```

21 #ขั้นที่ 2 : หา training points ซึ่งมีระยะทางใกล้เคียงกับ test point มากที่สุด จำนวน k อันดับแรก
22 sort_dist = sorted(dist.copy())[:k]
23
24 index = []
25 for s in sort_dist:
26     for i, value in enumerate(dist):
27         if s == value:
28             index.append(i)
29
30     if len(index) == k:
31         break;
32
33 print('ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ', [i + 1 for i in index])
34
35 #ขั้นที่ 3 : หาคำตอบจากตัวแปรตามของ training points ที่ใกล้ที่สุด k ตัว
36 #หาค่า mode สำหรับงาน classification
37 y_pred = [y_train[i] for i in index]
38 mode = int(stats.mode(y_pred)[0])
39
40 print('ขั้นที่ 3 : ผลการวินิจฉัย คือ', "เป็นโรค" if mode else "ไม่เป็นโรค")

```

ขั้นที่ 1 : ระยะทาง คือ [42.06, 35.9, 8.6, 5.83, 27.73, 39.92, 41.34, 34.48, 10.2, 31.62]
 ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ [4, 3, 9]
 ขั้นที่ 3 : ผลการวินิจฉัย คือ เป็นโรค

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn เพื่อคำนวณระยะทาง (ขั้นที่ 1) และหา k อันดับแรก (ขั้นที่ 2)

```
1 #Import Python library
2 from sklearn.neighbors import NearestNeighbors
3 from scipy import stats
4
5 #กำหนดค่าเริ่มต้น
6 x_train = [[25, 85], [30, 90], [45, 130], [35, 120], [50, 150], [23, 88], [60, 160], [55, 155], [40, 135], [28, 95]]
7 y_train = [0, 0, 1, 1, 1, 0, 1, 1, 0, 0]
8 x_test = [38, 125]
9 k = 3
10
11 #ขั้นที่ 1   คำนวณระยะทาง และ ขั้นที่ 2 : หา training points k อันดับแรก
12 net = NearestNeighbors(n_neighbors = k, metric = 'minkowski', p = 2)
13 net.fit(x_train,y_train)
14 dist, index = net.kneighbors([x_test])
15 index = index[0]
16
17 print('ขั้นที่ 1 : ระยะทาง 3 อันดับแรก คือ', dist)
18 print('ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ', [int(i + 1) for i in index])
19
20 #ขั้นที่ 3. หาคำตอบ จากคำตอบหรือตัวแปรตามของ training points ที่ใกล้ที่สุด k ตัว
21 y_pred = [y_train[i] for i in index]
22 mode = int(stats.mode(y_pred)[0])
23
24 print('ขั้นที่ 3 : ผลการวินิจฉัย คือ', "เป็นโรค" if mode else "ไม่เป็นโรค")
```

ขั้นที่ 1 : ระยะทาง 3 อันดับแรก คือ [[5.83095189 8.60232527 10.19803903]]

ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ [4, 3, 9]

ขั้นที่ 3 : ผลการวินิจฉัย คือ เป็นโรค

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python library
2 import pandas as pd
3 from sklearn.neighbors import KNeighborsClassifier
4
5 #Load Data
6 x = pd.read_csv('data/diabetes.csv')
7 x_train = x[['age','blood_glucose']].to_numpy()
8 y_train = x['diabetes']
9 x_test = [38, 125]
10 k = 3
11
12 #Build and Train Model
13 net = KNeighborsClassifier(n_neighbors = k, metric = 'minkowski', p = 2)
14 net.fit(x_train,y_train)
15
16 #Test Model
17 y_pred=net.predict([x_test])
18 print('ผลการวินิจฉัย คือ ' y_pred[0])
```

ผลการวินิจฉัย คือ เป็นโรค

8.3 Naïve Bayes (NB)

- อาศัยแนวคิดของความน่าจะเป็น [ทฤษฎีเบย์]
- ไม่ซับซ้อนสามารถเข้าใจได้ง่าย
- ไม่ Sensitive ต่อ Outliers
- เทคนิค Naïve Bayes มีดังนี้
 - Categorical Naive Bayes : ใช้สำหรับตัวแปรต้นแบบ Categorical
 - Gaussian Naive Bayes : ใช้สำหรับตัวแปรต้น แบบ Numerical

8.3.1 การคำนวณสำหรับ Naïve Bayes

สูตรการคำนวณ

$$\hat{y} = \arg \max_{k=0} P(C_k) \prod_{i=1}^m P(x_i | C_k)$$

เมื่อ

$\hat{y}$ คือ class label ที่ทำนายได้

m คือ จำนวนตัวแปรต้น

C_k คือ class k ในตัวแปรตาม

x_i คือ ข้อมูลตัวแปร i

$P(C_k)$ คือ Prior Probability หรือ ความน่าจะเป็นที่จะเกิดคลาส k

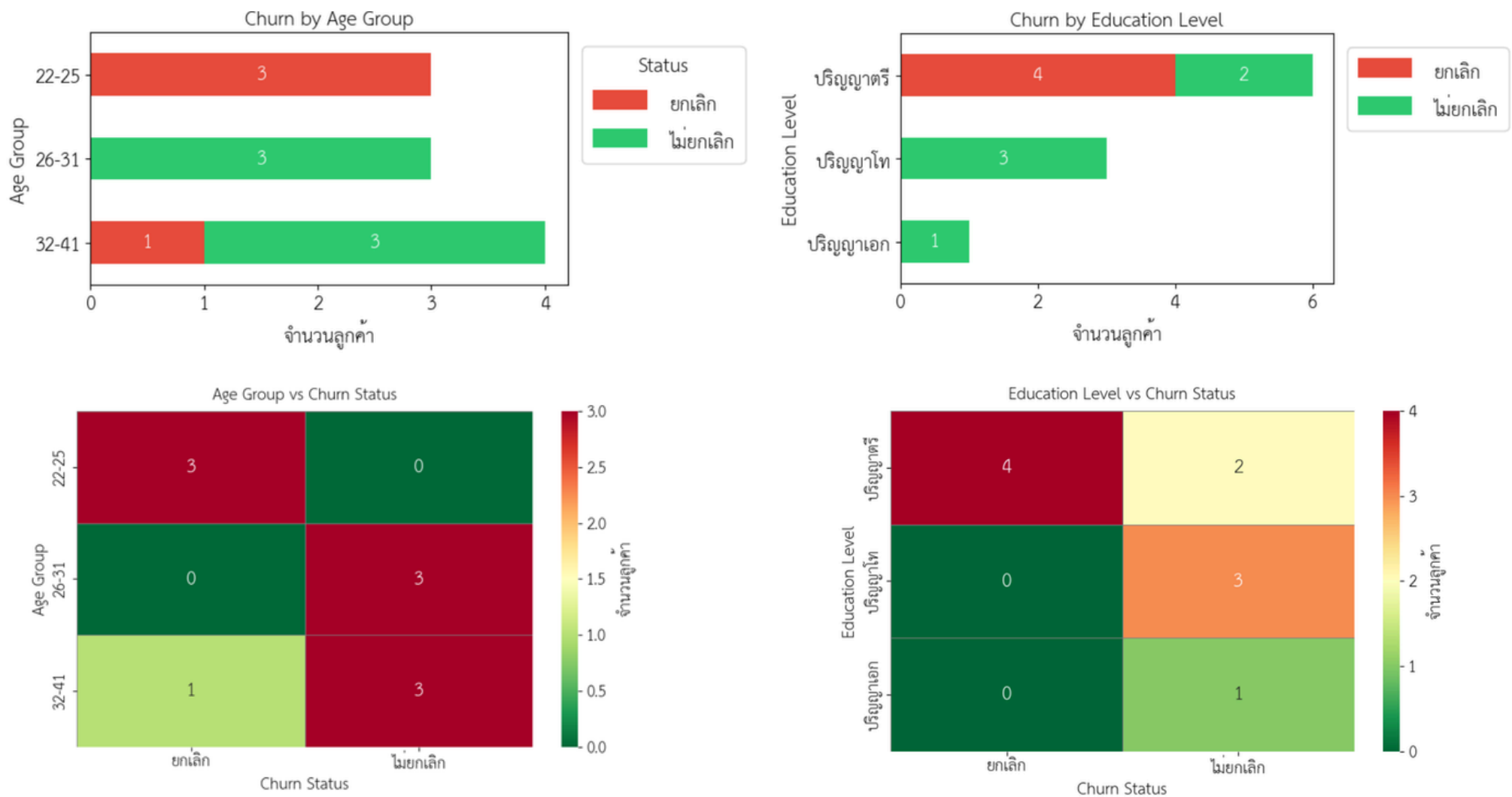
$P(x_i|C_k)$ คือ Conditional Probability (Likelihood) หรือ ความน่าจะเป็นที่จะพบข้อมูลตัวแปร i ในคลาส k

8.3.2 ตัวอย่างการคำนวณ Naïve Bayes

จงแสดงวิธีการคำนวณ Naïve Bayes เพื่อวิเคราะห์ว่า ลูกค้าจะยกเลิกการซื้อสินค้าหรือไม่

ตารางที่ 8-2ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ระดับการศึกษา	การยกเลิก
1	22-25	ปริญญาตรี	ยกเลิก
2	26-31	ปริญญาโท	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ยกเลิก
4	32-41	ปริญญาเอก	ไม่ยกเลิก
5	32-41	ปริญญาโท	ไม่ยกเลิก
6	32-41	ปริญญาตรี	ยกเลิก
7	26-31	ปริญญาโท	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ยกเลิก
9	26-31	ปริญญาตรี	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ไม่ยกเลิก



ภาพที่ 8-7 : ชุดข้อมูลฝึก (Training Set)

ตารางที่ 8-2ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ระดับการศึกษา	การยกเลิก
1	32-41	ปริญญาตรี	?
2	22-25	ปริญญาตรี	?

วิธีทำ

กำหนดให้ $x_1 = 32-41$, $x_2 =$ ปริญญาตรี

เมื่อ $k = 0$ (ไม่ยกเลิก)

$$\begin{aligned} P(C_0) \prod_{i=1}^2 P(x_i | C_0) &= P(C_0) \times P(x_1|C_0) \times P(x_2|C_0) \\ &= \left(\frac{6}{10}\right) \left(\frac{3}{6}\right) \left(\frac{2}{6}\right) = 0.1 \end{aligned}$$

เมื่อ $k = 1$ (ยกเลิก)

$$\begin{aligned} P(C_1) \prod_{i=1}^2 P(x_i | C_1) &= P(C_1) \times P(x_1|C_1) \times P(x_2|C_1) \\ &= \left(\frac{4}{10}\right) \left(\frac{1}{4}\right) \left(\frac{4}{4}\right) = 0.025 \end{aligned}$$

ตอบ ลูกค้านี้จะยัง “ไม่ยกเลิก” การซื้อสินค้า

รายการ	อายุ (ปี)	ระดับการศึกษา	การยกเลิก
1	22-25	ปริญญาตรี	ยกเลิก
2	26-31	ปริญญาโท	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ยกเลิก
4	32-41	ปริญญาเอก	ไม่ยกเลิก
5	32-41	ปริญญาโท	ไม่ยกเลิก
6	32-41	ปริญญาตรี	ยกเลิก
7	26-31	ปริญญาโท	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ยกเลิก
9	26-31	ปริญญาตรี	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ไม่ยกเลิก

กรณี ตัวแปรใด มีความที่เป็น 0 ให้แก้ไขด้วยวิธี **Laplace Smoothing** โดยการ
บวก 1 ในตัวตั้ง และ บวกจำนวนกลุ่มข้อมูลในตัวหาร เช่น อายุมี 3 กลุ่ม ต้องบวก 1 ในตัวตั้ง และ บวก 3 ในตัวหาร

กำหนดให้ $x_1 = 22-25$, $x_2 =$ ปริญญาตรี

เมื่อ $k = 0$ (ไม่ยกเลิก)

$$\begin{aligned} P(C_0) \prod_{i=1}^2 P(x_i | C_0) &= P(C_0) \times P(x_1|C_0) \times P(x_2|C_0) \\ &= \left(\frac{6}{10}\right) \left(\frac{\mathbf{0} + 1}{6 + 3}\right) \left(\frac{2}{6}\right) = 0.02 \end{aligned}$$

เมื่อ $k = 1$ (ยกเลิก)

$$\begin{aligned} P(C_1) \prod_{i=1}^2 P(x_i | C_1) &= P(C_1) \times P(x_1|C_1) \times P(x_2|C_1) \\ &= \left(\frac{4}{10}\right) \left(\frac{3 + 1}{4 + 3}\right) \left(\frac{4}{4}\right) = 0.23 \end{aligned}$$

รายการ	อายุ (ปี)	ระดับการศึกษา	การยกเลิก
1	22-25	ปริญญาตรี	ยกเลิก
2	26-31	ปริญญาโท	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ยกเลิก
4	32-41	ปริญญาเอก	ไม่ยกเลิก
5	32-41	ปริญญาโท	ไม่ยกเลิก
6	32-41	ปริญญาตรี	ยกเลิก
7	26-31	ปริญญาโท	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ยกเลิก
9	26-31	ปริญญาตรี	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ไม่ยกเลิก

ตอบ ลูกค้านี้จะ “**ยกเลิก**” การซื้อสินค้า

8.3.3 ตัวอย่างโค้ด Naïve Bayes

```
1 #Import Python library
2 from sklearn.naive_bayes import CategoricalNB
3 from sklearn.preprocessing import OrdinalEncoder
4
5 #Load Data
6 x = pd.read_csv('data/customer4.csv')
7 x_train = x[['age', 'education_level']].to_numpy()
8 y_train = x['churn']
9 x_test = [['32-41', 'ปริญญาตรี'], ['22-25', 'ปริญญาตรี']]
10
11 #Trainform Data by Using Ordinal Endcoding
12 cate_age = ['22-25', '26-31', '32-41']
13 cate_edu_level = ['ปริญญาตรี', 'ปริญญาโท', 'ปริญญาเอก']
14
15 trans = OrdinalEncoder(categories=[cate_age, cate_edu_level])
16 trans.fit(x_train)
17 x_train = trans.transform(x_train)
18
19 #Build and Train Model
20 net = CategoricalNB()
21 net.fit(x_train, y_train)
22
23 #Test Model
24 y_pred=net.predict(trans.transform(x_test))
25 print('ผลการวิเคราะห์ : ')
26 print(f'ลูกค้าคนที่มีอายุ {x_test[0][0]} ปี จบ{x_test[0][1]} จะ \033[1m{y_pred[0]}\033[0m การซื้อสินค้า')
27 print(f'ลูกค้าคนที่มีอายุ {x_test[1][0]} ปี จบ{x_test[1][1]} จะ \033[1m{y_pred[1]}\033[0m การซื้อสินค้า')
```

ผลการวิเคราะห์ :

ลูกค้าคนที่มีอายุ 32-41 ปี จบปริญญาตรี จะ ไม่ยกเลิก การซื้อสินค้า
ลูกค้าคนที่มีอายุ 22-25 ปี จบปริญญาตรี จะ ยกเลิก การซื้อสินค้า

8.4 Decision Tree (DT)

- อาศัยแนวคิดของความน่าจะเป็นเพื่อสร้างต้นไม้ตัดสินใจ
- ไม่ซับซ้อนสามารถเข้าใจได้ง่าย
- ไม่ Sensitive ต่อ Outliers
- เทคนิค Decision Tree มีดังนี้
 - ID3 (Iterative Dichotomiser 3) : ใช้สำหรับตัวแปรต้นแบบ Categorical ในงาน Classification เท่านั้น
 - C4.5 และ C5.0 : ใช้สำหรับตัวแปรต้นทั้งแบบ Categorical และ Numerical ในงาน Classification เท่านั้น
 - CART, Random Forest, และ XGBoost : ใช้สำหรับตัวแปรต้นทั้งแบบ Categorical และ Numerical ในงาน Classification และ Regression

Algorithm	ประเภทข้อมูล (Input)	ประเภทงาน (Output)	จุดเด่นเพิ่มเติม
ID3	Categorical เท่านั้น	Classification	ใช้ค่า Entropy และ Information Gain ในการเลือก Node
C4.5 / C5.0	Categorical / Numerical	Classification	ใช้ค่า Gain Ratio แทน Information Gain รองรับข้อมูลต่อเนื่องและจัดการ Missing Value ได้
CART	Categorical / Numerical	Classification / Regression	ใช้ Gini Index (Classification) และ MSE (Regression)
Random Forest	Categorical / Numerical	Classification / Regression	เป็น Ensemble Method (Bagging) เพื่อลดอาการ Overfitting
XGBoost	Categorical / Numerical	Classification / Regression	เป็น Ensemble Method (Boosting)

8.4.1 การคำนวณสำหรับ ID3

สูตรการคำนวณ

- การเทรนโมเดล (สร้างต้นไม้ตัดสินใจ)

$$Entropy(C) = - \sum_{k=0}^{K-1} P(C_k) \log_2 P(C_k)$$

$$Entropy(C \mid F_i) = - \sum_{k=0}^{K-1} P(C_k \mid F_{ij}) \log_2 P(C_k \mid F_{ij})$$

$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C \mid F_i)$$

เมื่อ

$Entropy(C)$ คือ Entropy ของ Class

K คือ จำนวน class

$Entropy(C, F_i)$ คือ Entropy ของตัวแปร i

m_i คือ จำนวนกลุ่มข้อมูลในตัวแปร i

$IG(C, F_i)$ คือ Information Gain ของตัวแปร i

F_{ij} คือ ตัวแปรตัวที่ i กลุ่มข้อมูลที่ j

- การทดสอบโมเดล

นำต้นไม้ที่ได้จากการเทรน มาใช้ในการตัดสินใจ (ทำนาย) โดยผลการทำนายจะอยู่ที่ Leaf Node

8.4.2 ตัวอย่างการคำนวณ ID3

จงแสดงวิธีการคำนวณ ID3 เพื่อวิเคราะห์ว่า ลูกค้าจะยกเลิกการซื้อสินค้าหรือไม่

ตารางที่ 8-3ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
1	22-25	ปริญญาตรี	หญิง	ยกเลิก
2	26-31	ปริญญาโท	ชาย	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ชาย	ยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
7	26-31	ปริญญาโท	หญิง	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ชาย	ยกเลิก
9	26-31	ปริญญาตรี	ชาย	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

ตารางที่ 8-3ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
1	32-41	ปริญญาตรี	ชาย	?
2	22-25	ปริญญาตรี	หญิง	?

วิธีทำ

การเทรนโมเดล (สร้างต้นไม้ตัดสินใจ)

- คำนวณ Entropy ของ Class

$$Entropy(C) = - \sum_{k=0}^1 P(C_k) \log_2 P(C_k)$$

$$= - \left(\frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10} \right)$$

$$= -((0.6)(-0.737) + (0.4)(-1.322)) = 0.97$$

class	จำนวน
0 = ไม่ยกเลิก	6
1 = ยกเลิก	4
รวม	10

- คำนวณ Entropy และ IG ของตัวแปร อายุ

$$Entropy(C | F_i) = - \sum_{k=0}^{K-1} P(C_k | F_{ij}) \log_2 P(C_k | F_{ij})$$

- กลุ่ม อายุ 22-25 $= - \left(\frac{0}{3} \log_2 \frac{0}{3} + \frac{3}{3} \log_2 \frac{3}{3} \right) = 0.0$
- กลุ่ม อายุ 26-31 $= - \left(\frac{3}{3} \log_2 \frac{3}{3} + \frac{0}{3} \log_2 \frac{0}{3} \right) = 0.0$
- กลุ่ม อายุ 32-41 $= - \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) = 0.81$

$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C | F_i)$$

$$= 0.97 - \left[\frac{3}{10} (0) + \frac{3}{10} (0) + \frac{4}{10} (0.81) \right] = 0.65$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
1	22-25	ปริญญาตรี	หญิง	ยกเลิก
2	26-31	ปริญญาโท	ชาย	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ชาย	ยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
7	26-31	ปริญญาโท	หญิง	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ชาย	ยกเลิก
9	26-31	ปริญญาตรี	ชาย	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

อายุ	ไม่ยกเลิก	ยกเลิก	รวม
22-25	0	3	3
26-31	3	0	3
32-41	3	1	4
รวม	6	4	10

- คำนวณ Entropy และ IG ของตัวแปร ระดับการศึกษา

$$Entropy(C | F_i) = - \sum_{k=0}^{K-1} P(C_k | F_{ij}) \log_2 P(C_k | F_{ij})$$

- กลุ่ม ปริญญาตรี $= - \left(\frac{2}{6} \log_2 \frac{2}{6} + \frac{4}{6} \log_2 \frac{4}{6} \right) = 0.92$
- กลุ่ม ปริญญาโท $= - \left(\frac{3}{3} \log_2 \frac{3}{3} + \frac{0}{3} \log_2 \frac{0}{3} \right) = 0.0$
- กลุ่ม ปริญญาเอก $= - \left(\frac{1}{1} \log_2 \frac{1}{1} + \frac{0}{1} \log_2 \frac{0}{1} \right) = 0.0$

$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C | F_i)$$

$$= 0.97 - \left[\frac{6}{10} (0.92) + \frac{3}{10} (0) + \frac{1}{10} (0) \right] = 0.42$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
1	22-25	ปริญญาตรี	หญิง	ยกเลิก
2	26-31	ปริญญาโท	ชาย	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ชาย	ยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
7	26-31	ปริญญาโท	หญิง	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ชาย	ยกเลิก
9	26-31	ปริญญาตรี	ชาย	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

ระดับการศึกษา	ไม่ยกเลิก	ยกเลิก	รวม
ปริญญาตรี	2	4	6
ปริญญาโท	3	0	3
ปริญญาเอก	1	0	1
รวม	6	4	10

- คำนวณ Entropy และ IG ของตัวแปร เพศ

$$Entropy(C | F_i) = - \sum_{k=0}^{K-1} P(C_k | F_{ij}) \log_2 P(C_k | F_{ij})$$

◦ กลุ่ม ชาย $= - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.92$
 ◦ กลุ่ม หญิง $= - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1.0$

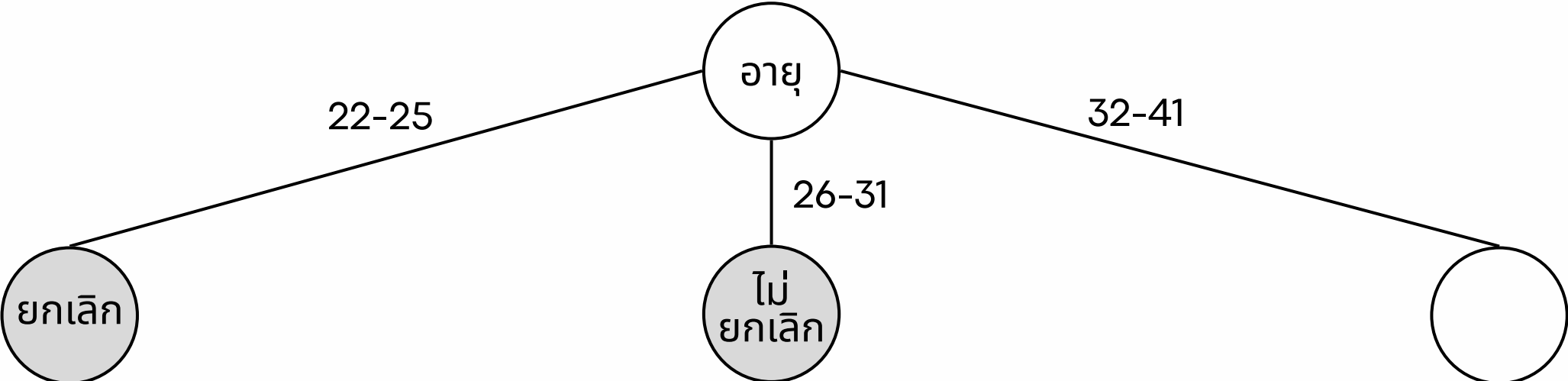
$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C | F_i)$$

$$= 0.97 - \left[\frac{6}{10} (0.92) + \frac{4}{10} (1.0) \right] = 0.02$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
1	22-25	ปริญญาตรี	หญิง	ยกเลิก
2	26-31	ปริญญาโท	ชาย	ไม่ยกเลิก
3	22-25	ปริญญาตรี	ชาย	ยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
7	26-31	ปริญญาโท	หญิง	ไม่ยกเลิก
8	22-25	ปริญญาตรี	ชาย	ยกเลิก
9	26-31	ปริญญาตรี	ชาย	ไม่ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

เพศ	ไม่ยกเลิก	ยกเลิก	รวม
ชาย	4	2	6
หญิง	2	2	4
รวม	6	4	10

- เลือก Root Node จากตัวแปรที่มีค่า IG สูงสุด
ดังนั้น ตัวแปร “อายุ” ถูกเลือก เนื่องจากมีค่า IG (0.65) สูงสุด จากนั้น แบ่งข้อมูลออกเป็น 3 กลุ่ม ตามจำนวนกลุ่มของ “อายุ” ดังรูป



รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเล็ก
1	22-25	ปริญญาตรี	หญิง	ยกเล็ก
3	22-25	ปริญญาตรี	ชาย	ยกเล็ก
8	22-25	ปริญญาตรี	ชาย	ยกเล็ก

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเล็ก
2	26-31	ปริญญาโท	ชาย	ไม่ยกเล็ก
7	26-31	ปริญญาโท	หญิง	ไม่ยกเล็ก
9	26-31	ปริญญาตรี	ชาย	ไม่ยกเล็ก

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเล็ก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเล็ก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเล็ก
6	32-41	ปริญญาตรี	หญิง	ยกเล็ก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเล็ก

หยุดการแยก node เมื่อ

- 1) “ตัวแปรตาม” มีค่าเหมือนกันหมด หรือ
- 2) “ตัวแปรต้น” มีค่าเหมือนกันหมด

ข้อมูลที่เหลือยังสามารถแบ่งต่อได้
ดังนั้น จะนำข้อมูลส่วนนี้ไปคำนวณต่อในรอบที่ 2

- คำนวณ Entropy ของ Class (รอบที่ 2)

$$\begin{aligned}
 Entropy(C) &= - \sum_{k=0}^1 P(C_k) \log_2 P(C_k) \\
 &= - \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) \\
 &= 0.81
 \end{aligned}$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

หมายเหตุ : รอบนี้จะไม่พิจารณา อายุ อีก เนื่องจากไม่มีความแตกต่างของอายุแล้ว

class	จำนวน
0 = ไม่ยกเลิก	3
1 = ยกเลิก	1
รวม	4

- คำนวณ Entropy และ IG ของตัวแปร ระดับการศึกษา

$$Entropy(C | F_i) = - \sum_{k=0}^{K-1} P(C_k | F_{ij}) \log_2 P(C_k | F_{ij})$$

- กลุ่ม ปริญญาตรี $= - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1.0$
- กลุ่ม ปริญญาโท $= - \left(\frac{1}{1} \log_2 \frac{1}{1} + \frac{0}{1} \log_2 \frac{0}{1} \right) = 0.0$
- กลุ่ม ปริญญาเอก $= - \left(\frac{1}{1} \log_2 \frac{1}{1} + \frac{0}{1} \log_2 \frac{0}{1} \right) = 0.0$

$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C | F_i)$$

$$= 0.81 - \left[\frac{2}{4}(1) + \frac{1}{4}(0) + \frac{1}{4}(0) \right] = 0.31$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

ระดับการศึกษา	ไม่ยกเลิก	ยกเลิก	รวม
ปริญญาตรี	1	1	2
ปริญญาโท	1	0	1
ปริญญาเอก	1	0	1
รวม	3	1	4

- คำนวณ Entropy และ IG ของตัวแปร เพศ

$$Entropy(C | F_i) = - \sum_{k=0}^{K-1} P(C_k | F_{ij}) \log_2 P(C_k | F_{ij})$$

$$= - \left(\frac{2}{2} \log_2 \frac{2}{2} + \frac{0}{2} \log_2 \frac{0}{2} \right) = 0.0$$

$$= - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1.0$$

○ กลุ่ม ชาย

○ กลุ่ม หญิง

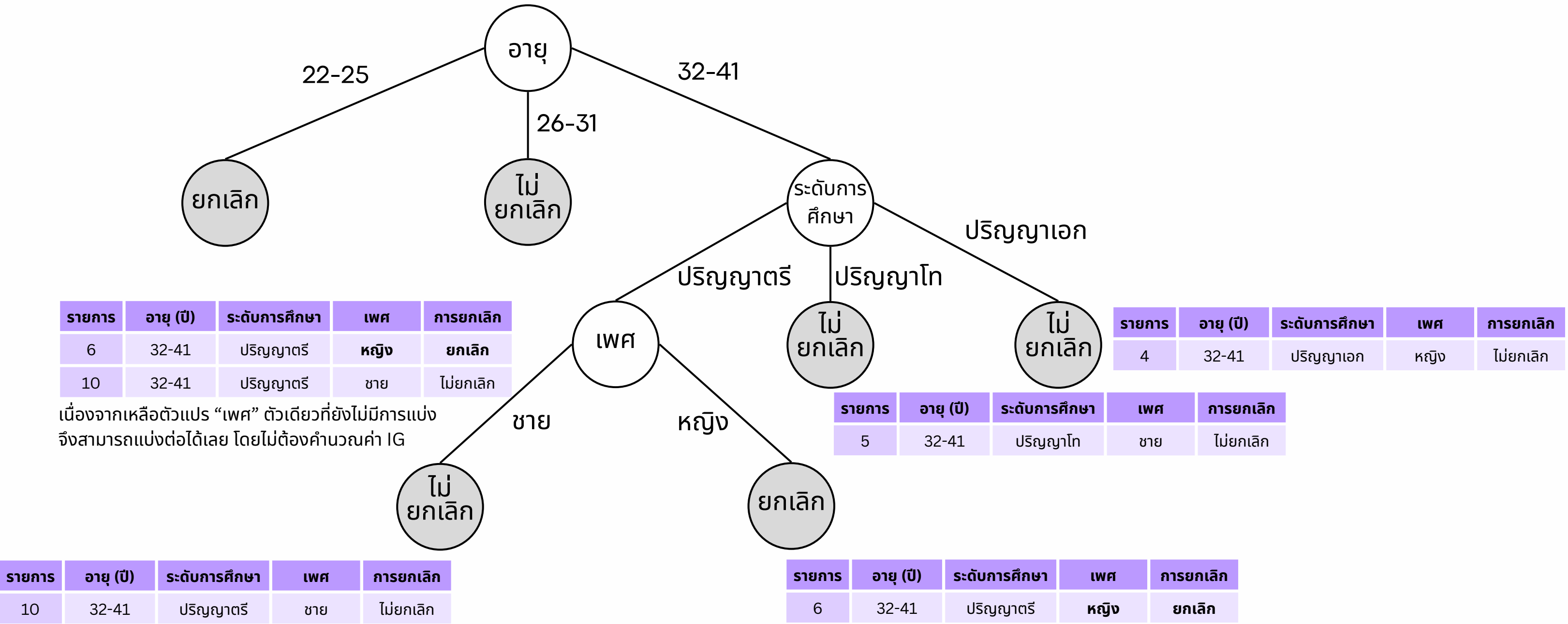
$$IG(C, F_i) = Entropy(C) - \sum_{j=1}^{m_i} P(F_{ij}) Entropy(C | F_i)$$

$$= 0.81 - \left[\frac{2}{4}(0) + \frac{2}{4}(1) \right] = 0.31$$

รายการ	อายุ (ปี)	ระดับการศึกษา	เพศ	การยกเลิก
4	32-41	ปริญญาเอก	หญิง	ไม่ยกเลิก
5	32-41	ปริญญาโท	ชาย	ไม่ยกเลิก
6	32-41	ปริญญาตรี	หญิง	ยกเลิก
10	32-41	ปริญญาตรี	ชาย	ไม่ยกเลิก

เพศ	ไม่ยกเลิก	ยกเลิก	รวม
ชาย	2	0	2
หญิง	1	1	2
รวม	3	1	4

- เลือก Node ถัดไป จากตัวแปรที่มีค่า IG สูงสุด
เนื่องจาก ตัวแปร “ระดับการศึกษา” และ “เพศ” มีค่า IG เท่ากัน จึงสามารถเลือกตัวแปรไหนก็ได้ ในที่นี้จะเลือก “ระดับการศึกษา” จากนั้น แบ่งข้อมูลออกเป็น 3 กลุ่ม ตามจำนวนกลุ่มของ “ระดับการศึกษา” ดังรูป



การทดสอบโมเดล

จากโมเดลต้นไม้ตัดสินใจ

เมื่อ

อายุ = 32-41

ระดับการศึกษา =ปริญญาตรี

เพศ = ชาย

ตอบ ลูกค้าน่าจะ**ไม่ยกเลิก**การซื้อสินค้า

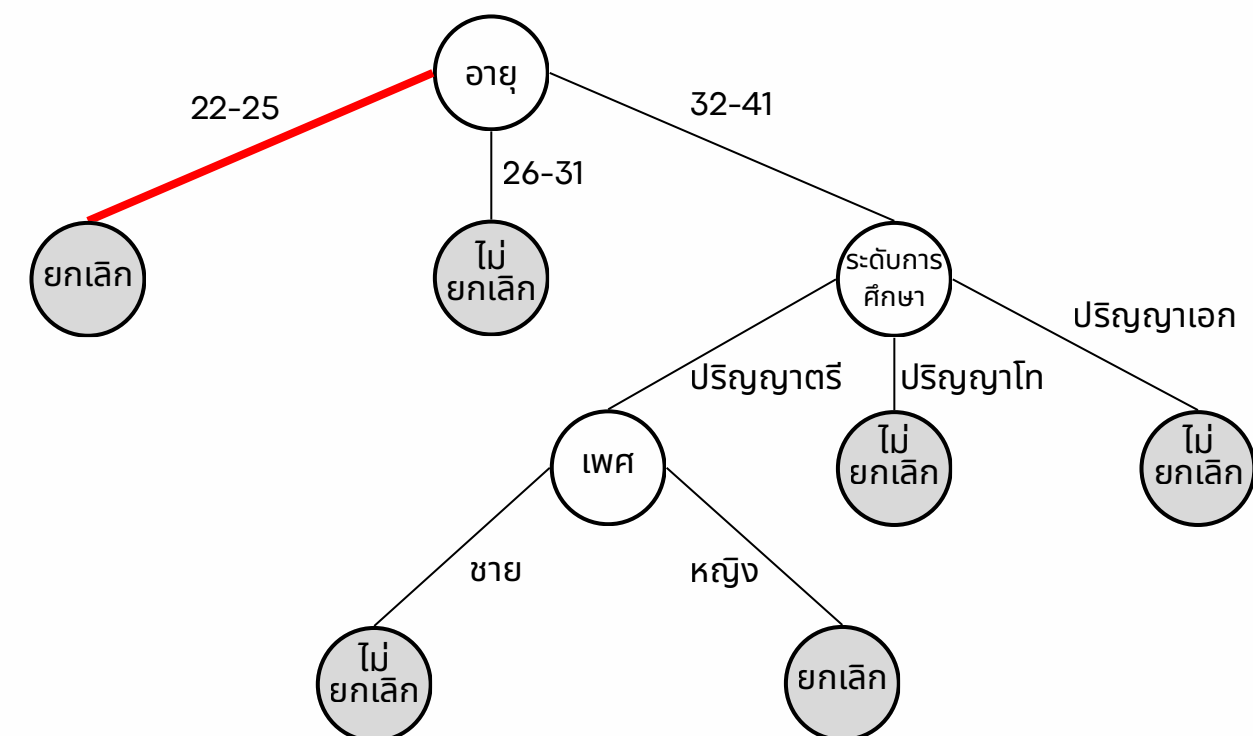
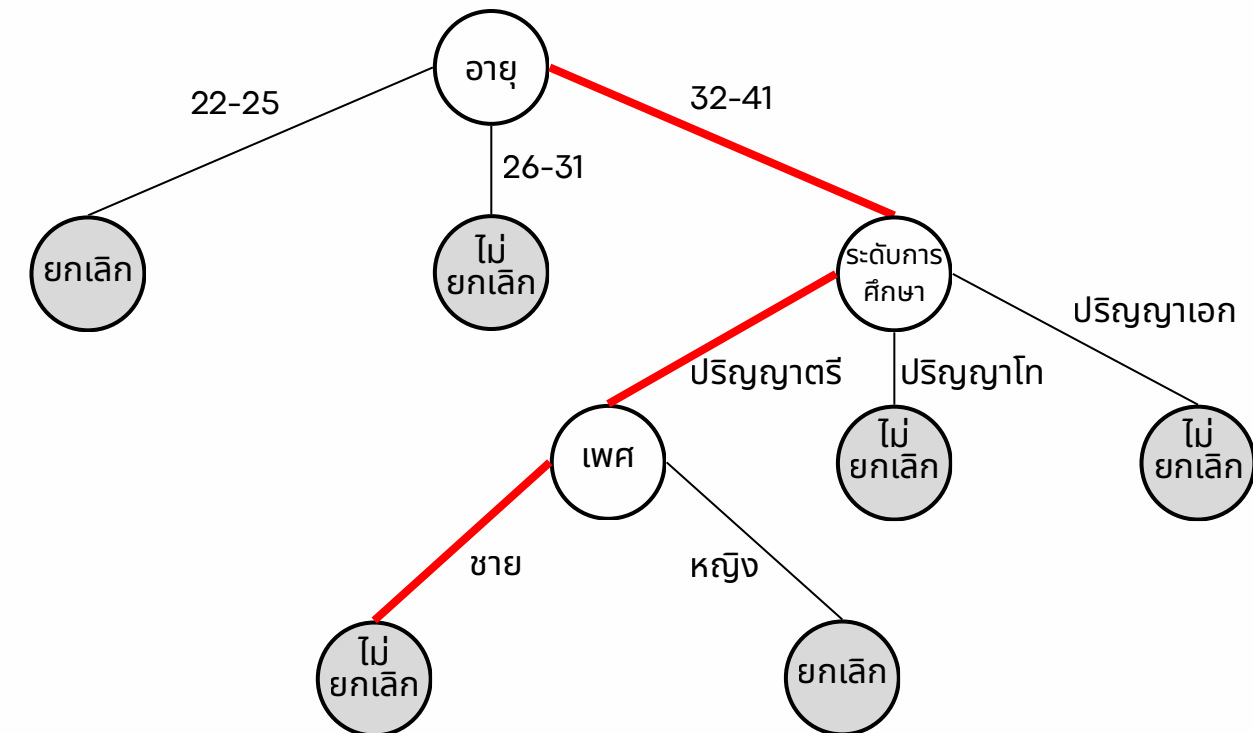
เมื่อ

อายุ = 22-25

ระดับการศึกษา = ปริญญาตรี

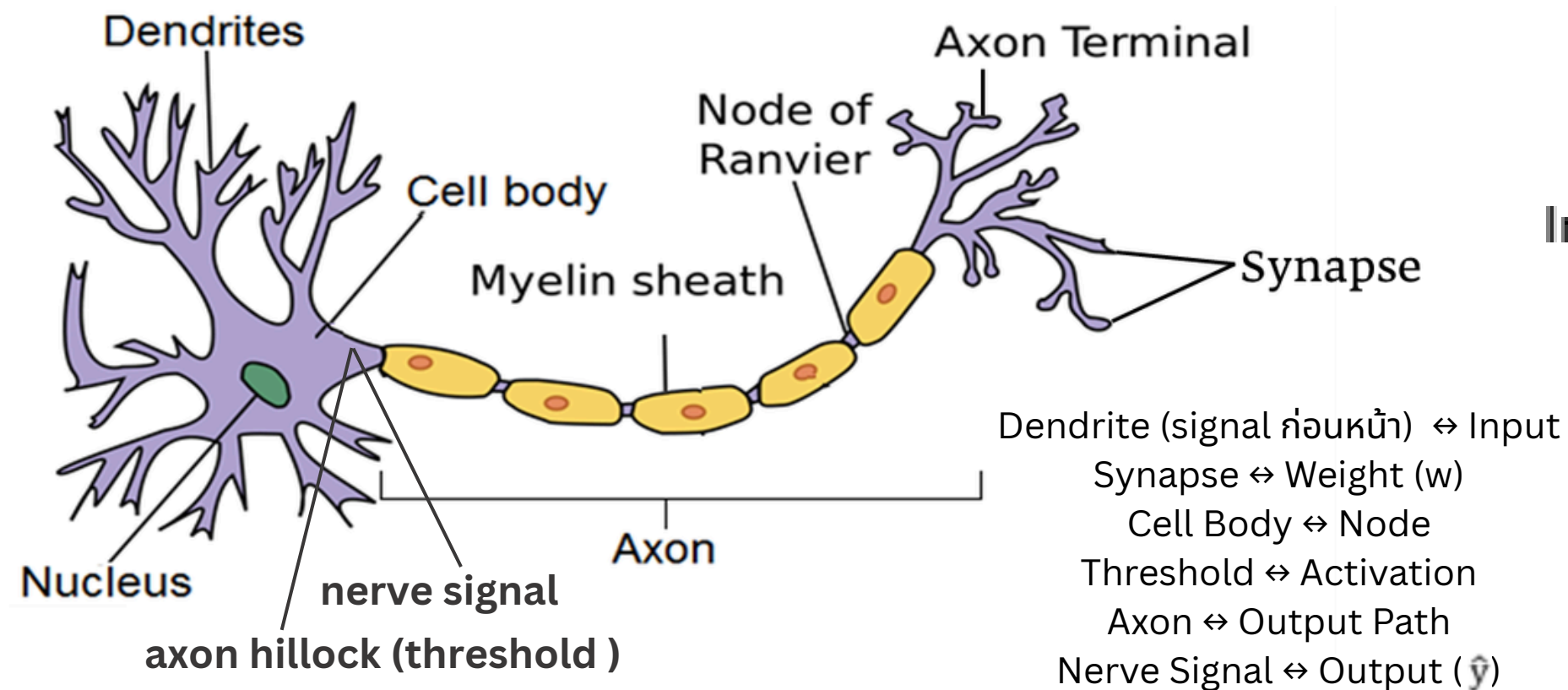
เพศ = หญิง

ตอบ ลูกค้าน่าจะ**ยกเลิก**การซื้อสินค้า

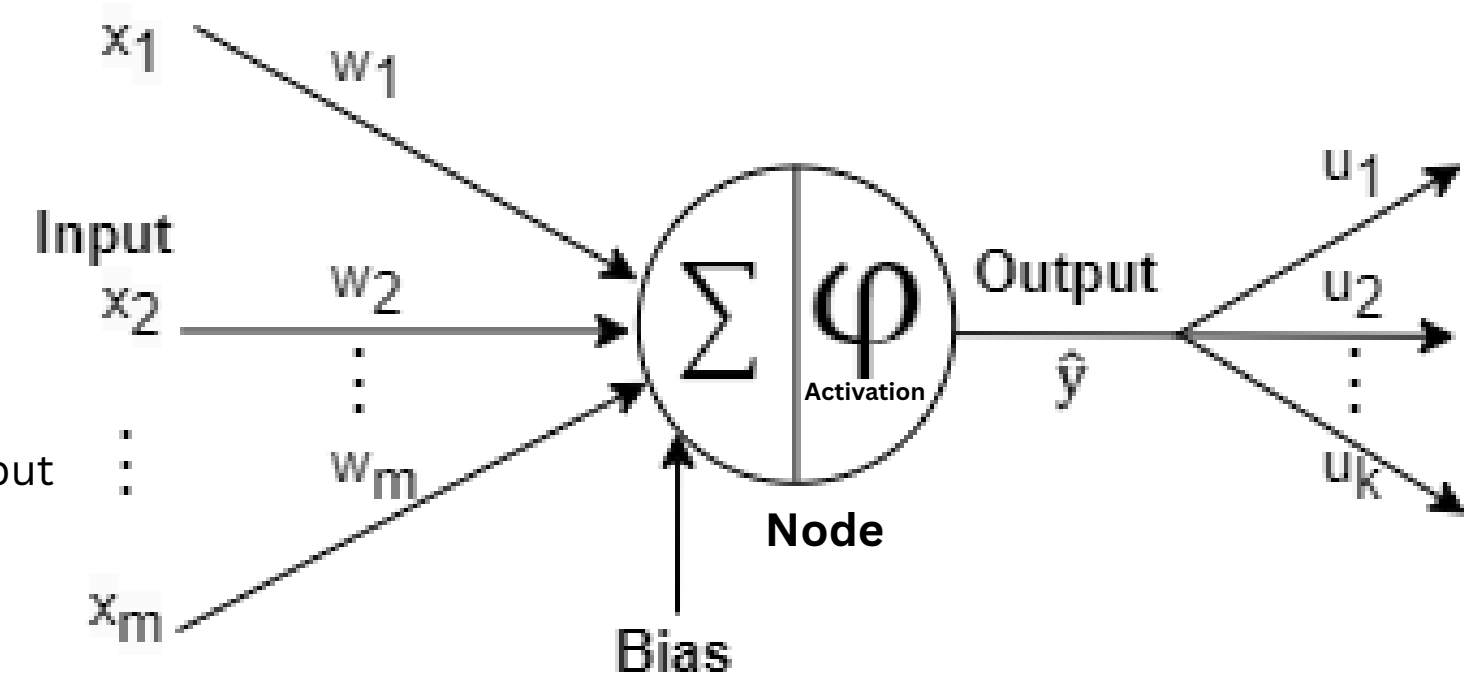


8.5 Artificial Neural Network (ANN)

โครงข่ายประสาทเทียม (Artificial Neural Network: ANN) คือ โครงข่ายประสาทเทียมที่สามารถเลียนแบบการทำงานของสมองมนุษย์ ใช้ในการประมวลผลข้อมูลและเรียนรู้รูปแบบ (patterns) จากข้อมูลที่ป้อนเข้าไป



Biological Neural Network

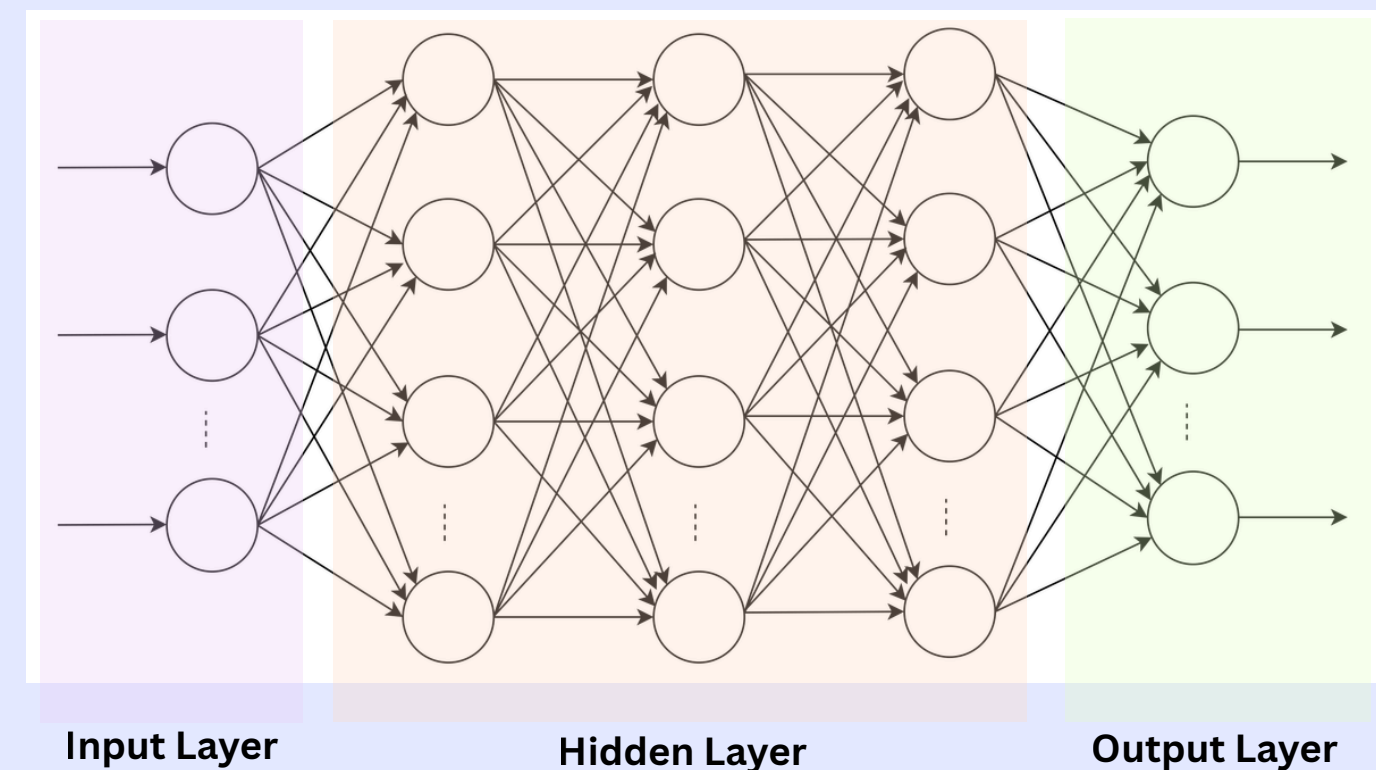


Artificial Neural Network (ANN)

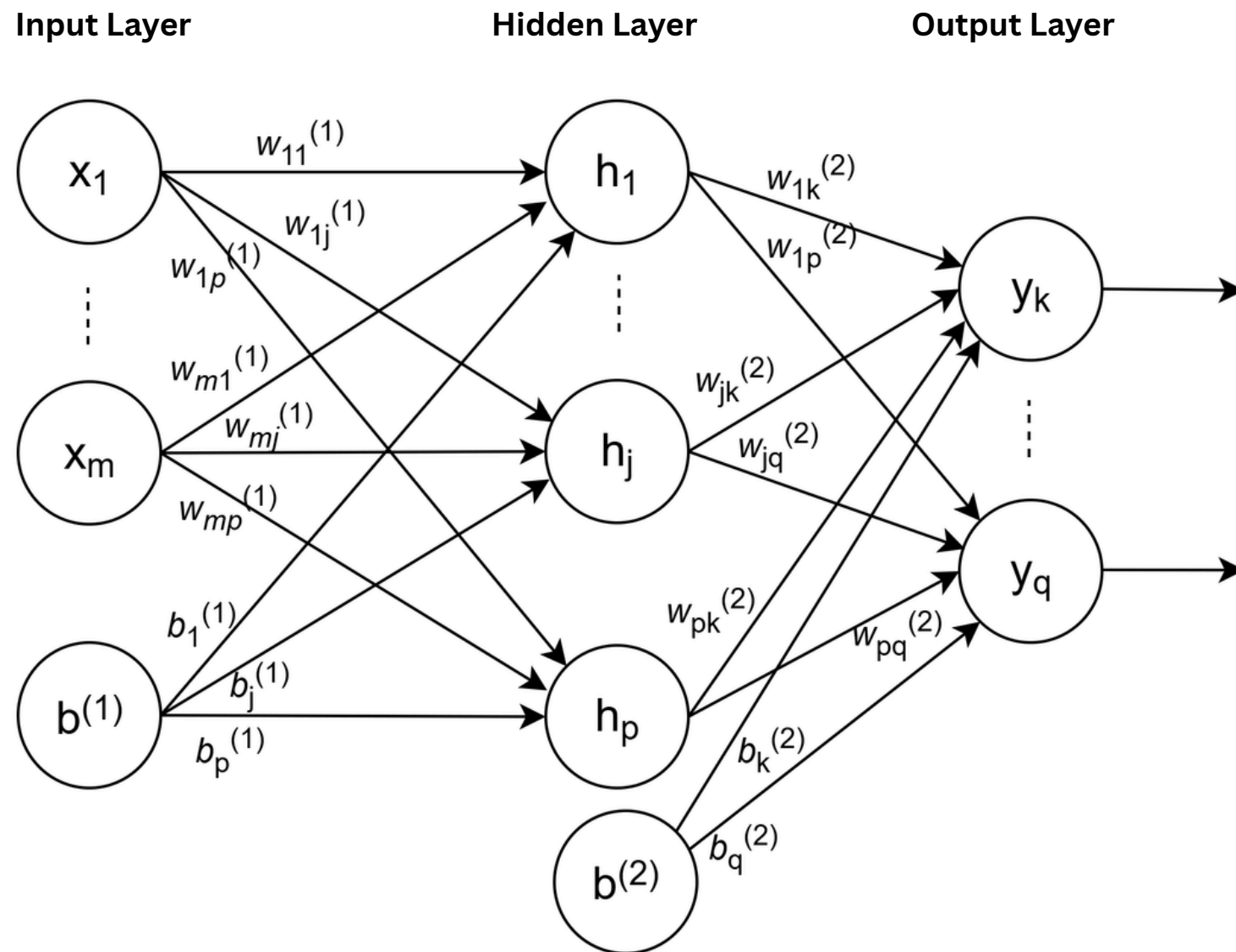
ภาพที่ 8-8 : เปรียบเทียบ Biological Neural Network กับ Artificial Neural Network (ANN)

8.5.1 Multilayer Perceptron (MLP)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Feed forward neural network
 - รับข้อมูลเข้าในทิศทางจาก Input → Hidden Layer(s) → Output
 - ใน output layer, คำนวณผลลัพธ์ (predicted) แล้วนำไปเทียบกับค่าจริง (actual) เพื่อคำนวณ Error
- เรียนรู้โดยใช้ Backpropagation
 - ย้อนกลับไปปรับ weight ใหม่ (อัปเดตความรู้) โดยใช้ error ที่คำนวณได้
- เป็น Universal Approximator
 - ในปี 1989 George Cybenko ได้พิสูจน์ว่า ANN ที่มีเพียง 1 Hidden Layer และใช้ Sigmoid เป็น Activation Function สามารถประมาณฟังก์ชันต่อเนื่องได้ทุกชนิด



8.5.1.1 โครงสร้างของ MLP



ภาพที่ 8-9 : โครงสร้างของ MLP

$$\mathbf{h} = \varphi(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$h_j = \varphi(w_{1j}^{(1)}x_1 + \dots + w_{mj}^{(1)}x_m + b_j^{(1)})$$

h_j คือ hidden node ที่ j

φ คือ activation function (เช่น ReLU หรือ Sigmoid)

$w_{1j}^{(1)}$ คือ ค่า weight ที่เชื่อมจาก input node ที่ 1 ไปยัง hidden node ที่ j

x_1 คือ ข้อมูล input node ที่ 1 ซึ่งมีทั้งหมด m โหนด

$b_j^{(1)}$ คือ ค่า bias ของ hidden node ที่ j

$$\hat{\mathbf{y}} = \varphi(\mathbf{w}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

$$\hat{y}_k = \varphi(w_{1k}^{(2)}h_1 + \dots + w_{jk}^{(2)}h_j + w_{pk}^{(2)}h_p + b_k^{(2)})$$

$\hat{y}_k$ คือ output node ตัวที่ k

φ คือ activation function เช่น Sigmoid สำหรับ binary class,

Softmax สำหรับ multi-class, และ linear/ReLU สำหรับ regression

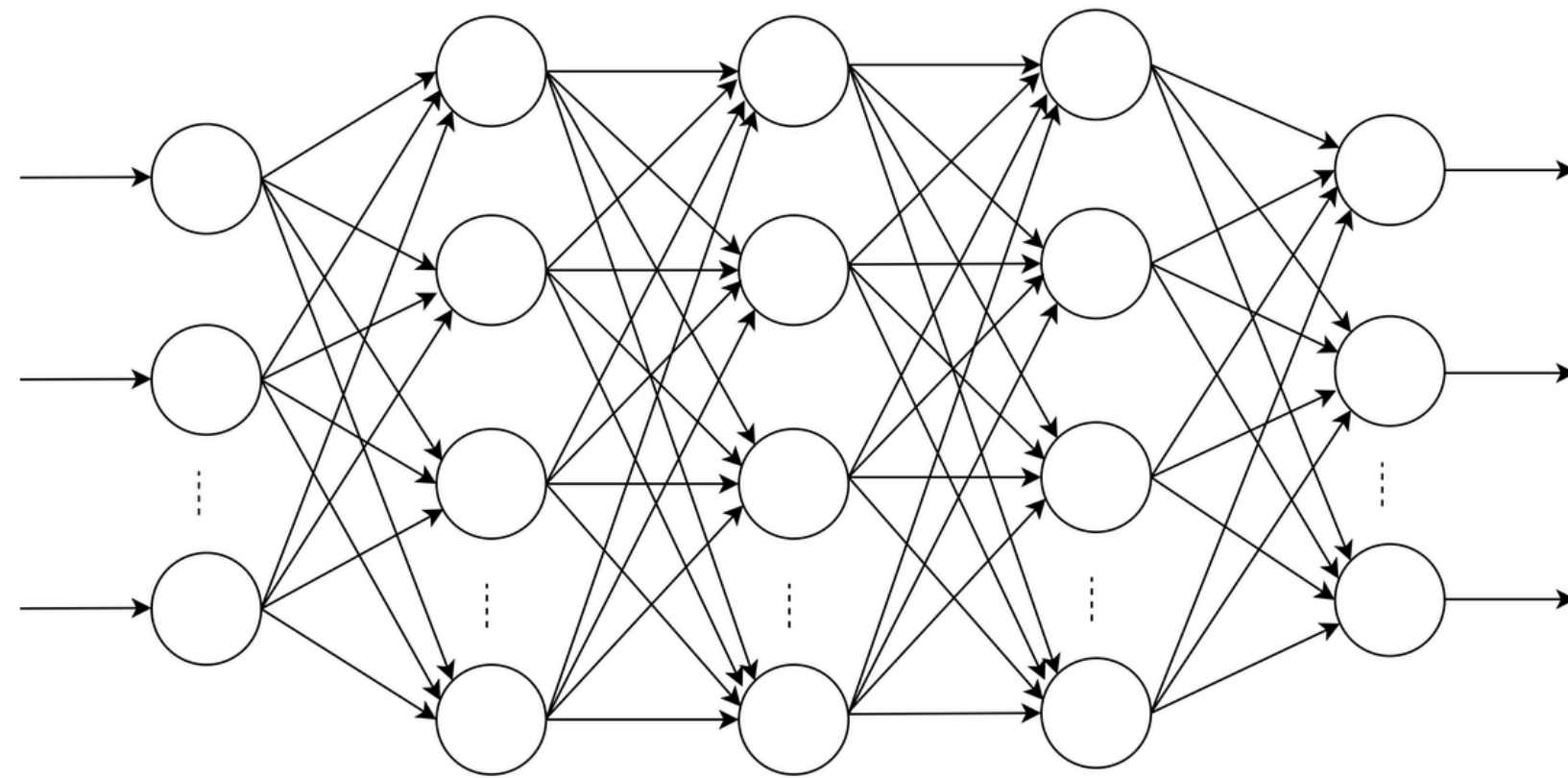
$w_{1k}^{(2)}$ คือ ค่า weight ที่เชื่อมจาก hidden node ที่ 1 ไปยัง output node ที่ k

h_1 คือ hidden node ที่ 1 ซึ่งมีทั้งหมด p โหนด

$b_k^{(2)}$ คือ ค่า bias ของ output node ที่ k

8.5.1.2 การคำนวณสำหรับ MLP

ขั้นตอนการเทรน (Train) โมเดล



1. forward → 2. loss (error) → 3. backprop → 4. adjust weight

ขั้นที่ 0 : กำหนดสถาปัตยกรรมและค่าเริ่มต้น

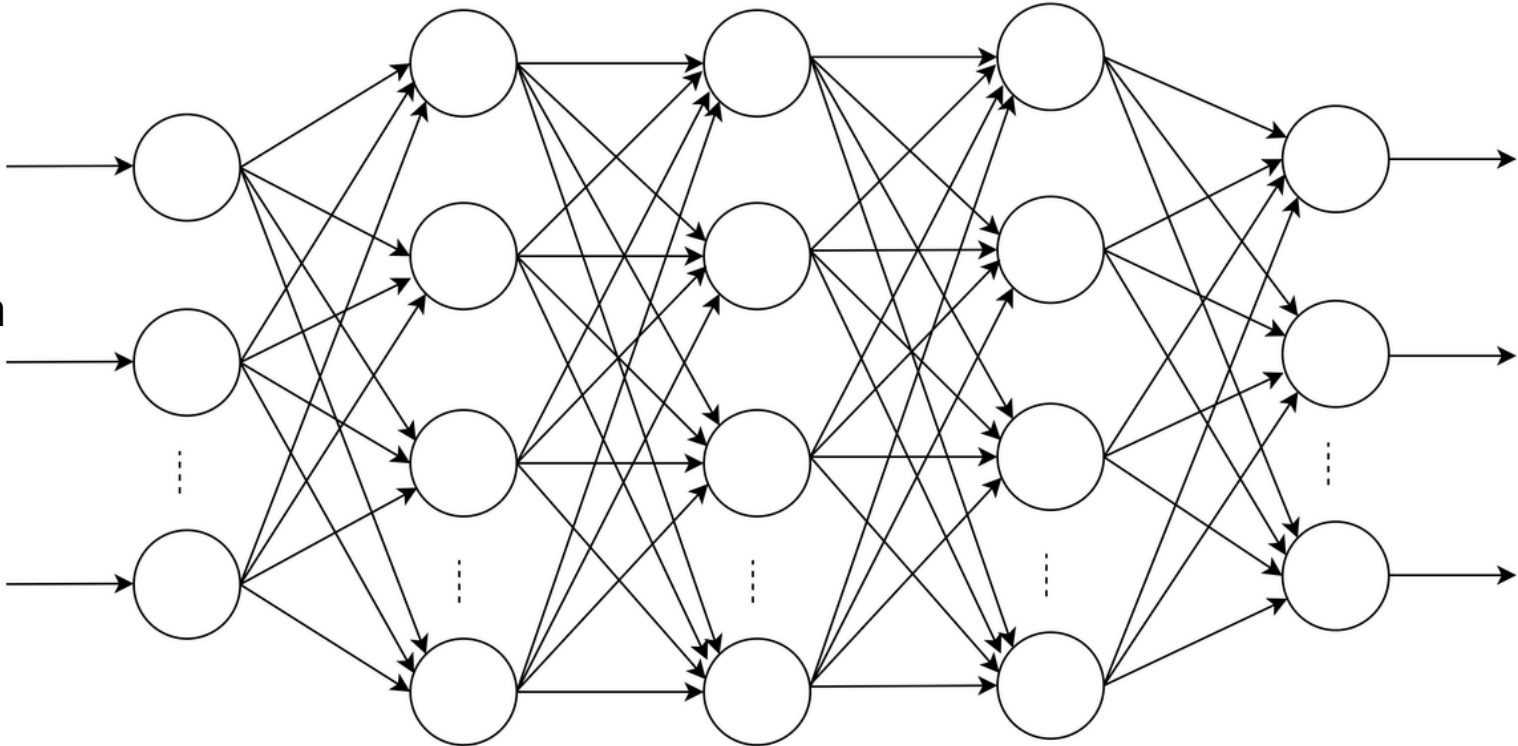
จำนวน Hidden Layer, จำนวน Node ในแต่ละ Layer, Activation Function, Optimizer, Lost function, Learning Rate, Momentum (เฉพาะใน sgd), จำนวน epoch, weight เริ่มต้น

- **การกำหนด จำนวน Hidden Layer**

- ปัญหาไม่ซับซ้อนมากใช้เพียง 1 layer ก็พอ
จำนวน hidden layer ที่มากกว่านี้มักจะไม่เพิ่มความถูกต้อง)
- ปัญหาซับซ้อน (ภาพ เสียง ข้อความ ฯลฯ) แม้จะใช้ MLP ได้
แต่แนะนำให้ใช้ DL อื่น เช่น CNN หรือ LSTM

การกำหนด จำนวน Node

- Input Layer
 - ถ้าเป็นข้อมูลแบบตาราง หรือ structure data ใช้จำนวน node เท่ากับจำนวน column
 - ถ้าเป็นข้อมูลภาพและเสียง (แปลงเสียงเป็นภาพ) ใช้จำนวน node เท่ากับจำนวน pixel ซึ่งคำนวณมาจากขนาดของภาพ (ขนาดภาพที่นิยม ภาพเล็ก=28×28=784, ภาพกลาง=32×32=1024, ภาพใหญ่=224×224=50176)
 - ถ้าเป็นข้อมูลข้อความ ใช้จำนวน node เท่ากับจำนวนคำ (word) ในประโยค หรือในเอกสาร
- Hidden Layer
 - ถ้า node น้อยเกินไป จะเกิดปัญหา underfitting มากเกินไป จะเกิดปัญหา overfitting
 - ถ้าเป็นข้อมูลแบบตาราง ไม่ควรมากกว่าจำนวนตัวแปรต้น (Input Node) หรืออาจใช้สูตร Hidden node ≈ (จำนวน Input Node + จำนวน Output Node) / 2
 - ถ้าเป็นข้อมูลภาพ (ภาพเล็ก 128–512 node, กลาง 256–1024 node, ใหญ่ 1024–4096 node)
 - ถ้าข้อมูลน้อย จำนวน node น้อย ถ้าข้อมูลมาก (เช่น คลิปสั้น/ล้านแถว) จำนวน node มาก
 - ทดลองเริ่มที่ค่าน้อย ๆ ก่อน (อาจเริ่มที่ครึ่งหนึ่งของจำนวนตัวแปรต้น) แล้วค่อย ๆ เพิ่มจำนวน node
- Output Layer
 - ถ้าเป็น regression หรือ binary class ให้ใช้ 1 node
 - ถ้าเป็น multi-class ให้ใช้จำนวน node เท่ากับจำนวน class

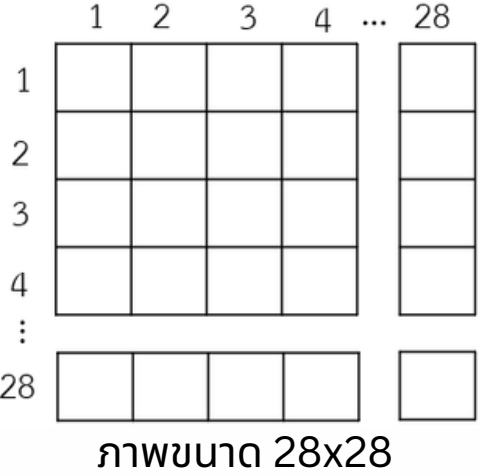


รายการที่รีวิวภาพยนตร์

รีวิวที่	รีวิว	ความรู้สึก
1	หนังเรื่องนี้ดีมาก สนุกสุดๆ	เชิงบวก
2	เป็นหนังที่ดี แต่ตอนจบไม่สนุกเลย	เชิงลบ
3	ฉันไม่ชอบหนังเรื่องนี้เลย	เชิงลบ

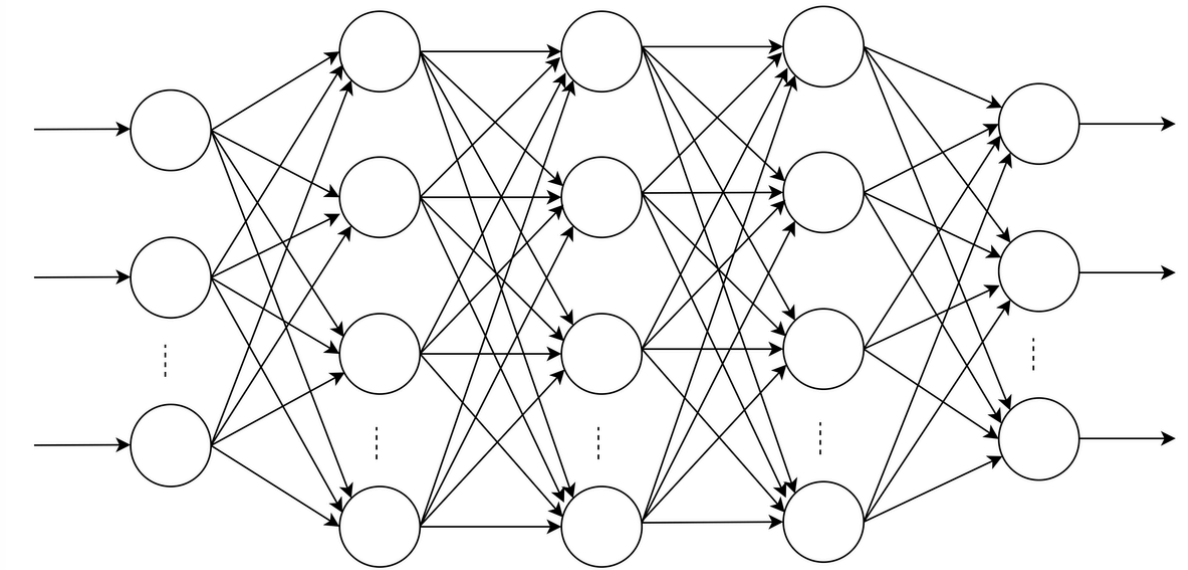
แปลงข้อมูลด้วยวิธี bag of words

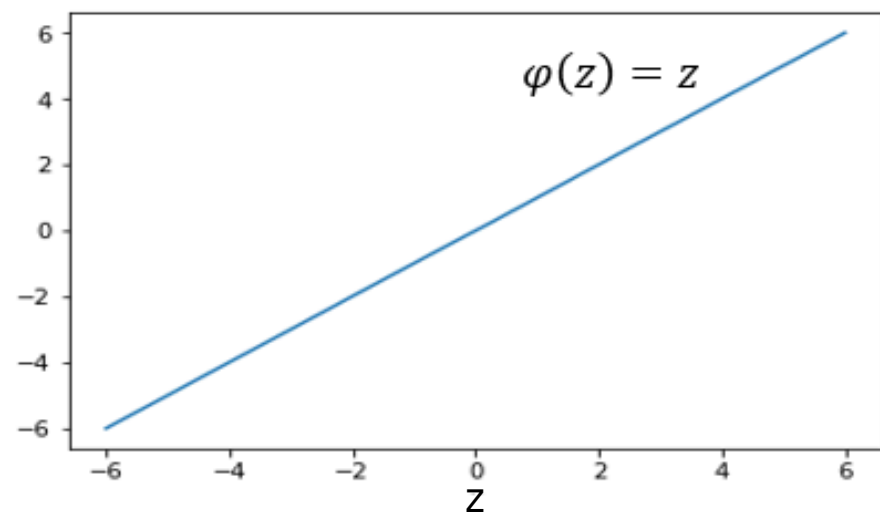
รีวิวที่	หนัง	ดี	มาก	สนุก	สุดๆ	ตอน	จบ	ไม่	ชอบ	เลย	ความรู้สึก
1	1	1	1	1	1	0	0	0	0	0	1
2	1	1	0	1	0	1	1	1	0	1	0
3	1	0	0	0	0	0	0	1	1	1	0



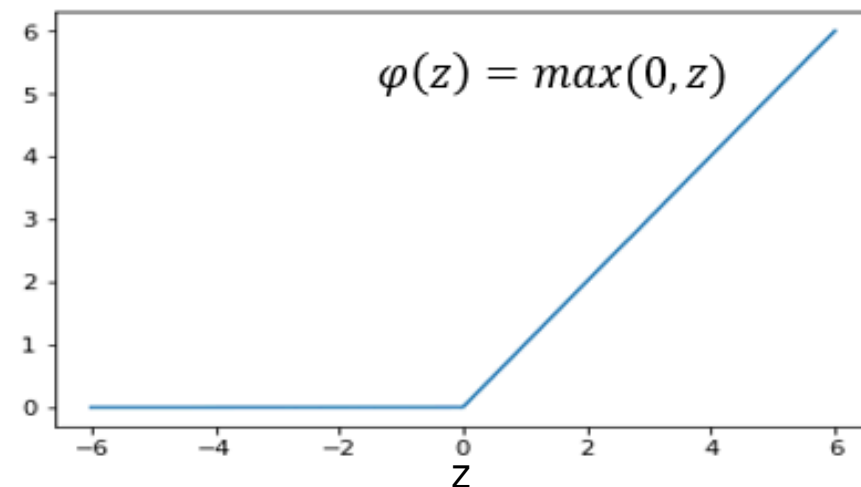
รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

- **การกำหนด Activation Function** เพื่อใช้เป็นตัวกระตุ้นหรือยับยั้งสัญญาณ
 - **Hidden layer**
 - ปัจจุบันนิยมใช้ ReLU เนื่องจากคำนวณได้เร็วกว่า, อดีตนิยมใช้ Sigmoid
 - **Output Layer**
 - Regression ใช้ Linear หรือ ReLU
 - Binary class ใช้ Sigmoid
 - Multi-class ใช้ Softmax

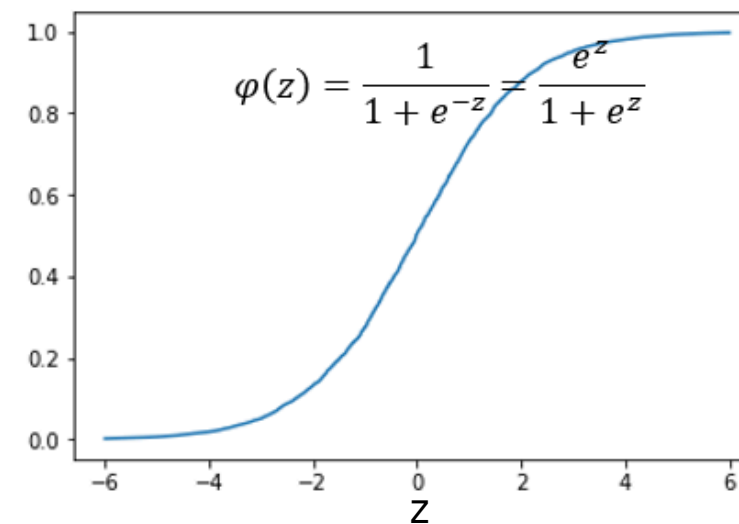




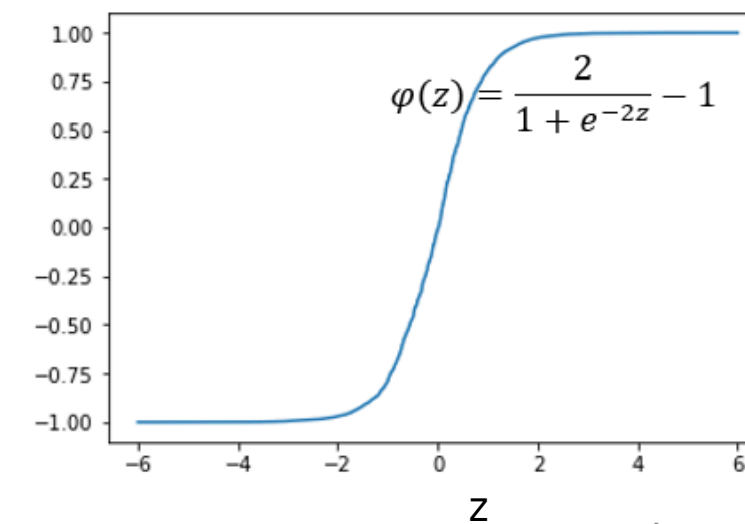
Identity (linear)



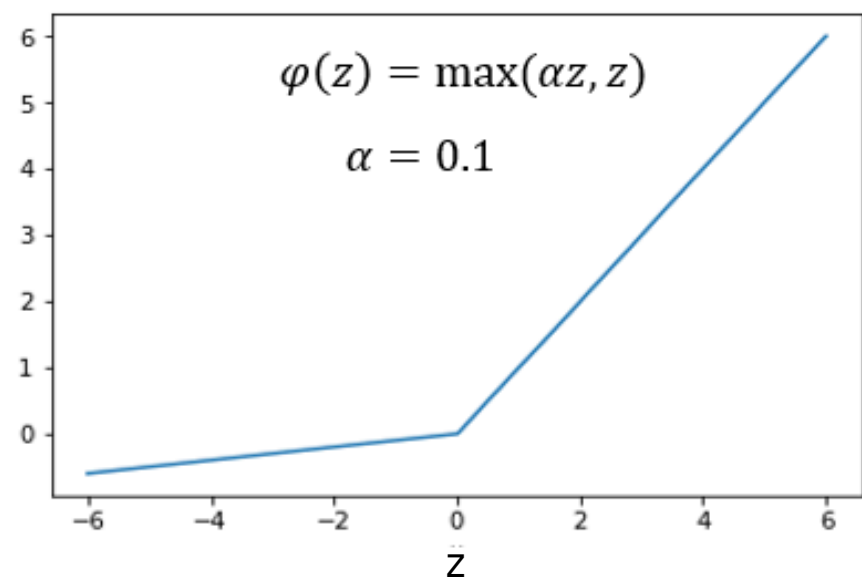
Rectified Linear Unit (ReLU)



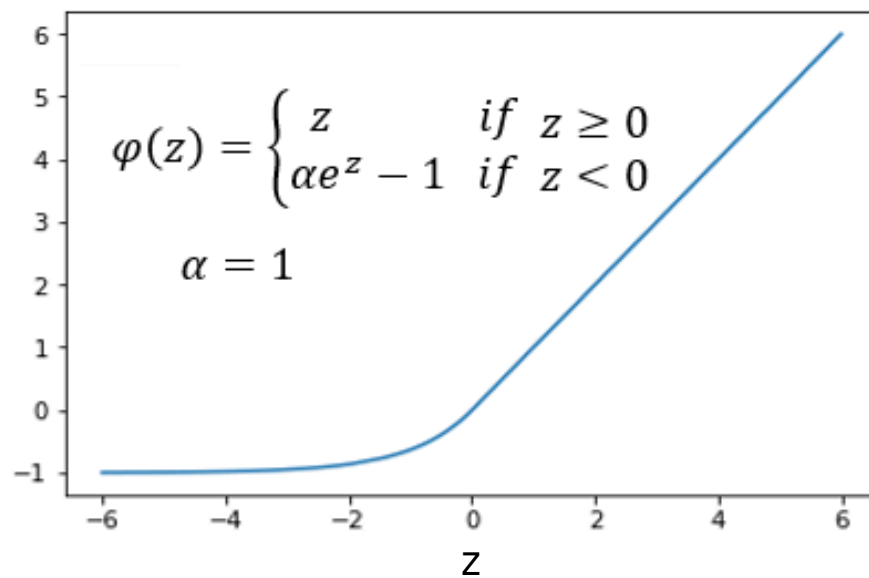
Logistic Sigmoid (logistic)



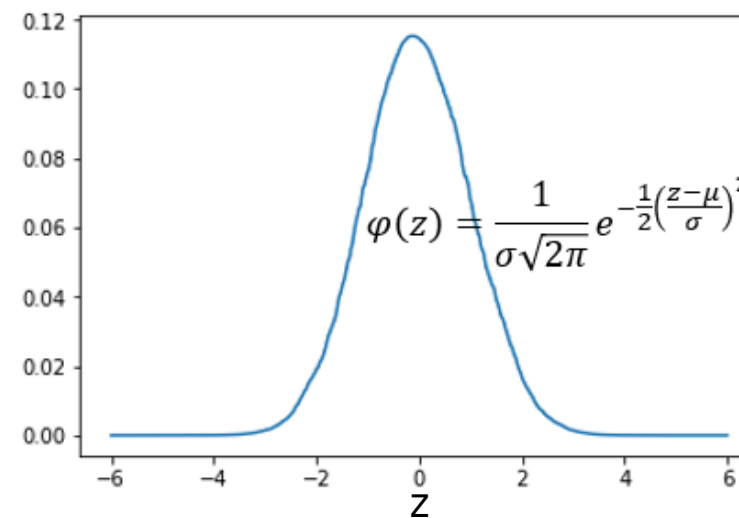
Hyperbolic Tangent Sigmoid (tanh)



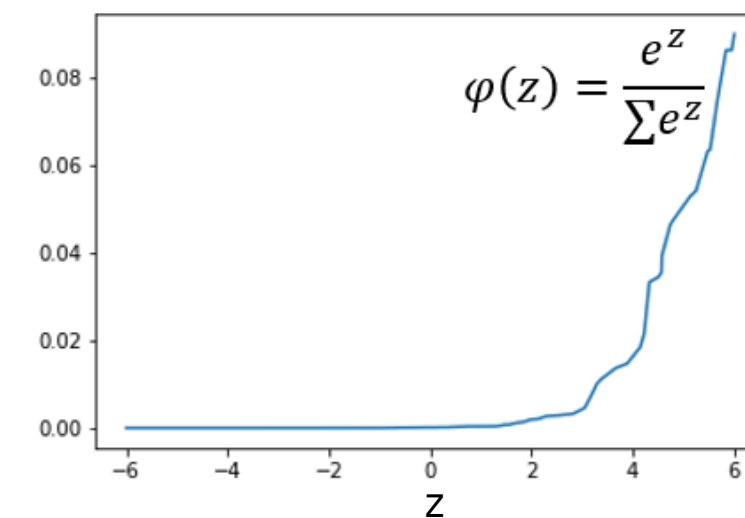
Leaky ReLU



Exponential Linear Unit (ELU)



Gaussian (RBF)



Softmax

ภาพที่ 8-10 : เปรียบเทียบกราฟของ Activation Function

- **การกำหนด Optimizer (Solver)** เพื่อใช้ในการปรับ weight
 - **Gradient Descent (GD)** ใช้ข้อมูลทุก data points เพื่อปรับ weight ใน 1 รอบ เรียกว่าการ Train แบบ offline หรือแบบ batch
 - **Stochastic Gradient Descent (SGD)** ใช้ข้อมูล 1 data point (online) หรือ บาง data points (mini-batch) เพื่อปรับ weight ใน 1 รอบย่อย -- SGD ไม่มี momentum ในตัวแต่เพิ่มได้
 - **Adaptive Moment Estimation (Adam)** เป็นอัลกอริทึมที่พัฒนามาจาก SGD โดยใช้มีการคำนวณค่า momentum ในตัว และการปรับ learning rate อัตโนมัติ (adaptive learning rate) ทำให้การเรียนรู้มีความเสถียรและรวดเร็วขึ้น **(นิยม ใช้ง่าย เสถียร)**
 - **Jimmy Ba Limited-memory Broyden–Fletcher–Goldfarb–Shanno (LBFGS)** เป็นอัลกอริทึมในกลุ่ม quasi-Newton methods ที่ใช้การประมาณ Hessian matrix (เมทริกซ์อนุพันธ์ลำดับสอง) แบบประหยัดหน่วยความจำ เหมาะสำหรับปัญหาที่มีจำนวนพารามิเตอร์มาก

หมายเหตุ : ใน GD การ Train 1 รอบ จะปรับ weight 1 ครั้ง แต่ใน SGD การ Train 1 รอบ จะปรับ weight หลายครั้ง

- **การกำหนด Lost function** เพื่อใช้ในการคำนวณ error
 - **Regression** ใช้ Mean Square Error (MSE)
 - **Binary class** ใช้ Binary Cross-Entropy (BCE)
 - **Multi-class** ใช้ Categorical Cross-Entropy (CE)
- **การกำหนด Learning Rate (η)** เพื่อใช้ในการควบคุมความ เร็ว/ช้า ของการเรียนรู้
 - η สูงเกินไป การเรียนรู้จะแกว่ง (divergence) มาก ถ้าต่ำเกินไปจะเรียนรู้จบ (convergence) ช้า
 - ปกติประมาณ 0.001
 - ถ้า loss ไม่ลด ให้ ลด Learning Rate ถ้า convergence ช้า ให้ เพิ่ม Learning Rate
- **การกำหนด Momentum (μ)** เพื่อใช้ในการควบคุมทิศทางของการเรียนรู้ (รักษาทิศความชันของ gradient)
 - ปกติประมาณ 0.9
 - ค่าประมาณ 0.9-0.99 เคลื่อนที่ราบเรียบ แต่เปลี่ยนทิศช้ากว่า
 - ค่าประมาณ 0.5-0.8 เคลื่อนที่เร็ว แต่แกว่งง่ายกว่า

- **การกำหนด จำนวน epoch** หรือจำนวนรอบของการ train
 - **ขึ้นกับขนาดข้อมูล** ถ้าข้อมูลขนาดใหญ่จำนวนรอบอาจน้อยลง ถ้าข้อมูลขนาดเล็กจำนวนรอบอาจมากขึ้น
 - **ใช้ Early Stopping** โดยดูค่า validation loss หากค่า loss ไม่เปลี่ยนหรือเปลี่ยนน้อยกว่าที่กำหนด การ train ก็จะหยุด
- **การกำหนด Weight เริ่มต้น**
 - เปรียบเสมือนการตั้งค่าความเข้มของสัญญาณตั้งต้น หรือเป็นการให้ความรู้เบื้องต้นแก่โมเดลก่อนการเรียนรู้จริง
 - โดยปกติจะใช้การสุ่ม (random)
 - ห้ามให้ค่าเริ่มต้นเป็นศูนย์ทั้งหมด (จะไม่สามารถเรียนรู้ได้)

ขั้นที่ 1 : คำนวณด้วยสมการ Forward เพื่อให้ได้ผลลัพธ์ หรือผลการทำนาย

$$\mathbf{h} = \varphi(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = \varphi(\mathbf{w}^{(2)}\mathbf{x} + \mathbf{b}^{(2)})$$

ขั้นที่ 2 : คำนวณ Loss Function (Error) เพื่อประเมินประสิทธิภาพของโมเดลในแต่ละรอบการเรียนรู้

- **Regression** (MSE + Linear)

$$\mathcal{L} = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$$

- **Binary class** (BCE + Sigmoid)

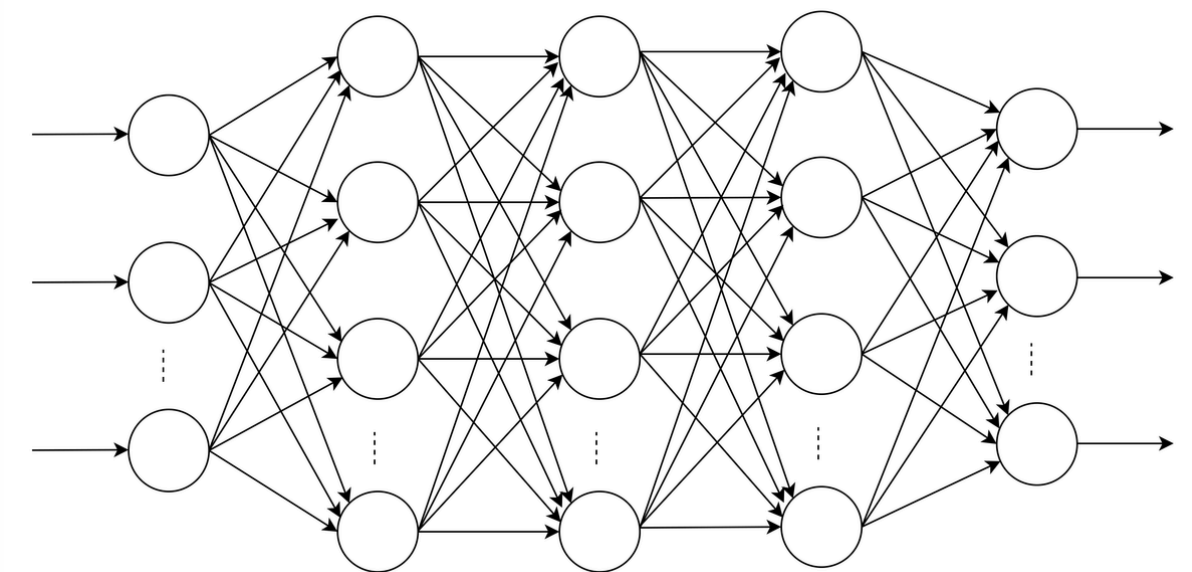
$$\hat{y} = \sigma(z^{(L)}),$$

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- **Multi-class** (CE + Softmax)

$$p = \text{softmax}(z^{(L)}),$$

$$\mathcal{L} = -\sum_k y_k \log p_k$$



ขั้นที่ 3 : ทำ Backpropagation เพื่อย้อนกลับไปเพื่อคำนวณค่า gradient

นิยาม error ที่ขั้นสุดท้าย:

- Softmax + CE: $\delta^{(L)} = p - y$ (สวยและเร็ว)
- Sigmoid + BCE: $\delta^{(L)} = \hat{y} - y$
- ทั่วไป: $\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot f'^{(L)}(z^{(L)})$

ย้อนกลับสำหรับชั้น $l = L - 1, \dots, 1$:

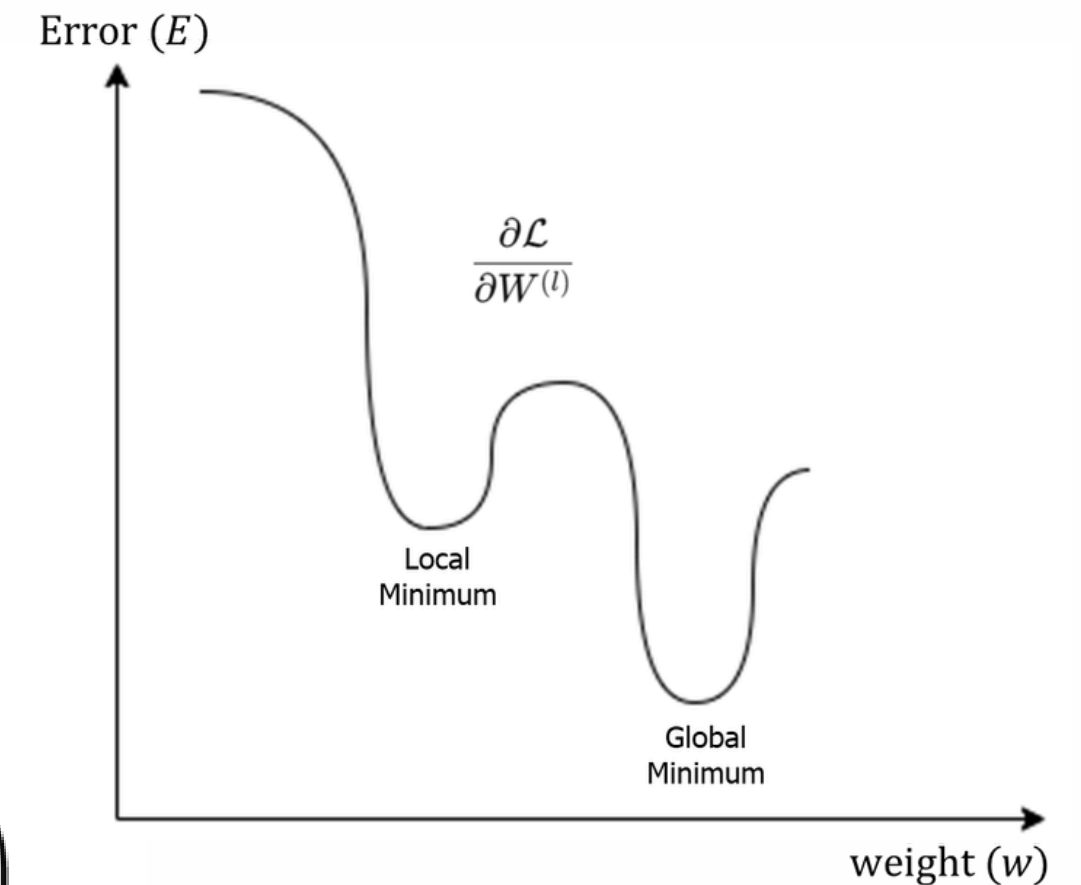
$$\delta^{(l)} = \left(W^{(l+1)}\right)^{\top} \delta^{(l+1)} \odot f'^{(l)}(z^{(l)})$$

เกรเดียนต์ต่อพารามิเตอร์:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^{\top}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

ขั้นที่ 4 : ปรับ Weight โดยใช้ Optimizer ร่วมกับค่า Gradient ที่คำนวณได้

$$\text{SGD: } W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$



ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่า

- ครบจำนวนรอบ (Epoch) ของการ Train ตามที่กำหนดไว้
- Early Stopping โดยดูค่า validation loss หากค่า loss ไม่เปลี่ยนหรือเปลี่ยนน้อยกว่าที่กำหนด การ train ก็จะหยุด

ขั้นตอนการทดสอบ (Test) โมเดล

ในการคำนวณหาผลลัพธ์ ($\hat{y}$)ให้นำเอา Test data เฉพาะส่วนที่เป็นตัวแปรต้น (x) ใส่เข้าไปในสมการที่ได้จากการ Train model โดยมีรูปแบบของสมการ ดังนี้

$$h_j = \varphi(w_{1j}^{(1)}x_1 + \dots + w_{mj}^{(1)}x_m + b_j^{(1)})$$

h_j คือ hidden node ที่ j

φ คือ activation function (เช่น ReLU หรือ Sigmoid)

$w_{1j}^{(1)}$ คือ ค่า weight ที่เชื่อมจาก input node ที่ 1 ไปยัง hidden node ที่ j

x_1 คือ ข้อมูล input node ที่ 1 ซึ่งมีทั้งหมด m โหนด

$b_j^{(1)}$ คือ ค่า bias ของ hidden node ที่ j

$$\hat{y}_k = \varphi(w_{1k}^{(2)}h_1 + \dots + w_{jk}^{(2)}h_j + w_{pk}^{(2)}h_p + b_k^{(2)})$$

$\hat{y}_k$ คือ output node ตัวที่ k

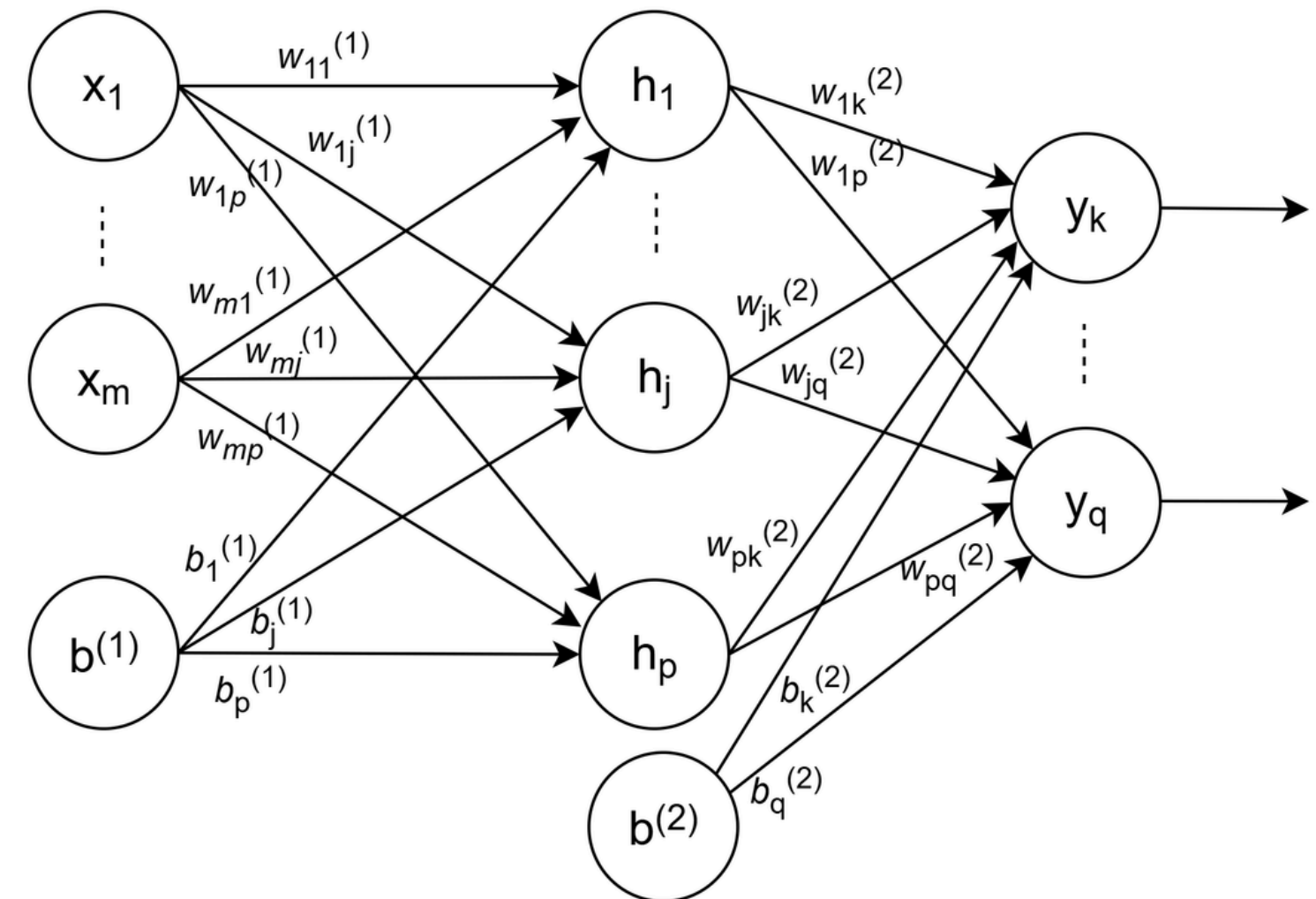
φ คือ activation function เช่น Sigmoid สำหรับ binary class,

Softmax สำหรับ multi-class, และ linear/ReLU สำหรับ regression

$w_{1k}^{(2)}$ คือ ค่า weight ที่เชื่อมจาก hidden node ที่ 1 ไปยัง output node ที่ k

h_1 คือ hidden node ที่ 1 ซึ่งมีทั้งหมด p โหนด

$b_k^{(2)}$ คือ ค่า bias ของ output node ที่ k



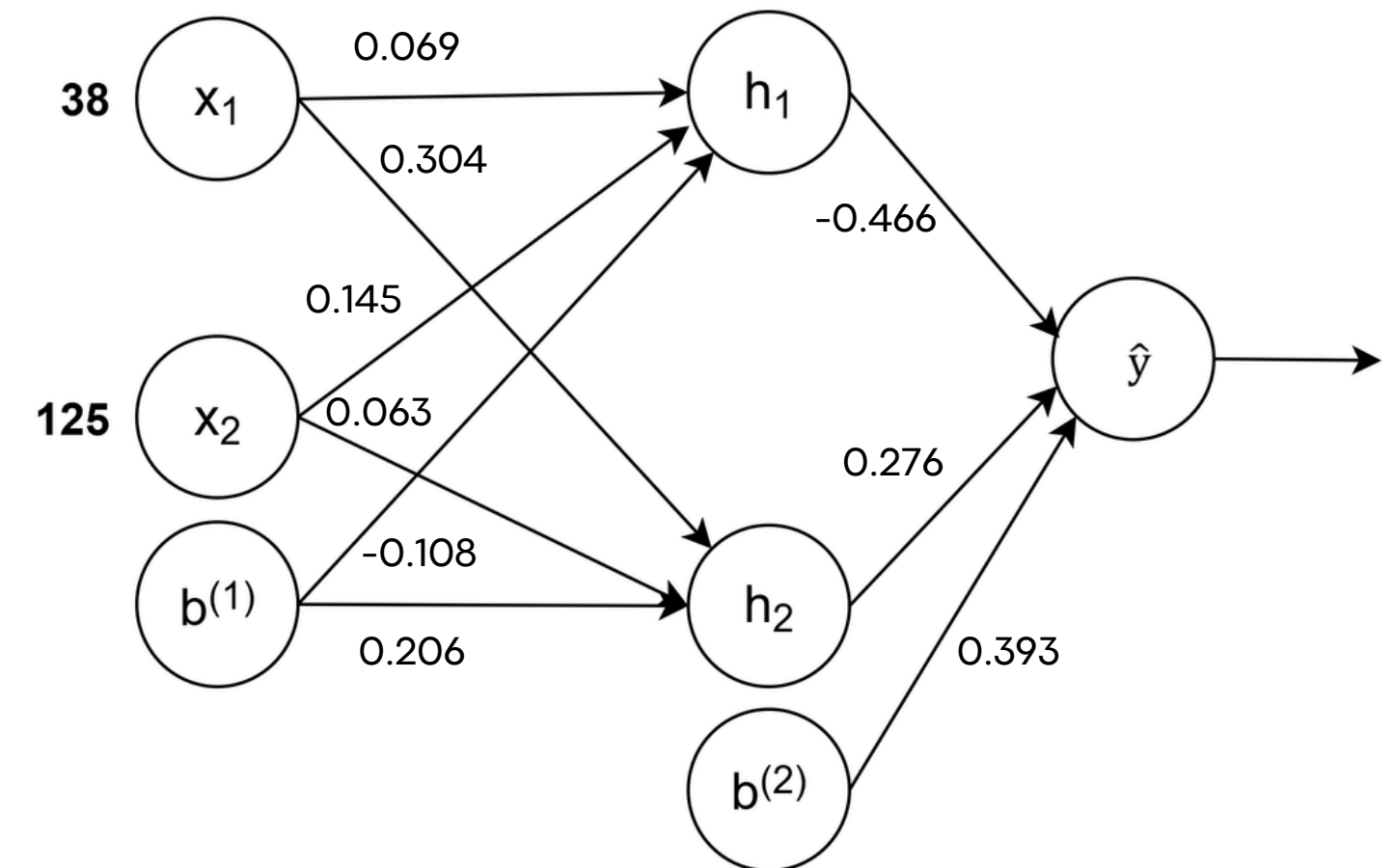
8.5.1.3 ตัวอย่างการคำนวณ MLP

จงแสดงวิธีการคำนวณ MLP เพื่อวินิจฉัยการป่วยเป็นโรคเบาหวาน
กำหนดให้

- Input Node = 2
- Hidden Node = 2 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.069 & 0.304 \\ 0.145 & 0.063 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.108, 0.206]$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = \begin{bmatrix} -0.466 \\ 0.276 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [0.393]$

ตารางที่ 8-4 : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาล ในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



วิธีทำ

จาก

$$h_1 = \varphi(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + b_1^{(1)})$$

$$h_2 = \varphi(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)})$$

$$\hat{y}_1 = \varphi(w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + b_1^{(2)})$$

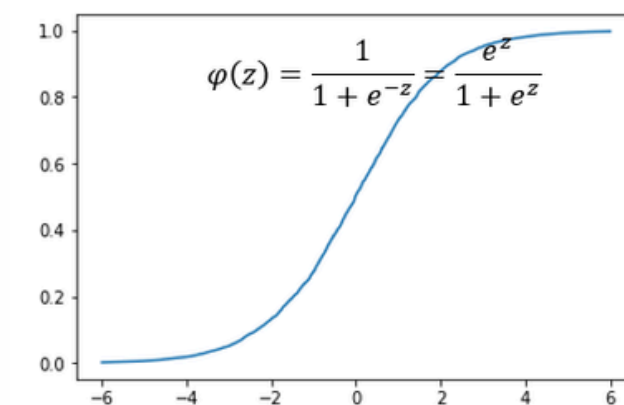
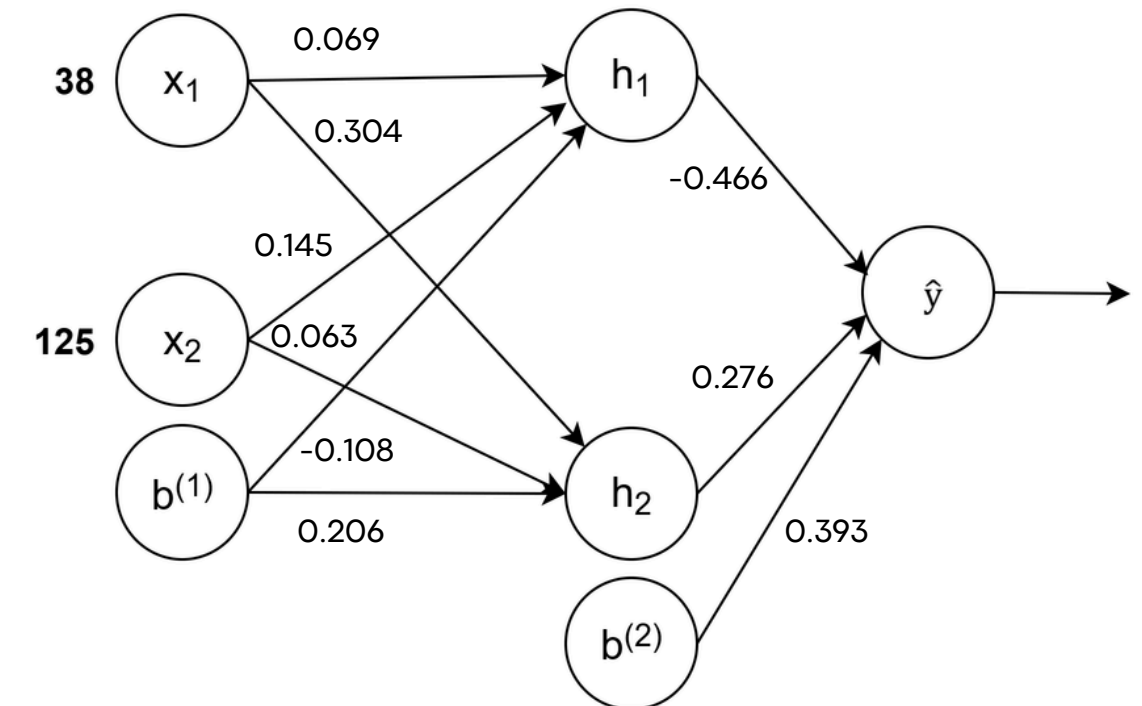
จะได้

$$\begin{aligned} h_1 &= \varphi(0.069 \times 38 + 0.145 \times 125 - 0.108) = \varphi(20.639) \\ &= \frac{1}{1 + e^{-20.639}} = 1 \end{aligned}$$

$$\begin{aligned} h_2 &= \varphi(0.304 \times 38 + 0.063 \times 125 + 0.206) = \varphi(19.633) \\ &= \frac{1}{1 + e^{-19.633}} = 1 \end{aligned}$$

$$\begin{aligned} \hat{y} &= \varphi(-0.466 \times 1 + 0.276 \times 1 + 0.393) = \varphi(0.203) \\ &= \frac{1}{1 + e^{-0.203}} = 0.55 \end{aligned}$$

เนื่องจากค่า prob ที่ได้มากกว่า 0.5 ทำให้ class label = 1
ดังนั้น ผู้ที่เข้ารับการตรวจโรค เป็นโรคเบาหวาน

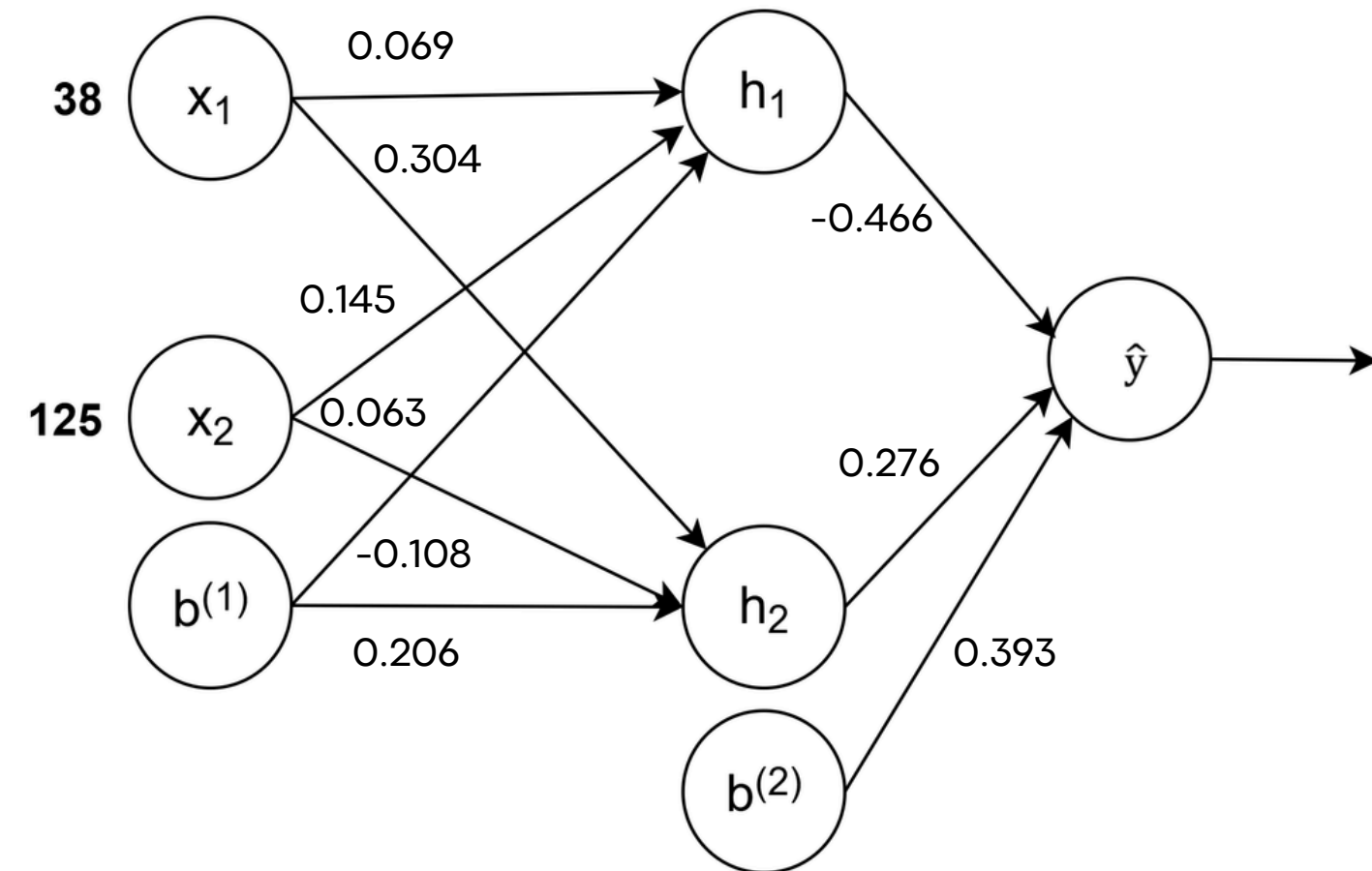


Logistic Sigmoid (logistic)

8.5.1.4 ตัวอย่างโค้ด MLP

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python Library
2 from sklearn.neural_network import MLPClassifier
3 import pandas as pd
4 import numpy as np
5
6 #Load Data
7 / df = pd.read_csv('data/diabetes.csv')
8 x_train = df[['age', 'blood_glucose']].to_numpy()
9 y_train = df['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
10 x_test = np.array([[38, 125]])
11
12 #Build Model
13 net = MLPClassifier(
14     hidden_layer_sizes=(2),
15     activation='logistic',
16     solver='sgd',
17     batch_size=len(x_train),
18     learning_rate_init=0.001,
19     max_iter=250,
20     random_state=0)
21
22 net.fit(x_train,y_train)
23 print(f'weight: {net.coefs_}') #show weight
24 print(f'bias: {net.intercepts_}') #show bias
25
26 #Test Model
27 prop = net.predict_proba(x_test)
28 print(f'probability: {prop[0][1]*100:.2f}%')
29 y_pred = net.predict(x_test)
30 print(f'Class Label: {y_pred[0]}') #show class label
```

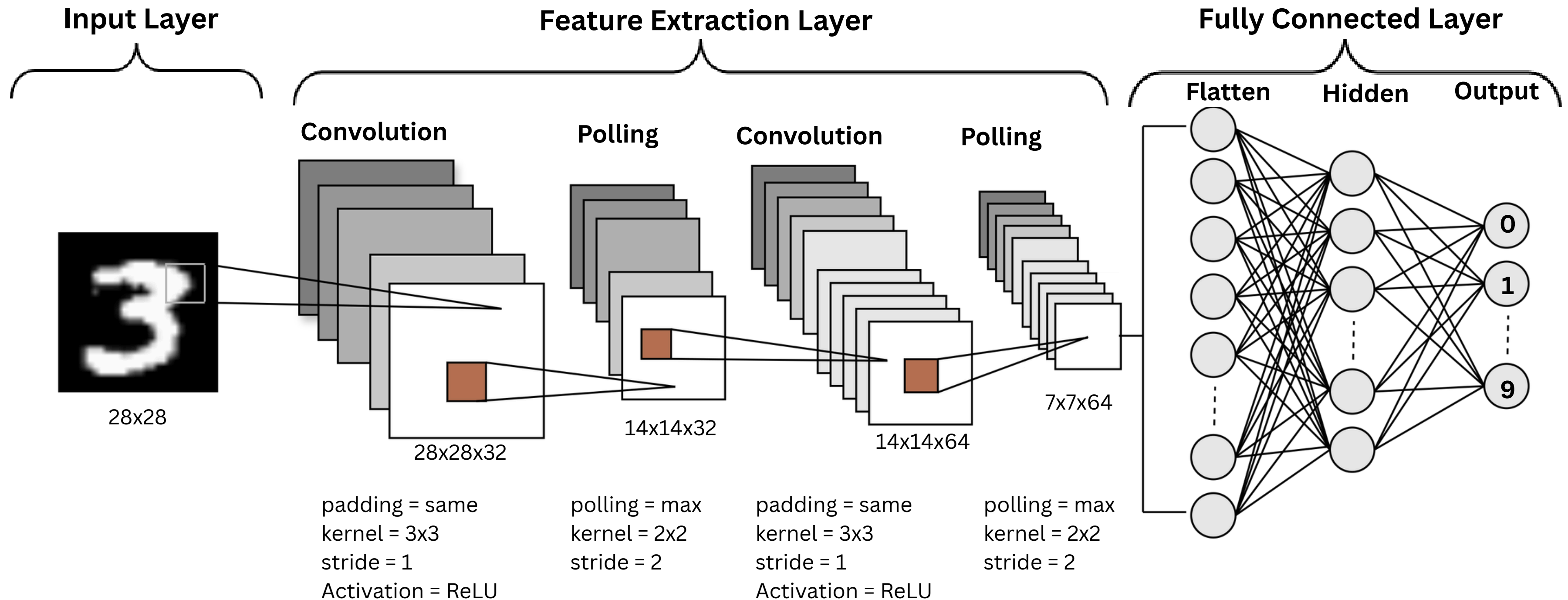


```
weight: [array([[0.06903344, 0.30430758],
                [0.14533409, 0.06344131]]), array([[ -0.46582199],
                [ 0.27584295]])]
bias: [array([ -0.10796832,  0.20632507]), array([0.39324777])]
probability: 55.06%
Class Label: 1
```

8.5.2 Convolutional Neural Network (CNN)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Feed forward neural network
รับข้อมูลเข้ามา (Input เช่น รูปภาพ) แล้วส่งผ่านชั้นต่าง ๆ เช่น Convolution Layer, Pooling Layer, Fully Connected Layer
- เรียนรู้โดยใช้ Backpropagation
ปรับปรุงความรู้ในทิศทางย้อนกลับ โดยใช้ error เพื่อคำนวณหาน้ำหนัก (Weight) ของฟิลเตอร์ (Filter/Kernel) และชั้น Fully Connected
- มีคุณสมบัติในการดึงคุณลักษณะ (Feature Extraction) อัตโนมัติ
ชั้น Convolution และ Pooling ช่วยดึงคุณลักษณะเชิงพื้นที่ (Spatial Features) จากข้อมูล เช่น ขอบ เส้น รูปร่าง และลักษณะซับซ้อนขึ้นเรื่อย ๆ
- นิยมใช้ในงานประมวลผลภาพและคอมพิวเตอร์วิชัน
เช่น Image Segmentation, Image Classification, Object Detection, Face Recognition

8.5.2.1 โครงสร้างของ CNN

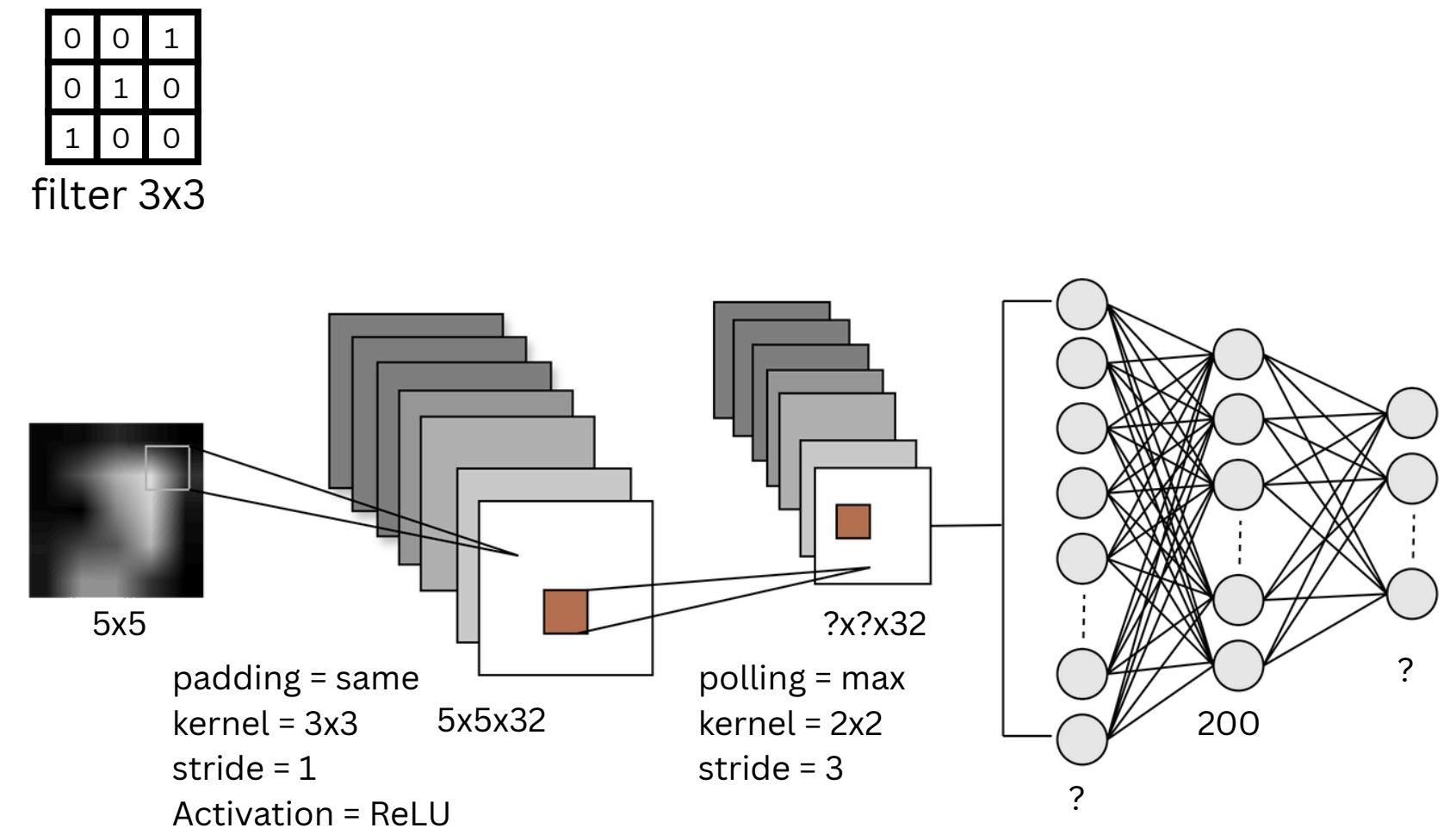


ภาพที่ 8-11 : โครงสร้างของ CNN

8.5.2.2 ตัวอย่างการคำนวณ CNN

กำหนดให้โครงสร้างของ CNN สำหรับทำนายตัวเลข 0-9 ดังนี้

- **Input Layer:** ใช้ภาพขาวดำ ขนาด 5×5 pixels (ใช้เลข 3 เป็นตัวอย่างการคำนวณ)
- **Feature Extraction Layer:** มี 1 convolution และ 1 polling layer ดังนี้
 - **Convolution:** มี 32 filter แต่ละ filter มีขนาด 3×3 pixel ดังแสดงในรูป, stride เท่ากับ 1 และมีการ padding เพื่อให้ได้ภาพขนาดเท่าเดิม, activation function คือ ReLU
 - **Pooling Layer:** ใช้ฟังก์ชัน max, ใช้จำนวน filter เท่าเดิม แต่ละ filter มีขนาด 2×2 pixel, stride เท่ากับ 3, ไม่มีการ padding
- **Fully Connected Layer:** มี 1 hidden layer
 - **Flatten Layer:** มีจำนวน node คำนวณมาจาก Layer ก่อนหน้า
 - **Hidden Layer:** มีจำนวน node เท่ากับ 200
 - **Output Layer:** มีจำนวน node เท่ากับ จำนวนตัวเลข 0-9



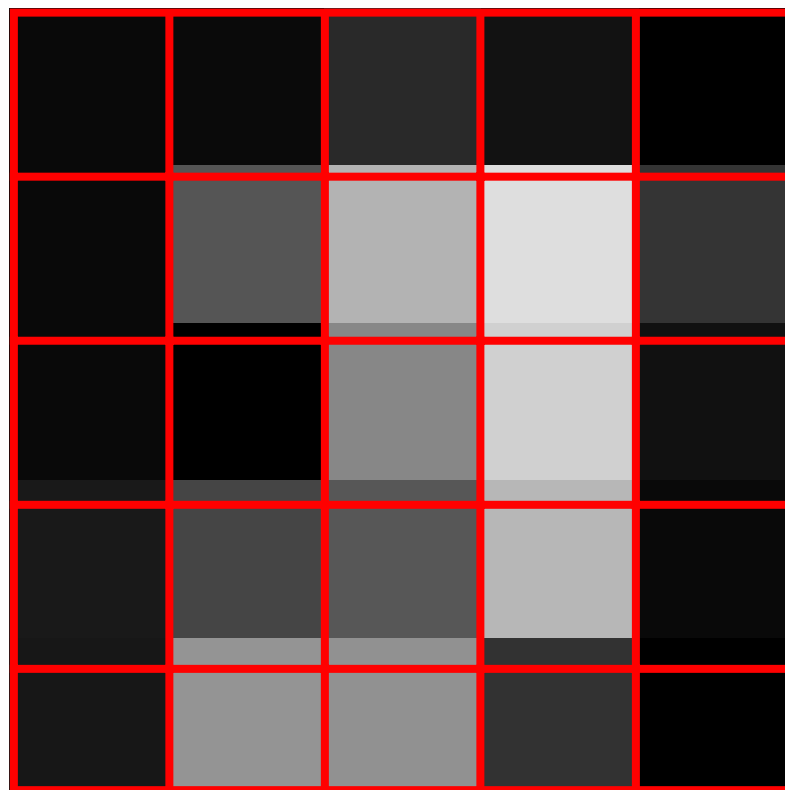
จงตอบคำถามต่อไปนี้

1. แสดงการคำนวณเพื่อหา feature map ใน convolution layer (ให้แสดงการคำนวณ ทั้งแบบไม่มี padding และมี padding ด้วย เพื่อดูความแตกต่าง)
2. แสดงการคำนวณเพื่อหา feature map ใน pooling Layer
3. แสดงการคำนวณหาจำนวน node ใน flatten layer
4. แสดงการคำนวณหาจำนวน weight ระหว่าง flatten layer กับ hidden layer
5. แสดงการคำนวณหาจำนวน node ใน output Layer
6. แสดงการคำนวณหาจำนวน weight ระหว่าง hidden layer กับ output layer

- **Input Layer**

เป็น Layer แรกของโครงสร้าง CNN มีจำนวน input nodes เท่ากับจำนวน pixels ของภาพ
ดังนั้น มีจำนวน input nodes = 25 (5x5)

- **หาค่าสีของภาพ**



ภาพขนาด 5x5

หาค่าสีของภาพ
[0-255]

9	23	65	51	9
9	74	172	218	52
13	18	129	202	22
25	90	99	151	13
23	148	145	48	2

ภาพขนาด 5x5

หมายเหตุ : ภาพดังกล่าวเป็นภาพของเลข 3 มีขนาด 5x5 ซึ่งทำขึ้นมาเพื่อแสดงตัวอย่างการคำนวณเท่านั้น
ในสถานการณ์จริงควรใช้ขนาดของภาพที่ใหญ่กว่านี้

- ทำ Normalization หรือ Standardization

สูตรการทำ standardization

$$\hat{x} = \frac{x - \mu}{\sigma}$$

เมื่อ $\hat{x}$ คือ ค่าสีใหม่ μ คือ ค่าเฉลี่ยของค่าสี
 x คือ ค่าสีเดิม σ คือ ส่วนเบี่ยงเบนมาตรฐานของค่าสี

สูตรการ normalization ให้อยู่ในช่วง 0-1

$$\hat{x} = \frac{x - \min}{\max - \min} \quad \text{หรือ} \quad = \frac{x}{\max}$$

เมื่อ $\min$ คือ ค่าต่ำสุดของค่าสี นิยมกำหนดเป็น 0
 $\max$ คือ ค่าสูงสุดของค่าสี นิยมกำหนดเป็น 255

ตัวอย่างการคำนวณ เมื่อ x เท่ากับ 9 จะได้

$$\hat{x} = \frac{x}{\max} = \frac{9}{255} = \frac{9}{255} = 0.035$$

9	23	65	51	9
9	74	172	218	52
13	18	129	202	22
25	90	99	151	13
23	148	145	48	2

ผลการ normalization
โดยใช้สูตร

$$\hat{x} = \frac{x}{\max}$$

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

- **Convolution Layer**

เป็น Layer ที่ดึงคุณลักษณะ (องค์ประกอบ) ของภาพออกมา โดยใช้ Filter หรือ Kernel จากโจทย์ กำหนดให้ filter มีขนาด 3x3 และมีค่าดังรูป (ค่าใน filter ก็คือ weight นั้นเอง)

0	0	1
0	1	0
1	0	0

filter 3x3

- **คำนวณโดยไม่ทำ padding**

- ภาพใหม่ที่ได้ (Feature Map) จะมีขนาดเล็กลง
- สามารถคำนวณหาขนาดของภาพใหม่ (Feature Map Size หรือ Output Size) ดังนี้

$$O = \frac{I - F + 2P}{S} + 1$$

เมื่อ O คือ Output Size
I คือ Input Size
F คือ Filter Size
P คือ Padding Size
S คือ Stride Size

ตัวอย่างการคำนวณหา Output Size เมื่อ I = 5, F = 3, P = 0 (กรณีไม่ทำ Padding), S = 1 จะได้

$$O = \frac{5 - 3 + 2(0)}{1} + 1 = 3$$

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

- ในการคำนวณ จะวาง Filter ทาบลงบนภาพ แล้วเลื่อน Filter จากซ้ายไปขวา และจากบนลงล่างตามค่า Stride
- ถ้าค่า Stride=1 จะเลื่อน Filter ไปทีละ 1 pixel

0	0	1
0	1	0
1	0	0

filter size = 3x3

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.051*1 + 0.29*1 + 0.255*1 = 0.596$$

0.596		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.071*1 + 0.675*1 + 0.2*1 = 0.946$

0.596	0.946	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 3

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.506*1 + 0.855*1 + 0.035*1 = 1.396$

0.596	0.946	1.396

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 4

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.098*1 + 0.071*1 + 0.675*1 = 0.844$$

0.596	0.946	1.396
0.844		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 5

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.353 \times 1 + 0.506 \times 1 + 0.855 \times 1 = 1.714$

0.596	0.946	1.396
0.844	1.714	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 6

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.388*1 + 0.792*1 + 0.204*1 = 1.384$

0.596	0.946	1.396
0.844	1.714	1.384

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 7

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.09 \times 1 + 0.353 \times 1 + 0.506 \times 1 = 0.949$

0.596	0.946	1.396
0.844	1.714	1.384
0.949		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 8

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.58*1 + 0.388*1 + 0.792*1 = 1.76$

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 9

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.569*1 + 0.592*1 + 0.086*1 = 1.247$

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	1.247

Output Size = 3x3

○ **คำนวณโดยทำ padding**

- ภาพใหม่ที่ได้ (Feature Map) จะมีขนาดเท่าเดิม
- สามารถคำนวณหา Padding Size ดังนี้

จากสมการเดิม $O = \frac{I - F + 2P}{S} + 1$

ให้ $O = I$ แล้วกลับสมการหาค่า P ได้ดังนี้

$$P = \frac{(I - 1)S - I + F}{2} = \frac{(5 - 1)1 - 5 + 3}{2} = 1 \quad \text{จะได้ Padding Size} = 1$$

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$0*1 + 0.035*1 + 0*1 = 0.035$

0.035				

Output Size = 5x5

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$0.035 \times 1 + 0.09 \times 1 + 0 \times 1 = 0.125$

0.035	0.125			

Output Size = 5x5

ทำการคำนวณไปจนถึงรอบที่ 25

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 25 จะได้ Feature Map รอบสุดท้าย ดังรูปด้านขวา

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$$0*1 + 0.008*1 + 0*1 = 0.008$$

Feature Map

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Output Size = 5x5

การคำนวณใน Convolution Layer มีทั้งหมด 32 filters ดังนั้น จะต้องคำนวณในลักษณะเดียวกันอีก 31 filters ซึ่ง filter แต่ละตัวจะแตกต่างกันไป เช่น

0	0	1
-1	1	-1
1	0	0

filter 3x3

1	0	-1
0	1	0
-1	0	1

filter 3x3

.....

0	-0.2	0.3
0.2	0.1	0.4
0.2	-0.1	0

filter 3x3

- **คำนวณ Activation Function**

จากโจทย์ กำหนดให้ activation function คือฟังก์ชัน ReLU นั่นคือ

$$\varphi(z) = \max(0, z)$$

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map **ก่อน**คำนวณ
Activation Function

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map **หลัง**คำนวณ
Activation Function

เนื่องจาก Feature Map ก่อนคำนวณ มีค่าเป็นบวกหมด ทำให้ Feature Map หลังคำนวณ มีค่าเท่าเดิม

• Polling Layer

- เป็นการย่อขนาดของ feature map โดยการเก็บเฉพาะค่าที่สำคัญของ feature ไว้
- ทำให้คำนวณเร็วขึ้น เนื่องจากภาพมีขนาดเล็กลง
- ลดการ overfitting เนื่องจาก โมเดลจะไม่จำรายละเอียดเกินไป

จากโจทย์ กำหนดให้ ใช้ฟังก์ชัน max, filter มีขนาด 2×2 pixel, stride เท่ากับ 3, ไม่มีการ padding

คำนวณหา Output Size ได้จาก $O = \frac{I - F + 2P}{S} + 1 = \frac{5 - 2 + 2(0)}{3} + 1 = 2$

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 3

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396
0.968	

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 4

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396
0.968	1.247

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

การคำนวณใน Pooling Layer มีทั้งหมด 32 filters เช่นเดียวกับ Convolution Layer ดังนั้น จะต้องคำนวณในลักษณะเดียวกันอีก 31 filters

- **Flatten Layer**

- เป็นการแปลงข้อมูลจากภาพหรือ Feature Map หลายมิติ ไปเป็นข้อมูล 1 มิติ
- เป็น Layer แรก ภายใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน node ใน flatten layer คือ

จำนวน Flatten Nodes = Width x Height x Depth

เมื่อ Width คือ ความกว้างของ Feature Map

Height คือ ความสูงของ Feature Map

Depth คือ จำนวน Filters

ดังนั้น

$$\text{จำนวน Flatten Nodes} = 2 \times 2 \times 32 = 128$$

- **Hidden Layer**

- เป็นการตีความว่า feature ไหนสำคัญต่อการจำแนก
- อยู่ระหว่าง Flatten Layer และ Output Layer ใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน weight ระหว่าง flatten layer กับ hidden layer

จำนวน Weights = (จำนวน Flatten Nodes + 1) x จำนวน Hidden Nodes

เมื่อ จำนวน Hidden Nodes เท่ากับ 200

ดังนั้น

$$\text{จำนวน Weights} = (128+1) \times 200 = 25,800$$

- **Output Layer**

- เป็นการจำแนกภาพหรือกลุ่มของข้อมูล
- เป็น Layer สุดท้าย ใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน weight ระหว่าง hidden layer กับ output layer

จำนวน Weights = (จำนวน Hidden Nodes + 1) x จำนวน Output Nodes

เมื่อ จำนวน Hidden Nodes เท่ากับ 200

จำนวน Output Nodes เท่ากับ 10 (0-9)

ดังนั้น

$$\text{จำนวน Weights} = (200+1) \times 10 = 2,010$$

สรุปคำตอบ

1. Feature Map ที่ได้จากการไม่ทำ Padding และ ทำ Padding

- Feature Map ที่ได้จากการไม่ทำ Padding เป็นดังนี้

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	1.247

Output Size = 3x3

- Feature Map ที่ได้จากการทำ Padding เป็นดังนี้

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Output Size = 5x5

2. Feature Map ใน Polling Layer

0.596	1.396
0.968	1.247

Output Size = 2x2

3. จำนวน Node ใน Flatten Layer เท่ากับ 128 โหนด
4. จำนวน Weight ระหว่าง Flatten Layer กับ Hidden Layer เท่ากับ 25,800
5. จำนวน Node ใน Output Layer เท่ากับ 10 โหนด
6. จำนวน Weight ระหว่าง Hidden Layer กับ Output Layer เท่ากับ 2,010

8.5.2.3 ตัวอย่างโค้ด CNN

ติดตั้ง Libraries

```
1 #ติดตั้ง TensorFlow สำหรับการสร้างโมเดล Deep Learning
2 %pip install tensorflow
3
4 #ติดตั้ง Keras ทำให้สามารถสร้างโมเดล Deep Learning ได้ง่ายขึ้น
5 %pip install keras
6
7 #ติดตั้ง OpenCV สำหรับการประมวลผลภาพและการทำงานกับวิดีโอ
8 %pip install opencv-python==4.13.0.92
```

Python

○ ส่วนของการสร้างโมเดล CNN

```
1  # Import python library
2  from tensorflow.keras.utils import load_img, img_to_array
3  from tensorflow.keras.models import Sequential
4  from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
5  from tensorflow.keras.optimizers import Adam
6  from tensorflow.keras.utils import to_categorical
7  from sklearn.model_selection import StratifiedKFold
8  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
9  import os
10 import numpy as np
11
12 # Define variables
13 path = "data/celeb"
14 model_path = "model/cnn_celeb.keras" # Path to save the final model
15 label_index_path = "model/cnn_label_index.txt" # Path to save mapping between class names and indices
16 img_size = (224, 224)
17
18 # Define Loading data function
19 def load_data():
20     images, labels = [], []
21     label_index = {name: idx for idx, name in enumerate(os.listdir(path))}
22     print(f"Label-index Mapping: {label_index}")
23
24     for label in os.listdir(path):
25         folder_path = os.path.join(path, label)
26         for file in os.listdir(folder_path):
27             img_path = os.path.join(folder_path, file)
28             img = load_img(img_path, target_size=img_size)
29             img = img_to_array(img) / 255.0 # Normalize
30             images.append(img) # Create images dataset
31             labels.append(label_index[label]) # Create labels based on folder names
32
33     return np.array(images), np.array(labels), label_index
34
```

```

35 # Define creating model function
36 def create_cnn_model(input_shape, num_classes):
37     model = Sequential([
38         Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=input_shape),
39         MaxPooling2D((2, 2)),
40
41         Conv2D(64, (3, 3), padding='same', activation='relu'),
42         MaxPooling2D((2, 2)),
43
44         Conv2D(128, (3, 3), padding='same', activation='relu'),
45         MaxPooling2D((2, 2)),
46
47         Flatten(),
48         Dense(256, activation='relu'),
49         Dropout(0.5),
50         Dense(num_classes, activation='softmax')
51     ])
52
53     model.compile(optimizer=Adam(learning_rate=0.0001),
54                 loss='categorical_crossentropy',
55                 metrics=['accuracy'])
56     return model
57
58
59 # Create a datasets
60 X, y, label_index = load_data()
61 y = to_categorical(y) # Convert labels to one-hot encoding
62
63 # Perform 3-fold cross-validation
64 ss = StratifiedKFold(n_splits=3, shuffle=True, random_state=42).split(X, np.argmax(y, axis=1))
65 input_shape = (224, 224, 3)
66 num_classes = y.shape[1]
67

```

```

68 results = []
69 for fold, (train_idx, val_idx) in enumerate(ss):
70     print(f"Training Fold {fold+1}...")
71     X_train, X_val = X[train_idx], X[val_idx]
72     y_train, y_val = y[train_idx], y[val_idx]
73
74     # Create and train model
75     model = create_cnn_model(input_shape, num_classes)
76     model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=10, batch_size=32, verbose=1)
77
78     # Evaluate model
79     y_pred = model.predict(X_val)
80     y_pred_classes = np.argmax(y_pred, axis=1)
81     y_true_classes = np.argmax(y_val, axis=1)
82
83     accuracy = accuracy_score(y_true_classes, y_pred_classes)
84     precision = precision_score(y_true_classes, y_pred_classes, average='weighted')
85     recall = recall_score(y_true_classes, y_pred_classes, average='weighted')
86     f1 = f1_score(y_true_classes, y_pred_classes, average='weighted')
87
88     results.append({
89         "fold": fold+1,
90         "accuracy": accuracy,
91         "precision": precision,
92         "recall": recall,
93         "f1_score": f1
94     })
95
96

```

```

97 # Display the evaluation results in each fold
98 print("\n=== Cross-Validation Results ===")
99 for result in results:
100     print(f"Fold {result['fold']}: ")
101         f"Accuracy={result['accuracy']:.4f}, "
102         f"Precision={result['precision']:.4f}, "
103         f"Recall={result['recall']:.4f}, "
104         f"F1 Score={result['f1_score']:.4f}")
105
106 # Display the average results across all folds
107 avg_accuracy = np.mean([r['accuracy'] for r in results])
108 avg_precision = np.mean([r['precision'] for r in results])
109 avg_recall = np.mean([r['recall'] for r in results])
110 avg_f1_score = np.mean([r['f1_score'] for r in results])
111
112 print(f"\nAverage Accuracy: {avg_accuracy:.4f}")
113 print(f"Average Precision: {avg_precision:.4f}")
114 print(f"Average Recall: {avg_recall:.4f}")
115 print(f"Average F1 Score: {avg_f1_score:.4f}")
116
117 # Retrain the final model on the entire dataset
118 print("\nRetraining final model on the entire dataset...")
119 final_model = create_cnn_model(input_shape, num_classes)
120 final_model.fit(X, y, epochs=10, batch_size=32, verbose=1)
121
122 # Save the model in .keras format
123 final_model.save(model_path)
124 print(f"\nFinal model saved to {model_path}")
125
126 # Save the labels to a .txt file
127 with open(label_index_path, 'w') as f:
128     for label, idx in label_index.items():
129         f.write(f"{label}:{idx}\n")
130 print(f"Labels saved to {label_index_path}")

```


ดูโครงสร้างของ CNN Model

```
1 model.summary()
```

✓ 0.0s

Python

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d_12 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_13 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_14 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten_4 (Flatten)	(None, 100352)	0
dense_8 (Dense)	(None, 256)	25,690,368
dropout_4 (Dropout)	(None, 256)	0
dense_9 (Dense)	(None, 10)	2,570

Total params: 77,358,560 (295.10 MB)

Trainable params: 25,786,186 (98.37 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 51,572,374 (196.73 MB)

○ ส่วนของการทดสอบโมเดล CNN ด้วยภาพ

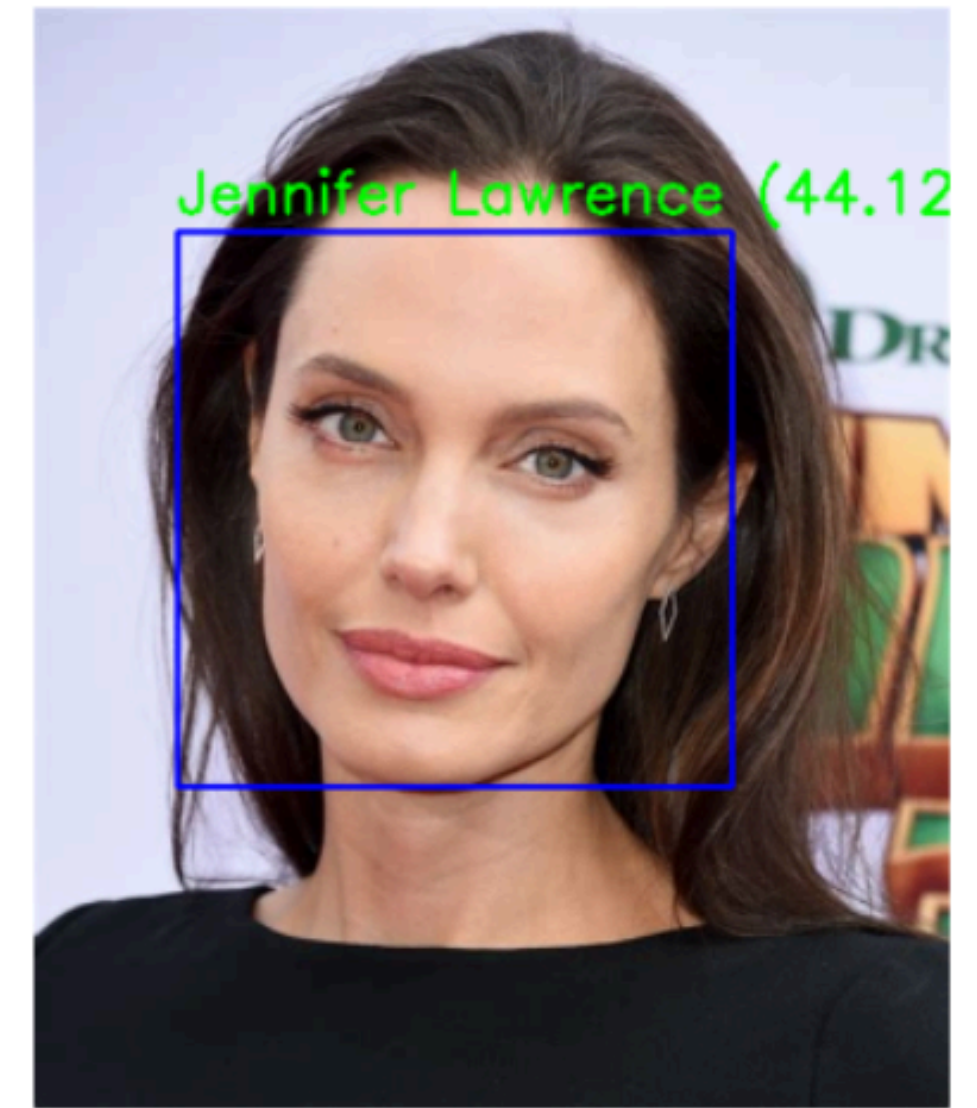
```
1 from tensorflow.keras.models import load_model # type: ignore
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Define variables
7 model_path = "model/cnn_celeb.keras" # Path of the final model
8 label_index_path = "model/cnn_label_index.txt" # Path of mapping between class names and indices
9 image_path = r"data/celeb/Angelina Jolie/001_fe3347c0.jpg" # Path to the input image
10
11 # Load the trained model
12 model = load_model(model_path)
13
14 # Load the label-index mapping from the .txt file
15 label_index = {}
16 index_label = {}
17 with open(label_index_path, 'r') as f:
18     for line in f:
19         label, idx = line.strip().split(":")
20         idx = int(idx)
21         label_index[label] = idx
22         index_label[idx] = label # Reverse the mapping
23
24 #print(f"Loaded label-index: {label_index}")
25 #print(f"Loaded index-label: {index_label}")
26
27 # Load Haar Cascade for face detection
28 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")
29
30 # Load and process the image
31 image = cv2.imread(image_path)
32
33 if image is None:
34     print(f"Error: Could not load image from {image_path}.")
35     exit()
36
37 # Convert the image to grayscale for face detection
38 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
39
```

```

40 # Detect faces in the image
41 faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minSize=(30, 30))
42
43 if len(faces) == 0:
44     print("No face detected.")
45 else:
46     for (x, y, w, h) in faces:
47         # Draw a rectangle around the face
48         cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)
49
50         # Crop the face from the image
51         face_region = image[y:y+h, x:x+w]
52
53         # Convert the face region to RGB and resize it for the model input
54         rgb_face = cv2.cvtColor(face_region, cv2.COLOR_BGR2RGB)
55         resized_face = cv2.resize(rgb_face, (224, 224)) / 255.0 # Normalize
56         input_face = np.expand_dims(resized_face, axis=0) # Add batch dimension
57
58         # Predict using the model
59         predictions = model.predict(input_face)
60         index = np.argmax(predictions) # Get the index of the highest confidence
61         label = index_label.get(index, "Unknown") # Use index_label to get the label
62         confidence = predictions[0][index] # Confidence score
63
64         # Add the label and confidence to the image
65         display_text = f"{label} ({confidence*100:.2f}%)"
66         cv2.putText(image, display_text, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
67
68 # Convert BGR to RGB for displaying with matplotlib
69 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
70
71 # Show the image with detected faces and predictions using matplotlib
72 plt.imshow(image_rgb)
73 plt.axis('off') # Hide axes
74 plt.title("Image - Face Recognition")
75 plt.show()

```

Image - Face Recognition



○ ส่วนของการทดสอบโมเดล CNN ด้วย Webcam

```
1  from tensorflow.keras.models import load_model # type: ignore
2  import cv2
3  import numpy as np
4
5  # Define variables
6  model_path = "model/cnn_celeb.keras" # Path of the final model
7  label_index_path = "model/cnn_label_index.txt" # Path of mapping between class names and indices
8
9  # Load the trained model
10 model = load_model(model_path)
11
12 # Load the label-index mapping from the .txt file
13 label_index = {}
14 index_label = {}
15 with open(label_index_path, 'r') as f:
16     for line in f:
17         label, idx = line.strip().split(":")
18         idx = int(idx)
19         label_index[label] = idx
20         index_label[idx] = label # Reverse the mapping
21
22 #print(f"Loaded label-index: {label_index}")
23 #print(f"Loaded index-label: {index_label}")
24
25 # Load Haar Cascade for face detection
26 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
27
28 # Webcam feed
29 cap = cv2.VideoCapture(0) # Open webcam (0 is the default camera)
30
31 # Check if the webcam opened successfully
32 if not cap.isOpened():
33     print("Error: Could not access the webcam.")
34     exit()
```

```

35
36 print("Press 'q' to quit the webcam.")
37 while True:
38     # Read a frame from the webcam
39     ret, frame = cap.read()
40     if not ret:
41         print("Failed to grab a frame.")
42         break
43
44     # Convert the frame to grayscale for face detection
45     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46
47     # Detect faces in the frame
48     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minSize=(30, 30))
49
50     if len(faces) == 0:
51         print("No face detected.")
52     else:
53         for (x, y, w, h) in faces:
54             # Draw a rectangle around the face
55             cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
56
57             # Crop the face from the frame
58             face_region = frame[y:y+h, x:x+w]
59
60             # Convert the face region to RGB and resize it for the model input
61             rgb_face = cv2.cvtColor(face_region, cv2.COLOR_BGR2RGB)
62             resized_face = cv2.resize(rgb_face, (224, 224)) / 255.0 # Normalize
63             input_face = np.expand_dims(resized_face, axis=0) # Add batch dimension
64
65             # Predict using the model
66             predictions = model.predict(input_face)
67             index = np.argmax(predictions)
68             label = index_label.get(index, "Unknown") # Use idx_to_label to get the label
69             confidence = predictions[0][index]
70
71             # Add the label and confidence to the frame
72             display_text = f"{label} ({confidence*100:.2f}%)"
73             cv2.putText(frame, display_text, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

```

```
74
75     # Show the frame in a window
76     cv2.imshow("Webcam - Face Recognition", frame)
77
78     # Exit the webcam feed when 'q' is pressed
79     if cv2.waitKey(1) == ord('q'):
80         break
81
82 # Release the webcam and close windows
83 cap.release()
84 cv2.destroyAllWindows()
```

8.5.2.4 Pre-trained CNN Model

Pre-trained model คือ โมเดลที่ผ่านการ Train บน Datasets ขนาดใหญ่มาแล้ว และสามารถนำมา Fine-tune หรือปรับใช้กับงานใหม่ได้ โดยไม่ต้องเริ่ม Train ใหม่จากศูนย์

Pre-trained CNN model คือ Pre-trained model ที่มีโครงสร้างแบบ CNN

ตารางที่ 8-5 : ตัวอย่างของ Pre-trained CNN model

Model	Main Version	Description
VGGNet	11, 13, 16, 19	โครงสร้างเข้าใจง่ายคล้ายกับ Classic CNN
ResNet	18, 34, 50, 101, 152	แก้ปัญหา Vanishing Gradient
Inception	V1–V4, Inception-ResNet	มีความหลากหลายในการทำ Feature Extraction
DenseNet	121, 161, 169, 201	นำ Feature Maps จากทุกชั้นก่อนหน้า มาใช้ซ้ำในชั้นถัดไป
MobileNet	V1–V4	ออกแบบมาสำหรับอุปกรณ์ที่มีทรัพยากรจำกัด (เช่น มือถือ)
EfficientNet	B0–B7, V2	เน้นการค้นหาสมดุลระหว่างประสิทธิภาพและความแม่นยำ

8.5.3 Recurrent Neural Network (RNN)

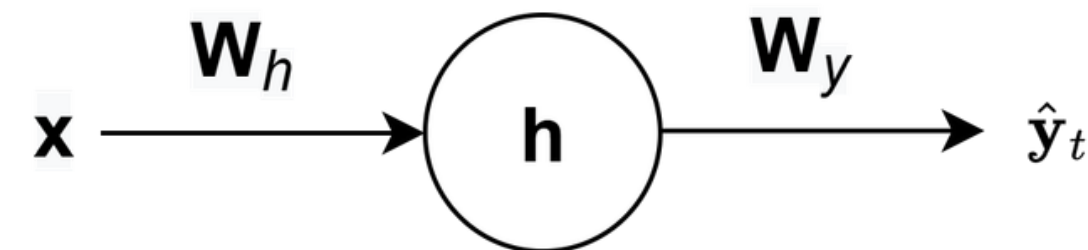
- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Recurrent neural network
รับข้อมูลเข้ามาแบบลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือสัญญาณเวลา โดยส่งข้อมูลผ่านชั้นต่าง ๆ เช่น Recurrent Layer (เช่น Simple RNN, LSTM, GRU) และ Fully Connected Layer
- เรียนรู้โดยใช้ Backpropagation Through Time (BPTT)
เป็นการปรับปรุงความรู้หรือน้ำหนัก (Weight) ย้อนกลับตามลำดับเวลา โดยใช้ค่า error จาก output แต่ละช่วงเวลามาคำนวณย้อนกลับ
- มีคุณสมบัติในการจดจำลำดับและบริบท (Sequence and Context Memory)
สามารถเก็บข้อมูลจากขั้นตอนก่อนหน้า (Previous Steps) เพื่อใช้ในการประมวลผลขั้นตอนถัดไป ทำให้เข้าใจความสัมพันธ์ในลำดับข้อมูลได้
- นิยมใช้ในงานประมวลผลข้อมูลลำดับ (Sequential Data Processing)
เช่น Time Series Forecasting, Natural Language Processing (NLP), Machine Translation

8.5.3.1 โครงสร้างของ RNN

- **Feedforward Network (นูนวน)**

$$\mathbf{h} = \varphi_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}} = \varphi_y(\mathbf{W}_y \mathbf{h} + \mathbf{b}_y)$$

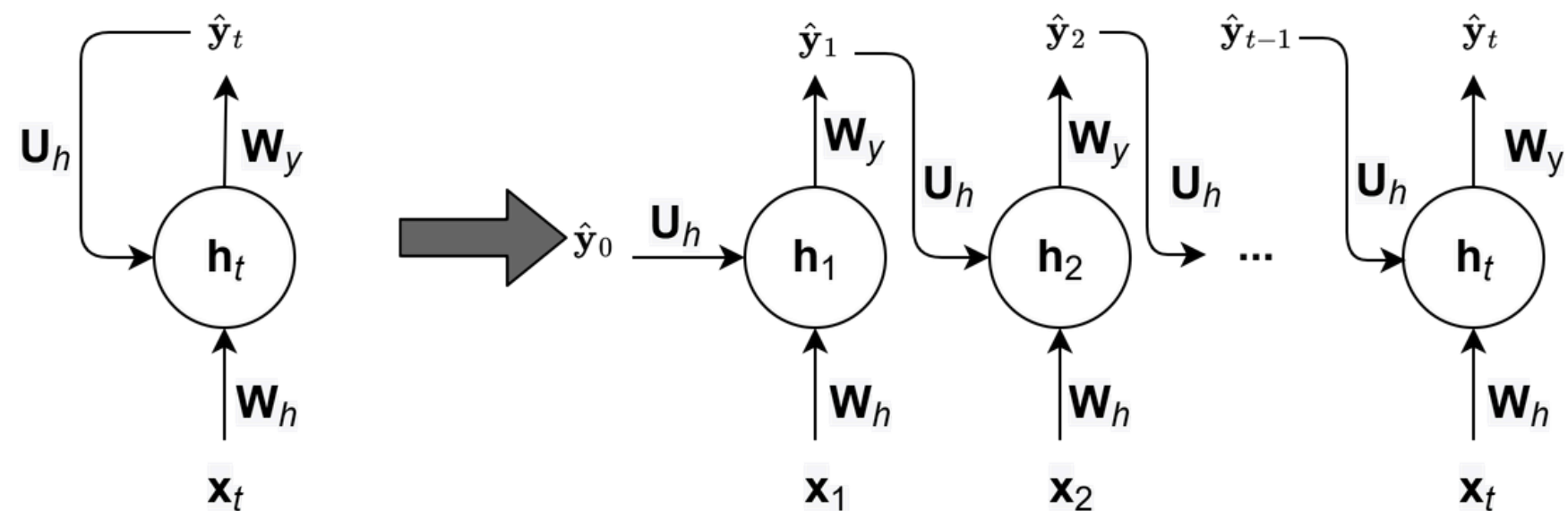


ภาพที่ 8-12 : โครงสร้างของ Feedforward Network

- **Jordan Recurrent Network**

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{y}_{t-1} + \mathbf{b}_h)$$

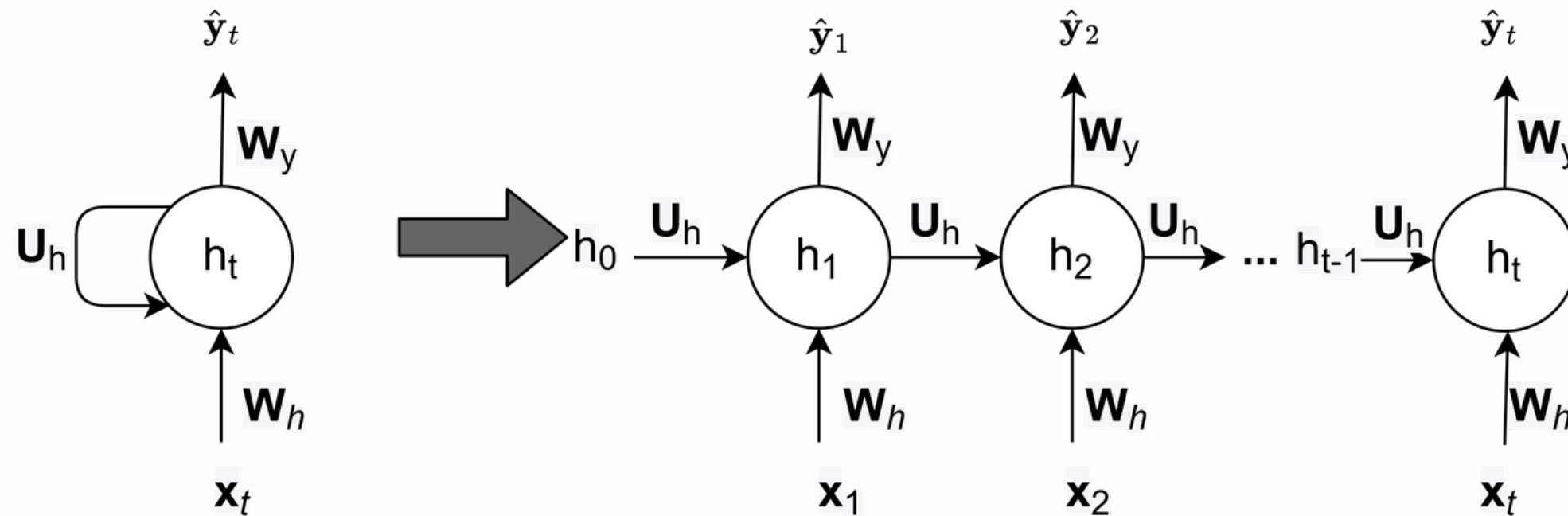
$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ภาพที่ 8-13 : โครงสร้างของ Jordan Recurrent Network

- **Elman Recurrent Network**

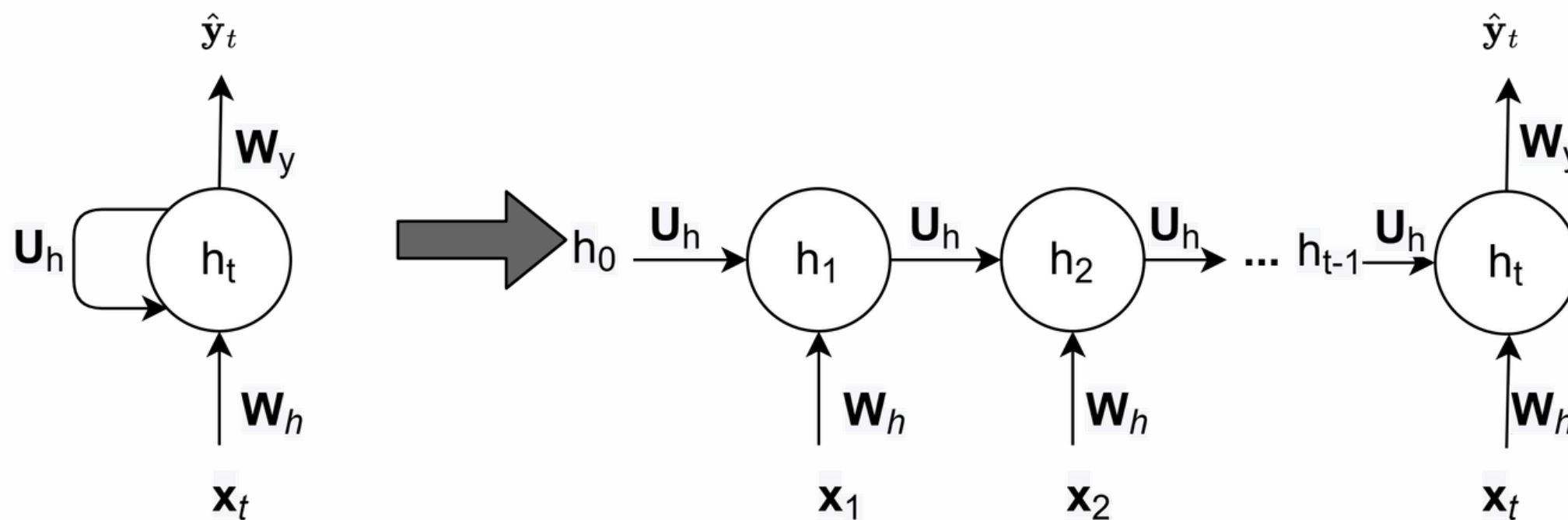
- แบบ Many to Many



$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

- แบบ Many to One



ภาพที่ 8-14 : โครงสร้างของ Elman Recurrent Network

8.5.3.2 ตัวอย่างการคำนวณ RNN (Elman Network)

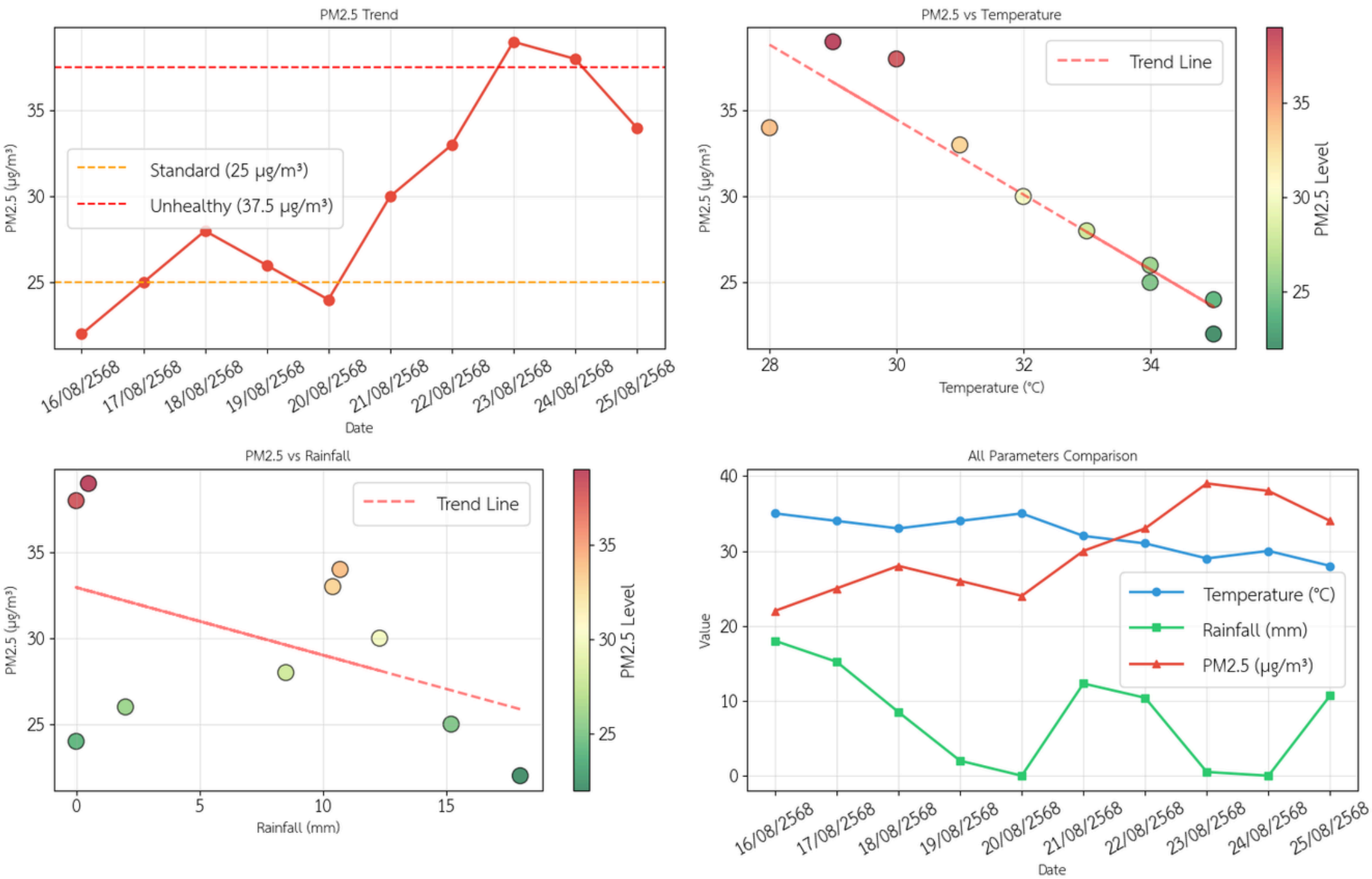
จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายแนวโน้มของค่า PM2.5 (สูงขึ้น/ต่ำลง) ณ วันที่ 19/8/2568

กำหนดให้

- Input Node = 3,
- Hidden Node = 1 (ReLU)
- Output Node = 1 (Logistic Sigmoid)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h=[0.2, -0.1, 0.3]$
- Hidden Recurrent Weight: $\mathbf{U}_h=0.4$
- Hidden Bias: $\mathbf{b}_h=0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y=0.8$
- Output Bias: $\mathbf{b}_y=-0.3$

ตารางที่ 8-6 : ชุดข้อมูล PM2.5

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)	change
16/8/2568	35	18	22	ขึ้น
17/8/2568	34	15.2	25	ขึ้น
18/8/2568	33	8.5	28	ขึ้น
19/8/2568	34	2	26	ลง
20/8/2568	35	0	24	ลง
21/8/2568	32	12.3	30	ขึ้น
22/8/2568	31	10.4	33	ขึ้น
23/8/2568	29	0.5	39	ขึ้น
24/8/2568	30	0	38	ลง
25/8/2568	28	10.7	34	ลง



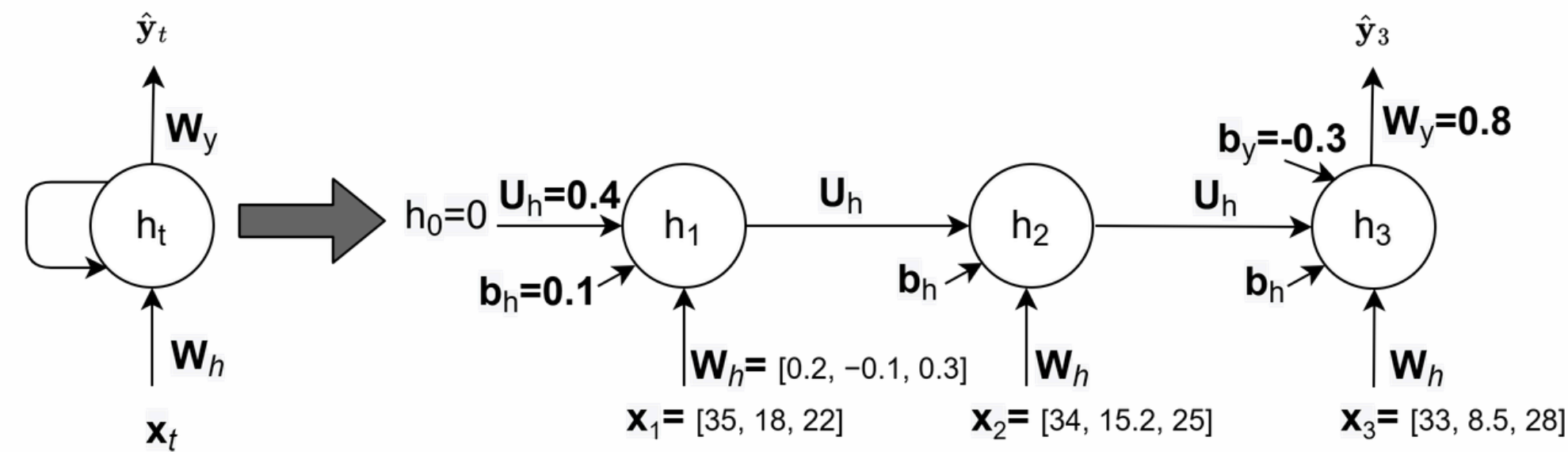
ภาพที่ 8-15 : ชุดข้อมูล PM2.5

วิธีทำ

กำหนดให้

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ค่า $\mathbf{W}_h$, $\mathbf{U}_h$, และ $\mathbf{b}_h$ จะเท่าเดิม

	วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)	change
$\mathbf{X}_1$	16/8/2568	35	18	22	ขึ้น
$\mathbf{X}_2$	17/8/2568	34	15.2	25	ขึ้น
$\mathbf{X}_3$	18/8/2568	33	8.5	28	ขึ้น

- คำนวณใน Hidden Layer (State)

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

เมื่อ $t=1$ (16/08/2568)

$$Z_1 = 0.2 \times 35 + (-0.1) \times 18 + 0.3 \times 22 + 0.4 \times 0 + 0.1 = 11.9$$

$$\mathbf{h}_1 = \max(11.9, 0) = 11.9$$

เมื่อ $t=2$ (17/08/2568)

$$Z_2 = 0.2 \times 34 + (-0.1) \times 15.2 + 0.3 \times 25 + 0.4 \times 11.9 + 0.1 = 17.64$$

$$\mathbf{h}_2 = \max(17.64, 0) = 17.64$$

เมื่อ $t=3$ (18/08/2568)

$$Z_3 = 0.2 \times 33 + (-0.1) \times 8.5 + 0.3 \times 28 + 0.4 \times 17.64 + 0.1 = 21.306$$

$$\mathbf{h}_3 = \max(21.306, 0) = 21.306$$

เมื่อ $\mathbf{W}_h = [0.2, -0.1, 0.3]$

$\mathbf{U}_h = 0.4$ $\mathbf{h}_0 = 0$ $\mathbf{b}_h = 0.1$

$\mathbf{x}_1 = [35, 18, 22]$

$\mathbf{x}_2 = [34, 15.2, 25]$

$\mathbf{x}_3 = [33, 8.5, 28]$

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำ ฝน (mm)	PM 2.5 (µg/m³)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28

- **คำนวณใน Output Layer**

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

$$\hat{\mathbf{y}}_3 = \text{ผลการทำนายของวันที่ 19/08/2568}$$

$$\hat{\mathbf{y}}_3 = \varphi_y(0.8 \times 21.306 - 0.3) = \varphi_y(16.7448)$$

$$\hat{\mathbf{y}}_3 = \frac{1}{1 + e^{-16.7448}} = 0.999999995$$

ดังนั้น ในวันที่ 19/8/2568 มีแนวโน้มว่าค่า PM2.5 จะสูงขึ้น

เมื่อ $\mathbf{W}_y = 0.8$
 $\mathbf{h}_3 = 21.306$
 $\mathbf{b}_y = -0.3$

8.5.4 Long Short-Term Memory Network (LSTM)

- **เป็น Recurrent Neural Network ชนิดพิเศษ**

ออกแบบมาเพื่อแก้ปัญหาเรื่อง การจำระยะยาว (Long-Term Memory) ของ RNN โดยสามารถรับข้อมูลแบบลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือสัญญาณเวลา แล้วส่งข้อมูลผ่านโครงสร้างหลักที่มี Cell State และ Gate ต่าง ๆ (Input Gate, Forget Gate, Output Gate) เพื่อควบคุมการเก็บและลืมข้อมูล ก่อนจะส่งต่อไปยัง Fully Connected Layer เพื่อคำนวณผลลัพธ์สุดท้าย

- **เรียนรู้โดยใช้ Backpropagation Through Time (BPTT)**

เป็นการปรับปรุงความรู้หรือน้ำหนัก (Weight) ย้อนกลับตามลำดับเวลา (Time Step) โดยใช้ค่า error ที่คำนวณจาก Actual/Predicted Outputs ในแต่ละช่วงเวลามาคำนวณย้อนกลับ เช่นเดียวกับกับ RNN แต่มีการจัดการปัญหา Gradient Vanishing โดยใช้โครงสร้างแบบ “Cell” ประกอบด้วย Gate 3 ประเภท ดังนี้

- Forget Gate: ควบคุมข้อมูลที่ควรถูกลืมจากสถานะก่อนหน้า
- Input Gate: ควบคุมข้อมูลใหม่ที่จะถูกเพิ่มเข้าไปในหน่วยความจำ
- Output Gate: ควบคุมข้อมูลที่จะถูกส่งออกไปยังชั้นถัดไป

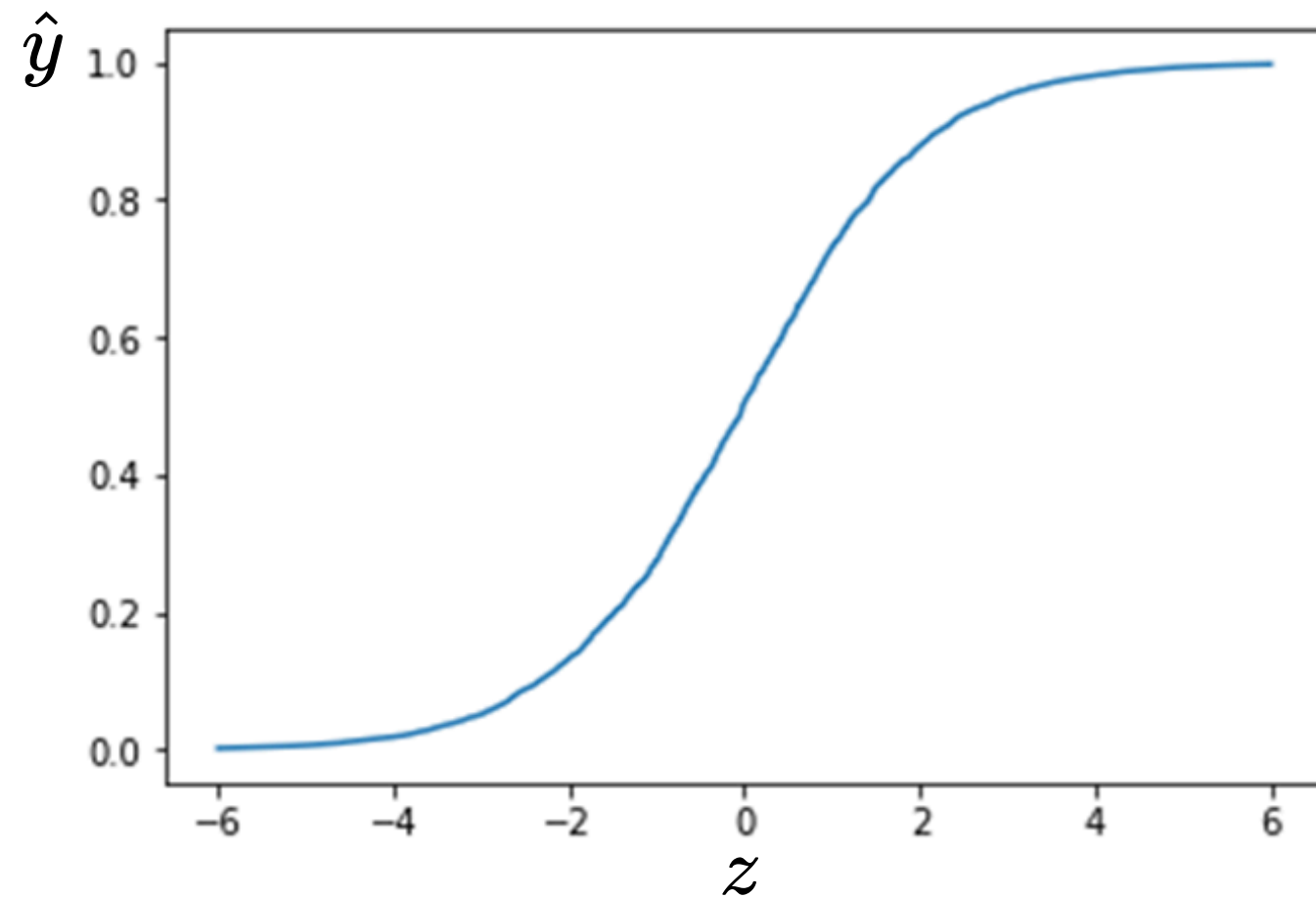
- **นิยมใช้ในงานประมวลผลข้อมูลลำดับ (Sequential Data Processing)**

เช่น Time Series Forecasting, Natural Language Processing (NLP), Machine Translation

ปัญหา Vanishing Gradient เป็นปัญหาที่เกิดขึ้นในระบบการ Backpropagation โดยเฉพาะใน RNN ที่มีจำนวน Time Step ยาว เมื่อมีการคำนวณย้อนกลับตามลำดับของข้อมูล (Backpropagation Through Time) ค่ากราดีเอนต์ (Gradient) จะถูก “คูณซ้ำ ๆ” ด้วยค่าที่มีขนาดเล็กกว่า 1 เช่น $0.9 \times 0.8 \times 0.5 \times \dots$ ส่งผลให้ค่า Gradient ค่อย ๆ เล็กลงจนเกือบเป็นศูนย์ ทำให้เครือข่าย (Network) ไม่สามารถปรับน้ำหนัก (Weight) ได้อย่างมีประสิทธิภาพ และไม่สามารถเรียนรู้จากข้อมูลในช่วงเวลายาว ๆ ได้อย่างเหมาะสม

8.6 Logistic Regression (LR)

- อาศัยแนวคิดของความน่าจะเป็นและฟังก์ชันโลจิสติก (Sigmoid) เพื่อประมาณค่าความน่าจะเป็น ของการเกิด เหตุการณ์ของ ตัวแปรตาม จาก ตัวแปรต้น
- ค่าที่ได้จะอยู่ในช่วง 0 ถึง 1 ซึ่งแทนค่าความน่าจะเป็น (Probability)
- Sigmoid Function ถือเป็นหัวใจสำคัญที่ใช้แปลงค่าจากสมการเส้นตรง (Linear) ให้กลายเป็นเส้นโค้งรูปตัว S เพื่อจำกัดค่าให้อยู่ระหว่าง 0 กับ 1
- การตัดสินใจจะใช้ ค่า Threshold เช่น ถ้าความน่าจะเป็น ≥ 0.5 ให้ถือว่าเป็น class 1 ถ้าต่ำกว่าให้เป็น class 0



ภาพที่ 8-16 : Logistic Sigmoid Function

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

$$P(y = 1|x) = \hat{y} = \frac{1}{1 + e^{-z}}$$

8.6.1 การคำนวณสำหรับ LR

ขั้นตอนการคำนวณ

- การเทรนโมเดล

ขั้นที่ 0 : กำหนดค่าเริ่มต้น

- การกำหนด Optimizer (Solver)
 - Gradient Descent (GD)
 - LBFGS
- การกำหนด Learning Rate (η)
 - ถ้าสูงเกินไป การเรียนรู้จะแกว่ง (divergence) มาก
 - ถ้าต่ำเกินไปจะเรียนรู้จบ (convergence) ช้า
- กำหนดค่า weight เริ่มต้น $\beta_0, \beta_1, \beta_2 \dots, \beta_m$
โดยการสุ่มค่าขนาดเล็ก
- การกำหนด จำนวน epoch
 - ถ้าข้อมูลขนาดใหญ่จำนวนรอบอาจน้อยลง
 - ถ้าข้อมูลขนาดเล็กจำนวนรอบอาจมากขึ้น

ขั้นที่ 1 : คำนวณค่า Linear Combination และ Probability

- Linear Combination

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_m x_{im} = \beta^T x_i$$

$$= \beta_0 + \sum_{j=1}^m \beta_j x_{ij} = \sum_{j=0}^m \beta_j x_{ij} \text{ เมื่อ } x_{i0} = 1$$

- Probability

$$P(y_i = 1|x_i) = \hat{y}_i = \frac{1}{1 + e^{-z_i}} \text{ เมื่อ } \hat{y}_i \text{ คือ ความน่าจะเป็นที่โมเดลทำนาย ของข้อมูลตัวที่ } i$$

ขั้นที่ 2 : คำนวณ Loss (Error)

$$L(\beta) = \frac{1}{n} \sum_{i=1}^n [-y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)]$$

ขั้นที่ 3 : คำนวณ Gradient โดยใช้ Chain Rule

$$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \beta_j}$$

- คำนวณแต่ละส่วน

$$(1) \quad \frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i}$$

$$(2) \quad \frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

(3) คูณ (1) และ (2)

$$\frac{\partial L}{\partial z_i} = \left(-\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i} \right) \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - y_i$$

$$(4) \quad \frac{\partial z_i}{\partial \beta_j} = x_{ij}$$

- รวมทุกส่วนเข้าด้วยกัน

$$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

ขั้นที่ 4 : ปรับค่า Weight

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$$

t คือ รอบการปรับ weight

$t = 0$ เป็นการกำหนด weight เริ่มต้น

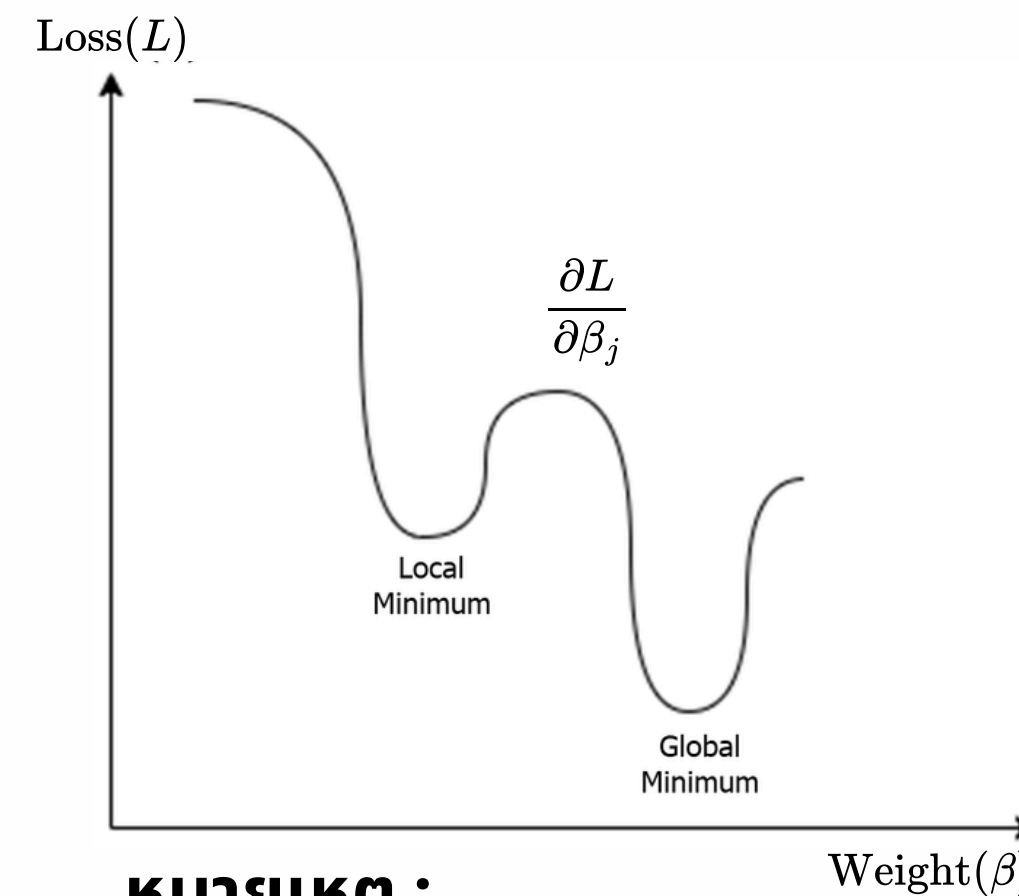
η คือ Learning Rate

ลบ gradient เพื่อลงไปหาค่า Loss ต่ำสุด

ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่าจะเข้าเงื่อนไขการหยุด

หยุดเมื่อ

- Loss ลดลงน้อยมาก หรือ
- Gradient เข้าใกล้ 0 หรือ
- ครบจำนวน Epoch ที่กำหนด



หมายเหตุ :

Loss Function = ภูเขา

Weight = ตำแหน่งบนภูเขา

Gradient = ความชัน

Gradient Descent = เดินลงเขาที่ละก้าวจนถึงจุดต่ำสุด

- **การทดสอบโมเดล**

- คำนวณ Linear Combination

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

- คำนวณ Probability

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

- ทำนายผลลัพธ์

ทำนายเป็น

class label = 1 ถ้าค่า Probability ≥ 0.5

class label = 0 ถ้าค่า Probability < 0.5

8.6.2 ตัวอย่างการคำนวณ LR

จงแสดงวิธีการคำนวณ Logistic Regression เพื่อวินิจฉัยโรคเบาหวาน โดยให้ใช้ค่า และชุดข้อมูลต่อไปนี้

- Solver = GD
- จำนวน Epoch = 2
- Learning Rate = 0.0001
- ค่าเริ่มต้นของ Weight :

$$\beta_0 = -20$$

$$\beta_1 (\text{อายุ}) = 0.05$$

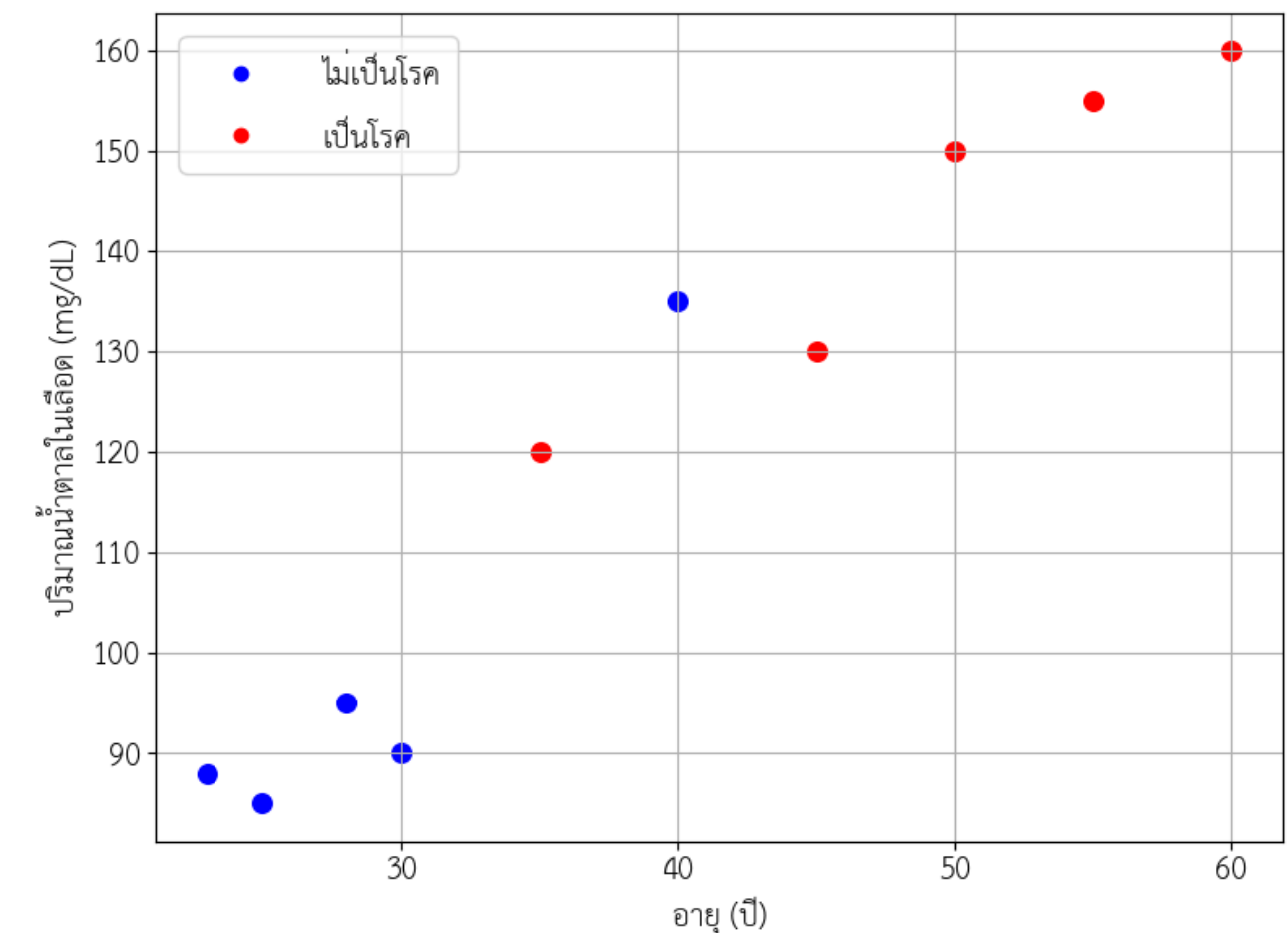
$$\beta_2 (\text{น้ำตาล}) = 0.15$$

ตารางที่ 8-7ก : ชุดข้อมูล (Training Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

ตารางที่ 8-7ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



ภาพที่ 8-17 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

- การเทรนโมเดล

ขั้นที่ 0 : กำหนดค่าเริ่มต้น

Solver = GD

จำนวน Epoch = 2

Learning Rate = 0.0001

Weight : $\beta_0 = -20$

β_1 (อายุ) = 0.05

β_2 (น้ำตาล) = 0.15

$x_1 =$ อายุ

$x_2 =$ ปริมาณน้ำตาลในเลือด

$y \in \{0, 1\}$ โดยที่ 0 = ไม่เป็นโรค, 1 = เป็นโรค

รอบที่ 1

ขั้นที่ 1+2 : คำนวณค่า Linear Combination, Probability, และ Loss

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

$$\hat{y}_i = \frac{1}{1 + e^{-z_i}}$$

$$L_i = -y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)$$

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	linear combination (z _i)	probability (y _i ^)	loss (L _i)
1	25	85	0	-20 + 0.05x25 + 0.15x85 = -6	1 / 1 + e^(-6) = 0.0025	-0ln(0.0025) - (1-0)ln(1-0.0025) = 0.0025
2	30	90	0	-20 + 0.05x30 + 0.15x90 = -5	1 / 1 + e^(-5) = 0.0067	-0ln(0.0067) - (1-0)ln(1-0.0067) = 0.0067
3	45	130	1	-20 + 0.05x45 + 0.15x130 = 1.75	1 / 1 + e^(-1.75) = 0.852	-1ln(0.852) - (1-1)ln(1-0.852) = 0.1602
4	35	120	1	-20 + 0.05x35 + 0.15x120 = -0.25	1 / 1 + e^(-0.25) = 0.4378	-1ln(0.4378) - (1-1)ln(1-0.4378) = 0.826
5	50	150	1	-20 + 0.05x50 + 0.15x150 = 5	1 / 1 + e^(-5) = 0.9933	-1ln(0.9933) - (1-1)ln(1-0.9933) = 0.0067
6	23	88	0	-20 + 0.05x23 + 0.15x88 = -5.65	1 / 1 + e^(-5.65) = 0.0035	-0ln(0.0035) - (1-0)ln(1-0.0035) = 0.0035
7	60	160	1	-20 + 0.05x60 + 0.15x160 = 7	1 / 1 + e^(-7) = 0.9991	-1ln(0.9991) - (1-1)ln(1-0.9991) = 0.0009
8	55	155	1	-20 + 0.05x55 + 0.15x155 = 6	1 / 1 + e^(-6) = 0.9975	-1ln(0.9975) - (1-1)ln(1-0.9975) = 0.0025
9	40	135	0	-20 + 0.05x40 + 0.15x135 = 2.25	1 / 1 + e^(-2.25) = 0.9047	-0ln(0.9047) - (1-0)ln(1-0.9047) = 2.3507
10	28	95	0	-20 + 0.05x28 + 0.15x95 = -4.35	1 / 1 + e^(-4.35) = 0.0127	-0ln(0.0127) - (1-0)ln(1-0.0127) = 0.0128
						SUM = 3.3725
						$L(\beta) = \frac{1}{n} \sum_{i=1}^n L_i$ L= 1/10 (3.3725) = 0.3373

ขั้นที่ 3 : คำนวณ Gradient

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	$\hat{y}_i$	$(\hat{y}_i - y_i)x_{i0}$	$(\hat{y}_i - y_i)x_{i1}$	$(\hat{y}_i - y_i)x_{i2}$
1	25	85	0	0.0025	(0.0025-0)*1=0.0025	(0.0025-0)*25=0.0625	(0.0025-0)*85=0.2125
2	30	90	0	0.0067	(0.0067-0)*1=0.0067	(0.0067-0)*30=0.201	(0.0067-0)*90=0.603
3	45	130	1	0.852	(0.852-1)*1=-0.148	(0.852-1)*45=-6.66	(0.852-1)*130=-19.24
4	35	120	1	0.4378	(0.4378-1)*1=-0.5622	(0.4378-1)*35=-19.677	(0.4378-1)*120=-67.464
5	50	150	1	0.9933	(0.9933-1)*1=-0.0067	(0.9933-1)*50=-0.335	(0.9933-1)*150=-1.005
6	23	88	0	0.0035	(0.0035-0)*1=0.0035	(0.0035-0)*23=0.0805	(0.0035-0)*88=0.308
7	60	160	1	0.9991	(0.9991-1)*1=-0.0009	(0.9991-1)*60=-0.054	(0.9991-1)*160=-0.144
8	55	155	1	0.9975	(0.9975-1)*1=-0.0025	(0.9975-1)*55=-0.1375	(0.9975-1)*155=-0.3875
9	40	135	0	0.9047	(0.9047-0)*1=0.9047	(0.9047-0)*40=36.188	(0.9047-0)*135=122.1345
10	28	95	0	0.0127	(0.0127-0)*1=0.0127	(0.0127-0)*28=0.3556	(0.0127-0)*95=1.2065

$$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)x_{ij}$$

SUM = 0.2098

SUM = 10.0241

SUM = 36.224

$\partial L/\partial B0 = 1/10(0.2098) = 0.021$
 $\partial L/\partial B0 = 1/10(10.0241) = 1.0024$
 $\partial L/\partial B0 = 1/10(36.224) = 3.6224$

ขั้นที่ 4 : ปรับค่า Weight

จาก $\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$

$$\beta_0^{(0)} = -20 \quad \beta_1^{(0)} = 0.05 \quad \beta_2^{(0)} = 0.15 \quad \eta = 0.0001$$

$$\frac{\partial L}{\partial \beta_0} = 0.021 \quad \frac{\partial L}{\partial \beta_1} = 1.0024 \quad \frac{\partial L}{\partial \beta_2} = 3.6224$$

จะได้

$$\beta_0^{(1)} = \beta_0^{(0)} - \eta \frac{\partial L}{\partial \beta_0} = -20 - 0.0001(0.021) = -20.0000$$

$$\beta_1^{(1)} = \beta_1^{(0)} - \eta \frac{\partial L}{\partial \beta_1} = 0.05 - 0.0001(1.0024) = 0.0499$$

$$\beta_2^{(1)} = \beta_2^{(0)} - \eta \frac{\partial L}{\partial \beta_2} = 0.15 - 0.0001(3.6224) = 0.1496$$

ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่าจะเข้าเงื่อนไขการหยุด

รอบที่ 2

ขั้นที่ 1+2 : คำนวณค่า Linear Combination, Probability, และ Loss

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

$$\hat{y}_i = \frac{1}{1 + e^{-z_i}}$$

$$L_i = -y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)$$

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	linear combination (z _i)	probability (y _i [^])	loss (L _i)
1	25	85	0	-20 + 0.0499x25 + 0.1496x85 = -6.0365	1 / 1 + e ^{^-(-6.0365)} = 0.0024	-0ln(0.0024) - (1-0)ln(1-0.0024) = 0.0024
2	30	90	0	-20 + 0.0499x30 + 0.1496x90 = -5.039	1 / 1 + e ^{^-(-5.039)} = 0.0064	-0ln(0.0064) - (1-0)ln(1-0.0064) = 0.0064
3	45	130	1	-20 + 0.0499x45 + 0.1496x130 = 1.6935	1 / 1 + e ^{^-(-1.6935)} = 0.8447	-1ln(0.8447) - (1-1)ln(1-0.8447) = 0.1688
4	35	120	1	-20 + 0.0499x35 + 0.1496x120 = -0.3015	1 / 1 + e ^{^-(-0.3015)} = 0.4252	-1ln(0.4252) - (1-1)ln(1-0.4252) = 0.8552
5	50	150	1	-20 + 0.0499x50 + 0.1496x150 = 4.935	1 / 1 + e ^{^-(-4.935)} = 0.9929	-1ln(0.9929) - (1-1)ln(1-0.9929) = 0.0071
6	23	88	0	-20 + 0.0499x23 + 0.1496x88 = -5.6875	1 / 1 + e ^{^-(-5.6875)} = 0.0034	-0ln(0.0034) - (1-0)ln(1-0.0034) = 0.0034
7	60	160	1	-20 + 0.0499x60 + 0.1496x160 = 6.93	1 / 1 + e ^{^-(-6.93)} = 0.999	-1ln(0.999) - (1-1)ln(1-0.999) = 0.001
8	55	155	1	-20 + 0.0499x55 + 0.1496x155 = 5.9325	1 / 1 + e ^{^-(-5.9325)} = 0.9974	-1ln(0.9974) - (1-1)ln(1-0.9974) = 0.0026
9	40	135	0	-20 + 0.0499x40 + 0.1496x135 = 2.192	1 / 1 + e ^{^-(-2.192)} = 0.8995	-0ln(0.8995) - (1-0)ln(1-0.8995) = 2.2976
10	28	95	0	-20 + 0.0499x28 + 0.1496x95 = -4.3908	1 / 1 + e ^{^-(-4.3908)} = 0.0122	-0ln(0.0122) - (1-0)ln(1-0.0122) = 0.0123
						SUM = 3.3568
						$L(\beta) = \frac{1}{n} \sum_{i=1}^n L_i$ L= 1/10 (3.3568) = 0.3357

ขั้นที่ 3 : คำนวณ Gradient

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	$\hat{y}_i$	$(\hat{y}_i - y_i)x_{i0}$	$(\hat{y}_i - y_i)x_{i1}$	$(\hat{y}_i - y_i)x_{i2}$
1	25	85	0	0.0024	(0.0024-0)*1=0.0024	(0.0024-0)*25=0.06	(0.0024-0)*85=0.204
2	30	90	0	0.0064	(0.0064-0)*1=0.0064	(0.0064-0)*30=0.192	(0.0064-0)*90=0.576
3	45	130	1	0.8447	(0.8447-1)*1=-0.1553	(0.8447-1)*45=-6.9885	(0.8447-1)*130=-20.189
4	35	120	1	0.4252	(0.4252-1)*1=-0.5748	(0.4252-1)*35=-20.118	(0.4252-1)*120=-68.976
5	50	150	1	0.9929	(0.9929-1)*1=-0.0071	(0.9929-1)*50=-0.355	(0.9929-1)*150=-1.065
6	23	88	0	0.0034	(0.0034-0)*1=0.0034	(0.0034-0)*23=0.0782	(0.0034-0)*88=0.2992
7	60	160	1	0.999	(0.999-1)*1=-0.001	(0.999-1)*60=-0.06	(0.999-1)*160=-0.16
8	55	155	1	0.9974	(0.9974-1)*1=-0.0026	(0.9974-1)*55=-0.143	(0.9974-1)*155=-0.403
9	40	135	0	0.8995	(0.8995-0)*1=0.8995	(0.8995-0)*40=35.98	(0.8995-0)*135=121.4325
10	28	95	0	0.0122	(0.0122-0)*1=0.0122	(0.0122-0)*28=0.3416	(0.0122-0)*95=1.159
					SUM = 0.1831	SUM = 8.9873	SUM = 32.8777
$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)x_{ij}$					$\frac{\partial L}{\partial \beta_0} = 1/10(0.1831) = 0.0183$	$\frac{\partial L}{\partial \beta_1} = 1/10(8.9873) = 0.8987$	$\frac{\partial L}{\partial \beta_2} = 1/10(32.8777) = 3.2878$

ขั้นที่ 4 : ปรับค่า Weight

จาก $\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$

$$\beta_0^{(1)} = -20 \quad \beta_1^{(1)} = 0.0499 \quad \beta_2^{(1)} = 0.1496 \quad \eta = 0.0001$$

$$\frac{\partial L}{\partial \beta_0} = 0.0183 \quad \frac{\partial L}{\partial \beta_1} = 0.8987 \quad \frac{\partial L}{\partial \beta_2} = 3.2878$$

จะได้

$$\beta_0^{(2)} = \beta_0^{(1)} - \eta \frac{\partial L}{\partial \beta_0} = -20 - 0.0001(0.0183) = -20.0000$$

$$\beta_1^{(2)} = \beta_1^{(1)} - \eta \frac{\partial L}{\partial \beta_1} = 0.0499 - 0.0001(0.8987) = 0.0498$$

$$\beta_2^{(2)} = \beta_2^{(1)} - \eta \frac{\partial L}{\partial \beta_2} = 0.1496 - 0.0001(3.2878) = 0.1493$$

ขั้นที่ 5 : หยุดการเทรนโมเดล เนื่องจากครบจำนวนรอบ (Epoch)

นำ Weight ที่คำนวณได้ในรอบสุดท้ายไปใช้ในการทดสอบ (ใช้งาน) โมเดล

- การทดสอบโมเดล

จาก

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \quad \beta_0 = -20.0000 \quad \beta_1 = 0.0498 \quad \beta_2 = 0.1493$$

$$x_1 = 38 \quad x_2 = 125$$

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

จะได้

$$z = -20 + 0.0498(38) + 0.1493(125) = 0.5549$$

$$\hat{y} = \frac{1}{1 + e^{-0.5549}} = 0.6353$$

ตอบ ค่า Probability = 0.6353 ซึ่ง ≥ 0.5 ทำให้ class label = 1
สรุปได้ว่า ผู้ที่เข้ารับการตรวจ **เป็นโรคเบาหวาน**

8.6.3 ตัวอย่างโค้ด LR

- เขียนโค้ดแบบไม่ใช้ไลบรารี ML

```
1  #Import Python library
2  import numpy as np
3
4  #Load Data
5  df = pd.read_csv('data/diabetes.csv')
6  x = df[['age', 'blood_glucose']].to_numpy()
7  y = df['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
8  x_test = np.array([38, 125])
9
10 # Parameters
11 n = len(y)
12 lr = 0.0001
13 epochs = 2
14
15 b0 = -20.0
16 b1 = 0.05
17 b2 = 0.15
18
19 # Sigmoid Function
20 def sigmoid(z):
21     return 1 / (1 + np.exp(-z))
22
```



```

23 # การเทรนโมเดล
24 for epoch in range(epochs):
25     # 1. คำนวณค่า Linear Combination และ Probability
26     z = (b0 + b1 * x[:, 0] + b2 * x[:, 1]).round(4)
27     y_hat = sigmoid(z).round(4)
28
29     # 2. คำนวณ Loss (Error)
30     loss = np.mean(-y * np.log(y_hat) - (1 - y) * np.log(1 - y_hat))
31
32
33     # 3. คำนวณ Gradients
34     db0 = np.mean(y_hat - y)
35     db1 = np.mean((y_hat - y) * x[:, 0])
36     db2 = np.mean((y_hat - y) * x[:, 1])
37
38     # 4. ปรับค่า Weight
39     b0 -= (lr * db0).round(4)
40     b1 -= (lr * db1).round(4)
41     b2 -= (lr * db2).round(4)
42
43     print(f"Epoch {epoch+1}")
44     print(f"Loss = {loss:.4f}")
45     print(f"b0 = {b0:.4f}, b1 = {b1:.4f}, b2 = {b2:.4f}\n")
46

```



```
47 # การทดสอบโมเดล
48 z_test = b0 + b1 * x_test[0] + b2 * x_test[1]
49 prob = sigmoid(z_test)
50 y_pred = 1 if prob >= 0.5 else 0
51
52 print("=== Test Result ===")
53 print(f"Probability {prob:.4f}")
54 print(f"Predicted Class = {y_pred}")
```

Epoch 1

Loss = 0.3373

b0 = -20.0000, b1 = 0.0499, b2 = 0.1496

Epoch 2

Loss = 0.3357

b0 = -20.0000, b1 = 0.0498, b2 = 0.1493

=== Test Result ===

Probability = 0.6353

Predicted Class = 1

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

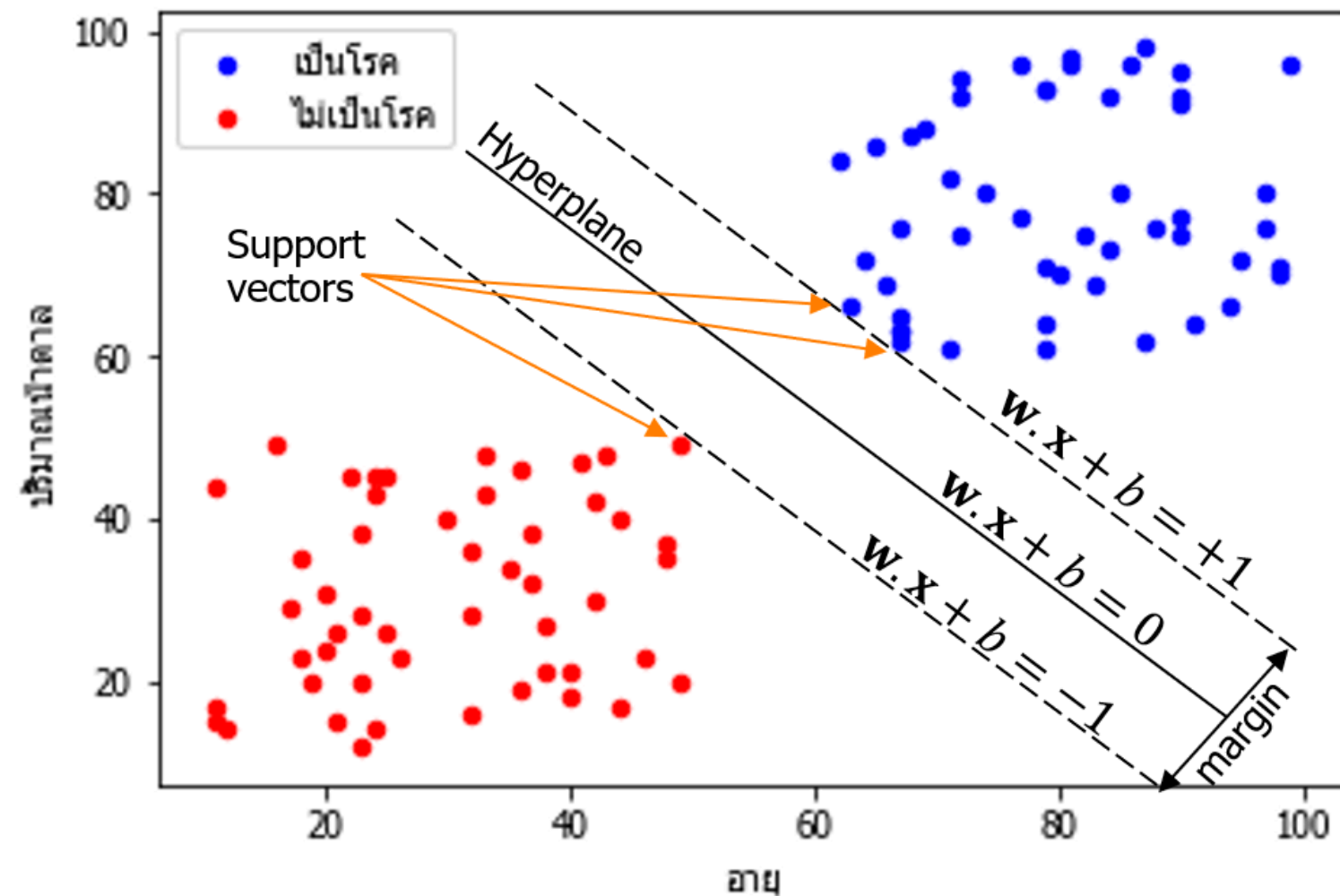
```
1 # Import Python library
2 import pandas as pd
3 from sklearn.linear_model import LogisticRegression
4
5 # Load Data
6 x = pd.read_csv('data/diabetes.csv')
7 X_train = x[['age', 'blood_glucose']].to_numpy()
8 y_train = x['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
9 X_test = [[38, 125]]
10
11 # Build and Train Model
12 model = LogisticRegression()
13 model.fit(X_train, y_train)
14
15 # Test Model
16 y_prob = model.predict_proba(X_test)
17 y_pred = model.predict(X_test)
18
19 print("=== Test Result ===")
20 print(f"Probability = {y_prob[0][1]:.4f}")
21 print(f"Predicted Class = {y_pred[0]}")
```

Probability = 0.5030

Predicted Class = 1

8.7 Support Vector Machine (SVM)

- อาศัยแนวคิดของ Margin Maximization ในการตัดสินใจ โดยพยายามแบ่งกลุ่มข้อมูลด้วย Optimal Hyperplane ที่สร้างระยะห่างจากกลุ่มข้อมูลให้มากที่สุด ซึ่งจุดที่อยู่ชิดขอบการแบ่งนี้เรียกว่า Support Vectors
- จัดการกับข้อมูลที่ไม่เป็นเชิงเส้นผ่าน Kernel Function
- รองรับทั้งการทำงานแบบ Binary และ Multi-class Classification



ภาพที่ 8-18 : ตัวอย่างการจำแนกกลุ่มข้อมูลด้วย SVM

8.7.1 การคำนวณสำหรับ SVM

ขั้นตอนการคำนวณ

- การเทรนโมเดล

- กำหนดค่าเริ่มต้น

$$x \in \mathbb{R}^d \quad y \in \{-1, +1\}$$

- คำนวณ Kernel Function

$$\phi(x_i)^T \phi(x_j) = K(x_i, x_j) \text{ เมื่อกำหนด Kernel Function เป็น Linear จะได้ } K(x_i, x_j) = x_i^T x_j$$

- คำนวณหา Alpha

$$\max_{\alpha} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$\text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C \quad \text{โดยปกติ } C \text{ จะมีค่าเท่ากับ } 1$$

- คำนวณ Weight (ใช้เฉพาะ Linear)

$$w = \sum_{i=1}^n \alpha_i y_i \phi(x_i)$$

- คำนวณ Bias

เลือก x_k ที่เป็น Support Vector มา 1 ตัว ที่มีค่า $\alpha_i > 0$ เพื่อมาคำนวณหา Bias

$$b = y_k - \sum_{i=1}^n \alpha_i y_i K(x_i, x_k)$$

subject to $y_k(w^T \phi(x_k) + b) = 1$

- **การทดสอบโมเดล**

ใช้ Decision Function ในการตัดสินใจ (ทำนาย)

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b \quad \text{เมื่อใช้ Linear เป็น Kernel Function จะได้} \quad f(x) = w^T x + b$$

$$\hat{y} = \text{sign}(f(x))$$

ถ้าค่า $f(x) > 0$ ให้ค่าเป็น 1

ถ้าค่า $f(x) < 0$ ให้ค่าเป็น -1

8.7.2 ตัวอย่างโค้ด SVM

```
1 # Import Python library
2 import pandas as pd
3 from sklearn.svm import LinearSVC
4
5 # Load Data
6 x = pd.read_csv('data/diabetes.csv')
7 X_train = x[['age', 'blood_glucose']].to_numpy()
8 y_train = x['diabetes'].map({'ไม่เป็นโรค': -1, 'เป็นโรค': 1})
9 X_test = [[38, 125]]
10
11 # Build and Train Model
12 model = LinearSVC(C=1.0)
13 model.fit(X_train, y_train)
14
15 # Test Model
16 w = model.coef_[0]
17 b = model.intercept_[0]
18 y_pred = model.predict(X_test)
19
20 print("=== Test Result ===")
21 print(f"w = {w.round(4)}, b = {b:.4f}")
22 print(f"Predicted Class = {y_pred[0]}")

```

=== Test Result ===
w = [0.1527 -0.04], b = -0.9955
Predicted Class = -1

8.8 แบบฝึกหัด

ตารางที่ 8-8 : ชุดข้อมูลผู้ป่วยโรคเบาหวาน (สำหรับข้อ 1-3)

ตารางที่ 8-8ก : ชุดข้อมูลฝึก (Training Set)

รายการ	ดัชนีมวลกาย (BMI)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	20	90	ไม่เป็นโรค
2	22	100	ไม่เป็นโรค
3	21	88	ไม่เป็นโรค
4	23	85	ไม่เป็นโรค
5	28	135	เป็นโรค
6	30	137	เป็นโรค
7	32	140	เป็นโรค
8	29	123	เป็นโรค
9	31	145	เป็นโรค
10	27	125	เป็นโรค

ตารางที่ 8-8ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ดัชนีมวลกาย (BMI)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	100	?

- 1.จงแสดงวิธีการคำนวณ KNN เพื่อวินิจฉัยโรคเบาหวาน **กำหนดให้** $k=3$
- 2.จงแสดงวิธีการคำนวณ Naïve Bays เพื่อวินิจฉัยโรคเบาหวาน
- 3.จงแสดงวิธีการคำนวณ MLP เพื่อวินิจฉัยโรคเบาหวาน

กำหนดให้

- Input Node = 2
- Hidden Node = 3 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Hidden Weights (Input → Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.062, 0.259, 0.13 \\ 0.06, -0.147, 0.185 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.079, 0.495, 0.586]$
- Output Weight (Hidden → Output): $\mathbf{W}_y = \begin{bmatrix} -0.089 \\ 0.411 \\ 0.116 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [0.172]$

ตารางที่ 8-9 : ชุดข้อมูลอัตราการลาออกจากงาน (สำหรับข้อ 4-7)

ตารางที่ 8-9ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ	เพศ	ระดับการศึกษา	ลาออก
1	21-30	หญิง	ปริญญาโท	ใช่
2	21-30	ชาย	ปริญญาโท	ไม่ใช่
3	31-40	ชาย	ปริญญาตรี	ใช่
4	21-30	หญิง	ปริญญาเอก	ไม่ใช่
5	41-50	หญิง	ปริญญาโท	ใช่
6	31-40	หญิง	ปริญญาโท	ไม่ใช่
7	21-30	หญิง	ปริญญาโท	ไม่ใช่
8	41-50	หญิง	ปริญญาเอก	ไม่ใช่
9	21-30	ชาย	ปริญญาตรี	ไม่ใช่
10	31-40	ชาย	ปริญญาเอก	ใช่

ตารางที่ 8-9ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ	เพศ	ระดับการศึกษา	ลาออก
1	21-30	ชาย	ปริญญาตรี	?

4. จงแสดงวิธีการคำนวณ KNN เพื่อวิเคราะห์การลาออกจากงาน **กำหนดให้** $k=3$

5. จงแสดงวิธีการคำนวณ Naïve Bays เพื่อวิเคราะห์การลาออกจากงาน

6. จงแสดงวิธีการคำนวณ ID3 เพื่อวิเคราะห์การลาออกจากงาน

7. จงแสดงวิธีการคำนวณ LR เพื่อวิเคราะห์การลาออกจากงาน

กำหนดให้

- Solver = GD
- จำนวน Epoch = 2
- Learning Rate = 0.0001
- ค่าเริ่มต้นของ Weight :

$$\beta_0 = -20$$

$$\beta_1 (\text{อายุ}) = 0.05$$

$$\beta_2 (\text{น้ำตาล}) = 0.15$$

ตารางที่ 8-10 : ชุดข้อมูลราคาหุ้น (สำหรับข้อ 8)

date	open	high	low	close	change
1/8/2568	100	102	98	99	ลง
4/8/2568	99	100	95	96	ลง
5/8/2568	96	98	95	97	ขึ้น
6/8/2568	97	97	92	93	ลง
7/8/2568	93	95	91	92	ลง
8/8/2568	92	94	90	94	ขึ้น
11/8/2568	94	94	88	89	ลง
12/8/2568	89	90	86	87	ลง
13/8/2568	87	88	85	88	ขึ้น
14/8/2568	88	88	82	84	ลง

8. จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายแนวโน้มราคาหุ้น (สูงขึ้น/ต่ำลง) ณ วันที่ 15/8/2568

กำหนดให้

- Input Node = 4,
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Time Step = 2
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [0.01, -0.01, 0.02, -0.02]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.5$
- Hidden Bias: $\mathbf{b}_h = -0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 2$
- Output Bias: $\mathbf{b}_y = -0.5$



THANK YOU FOR YOUR ATTENTION

 pichai_jo@rmutto.ac.th

